

第4章

处理器

付俊宁 吕卫 译

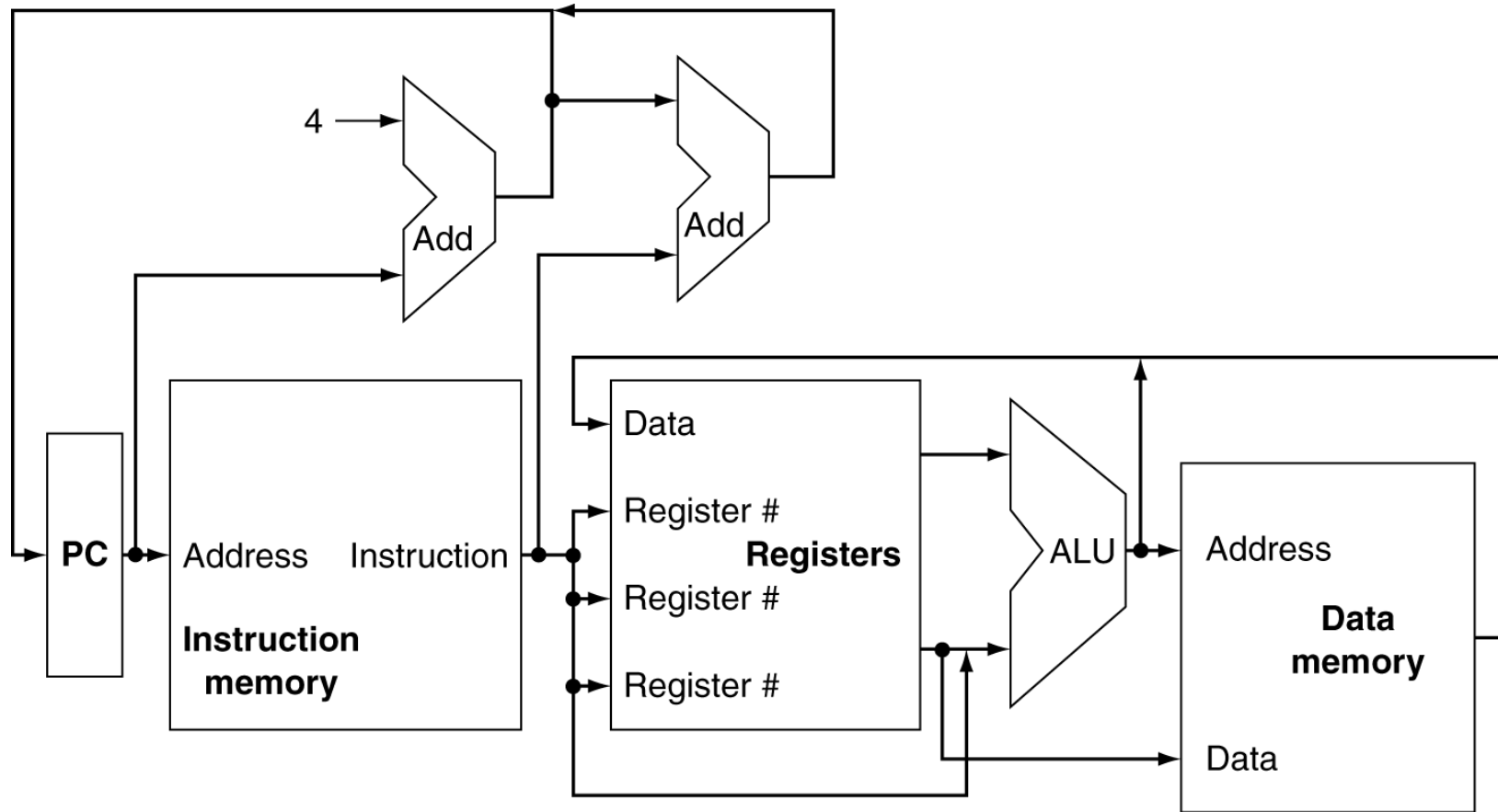
引言

- CPU性能因素
 - 指令数目
 - 取决于ISA和编译器
 - CPI 和时钟周期长度
 - 取决于CPU硬件
- 我们将要考察两个RISC-V的实现
 - 一个是简化版本
 - 一个是更实际的流水线版本
- 用ISA的简单子集展现CPU设计的多数方面
 - 存储器访问: ld, sd
 - 算术/逻辑运算: add, sub, and, or
 - 控制转移: beq

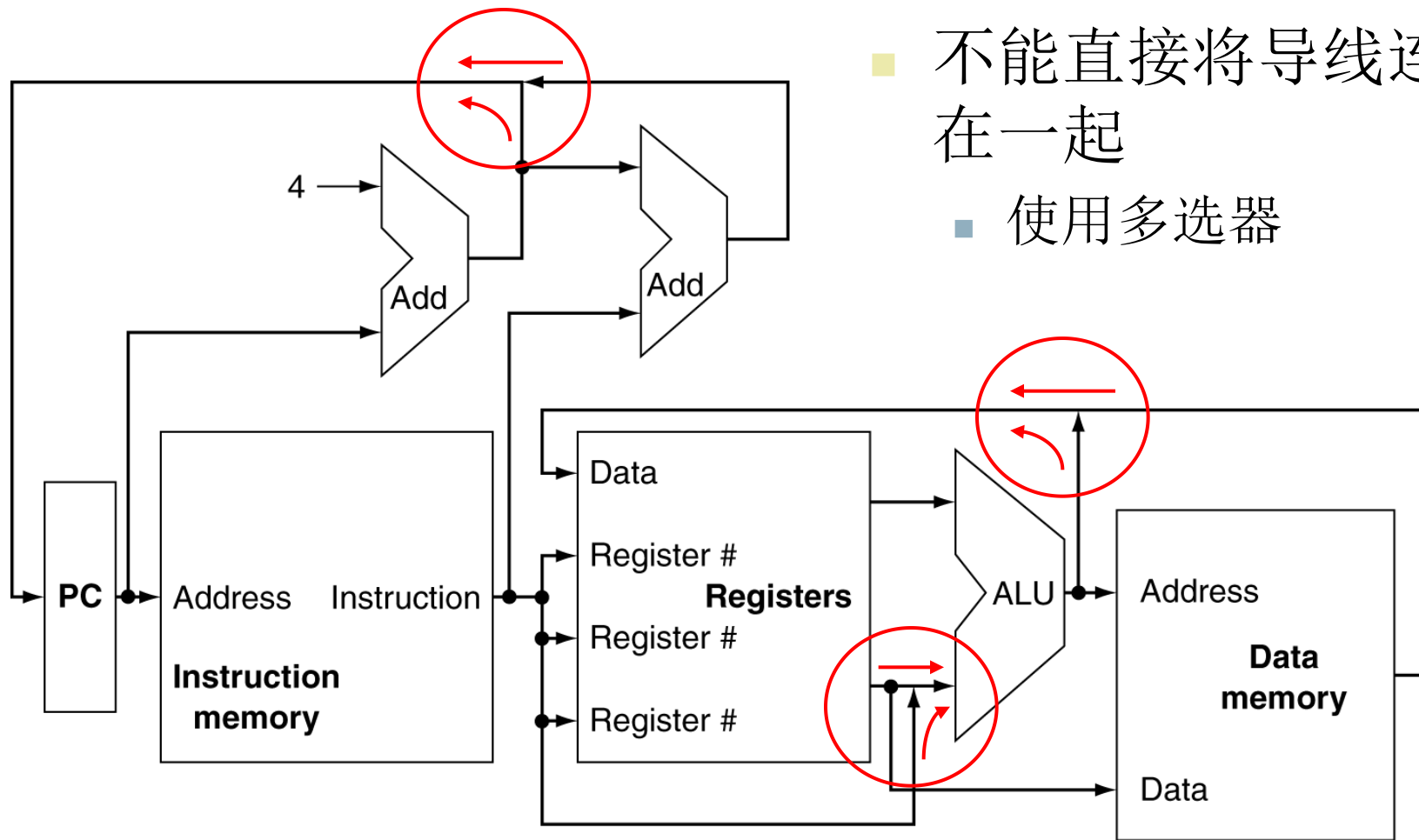
指令的执行

- $PC \rightarrow$ 指令存储器，取得指令
- 寄存器号 $\rightarrow$ 寄存器文件，读取寄存器
- 根据指令类型
 - 用ALU计算
 - 算术运算结果
 - 用于取数/存数的存储器地址
 - 分支比较
 - 访问数据存储器以取数/存数
 - $PC \leftarrow$ 目标地址或 $PC + 4$

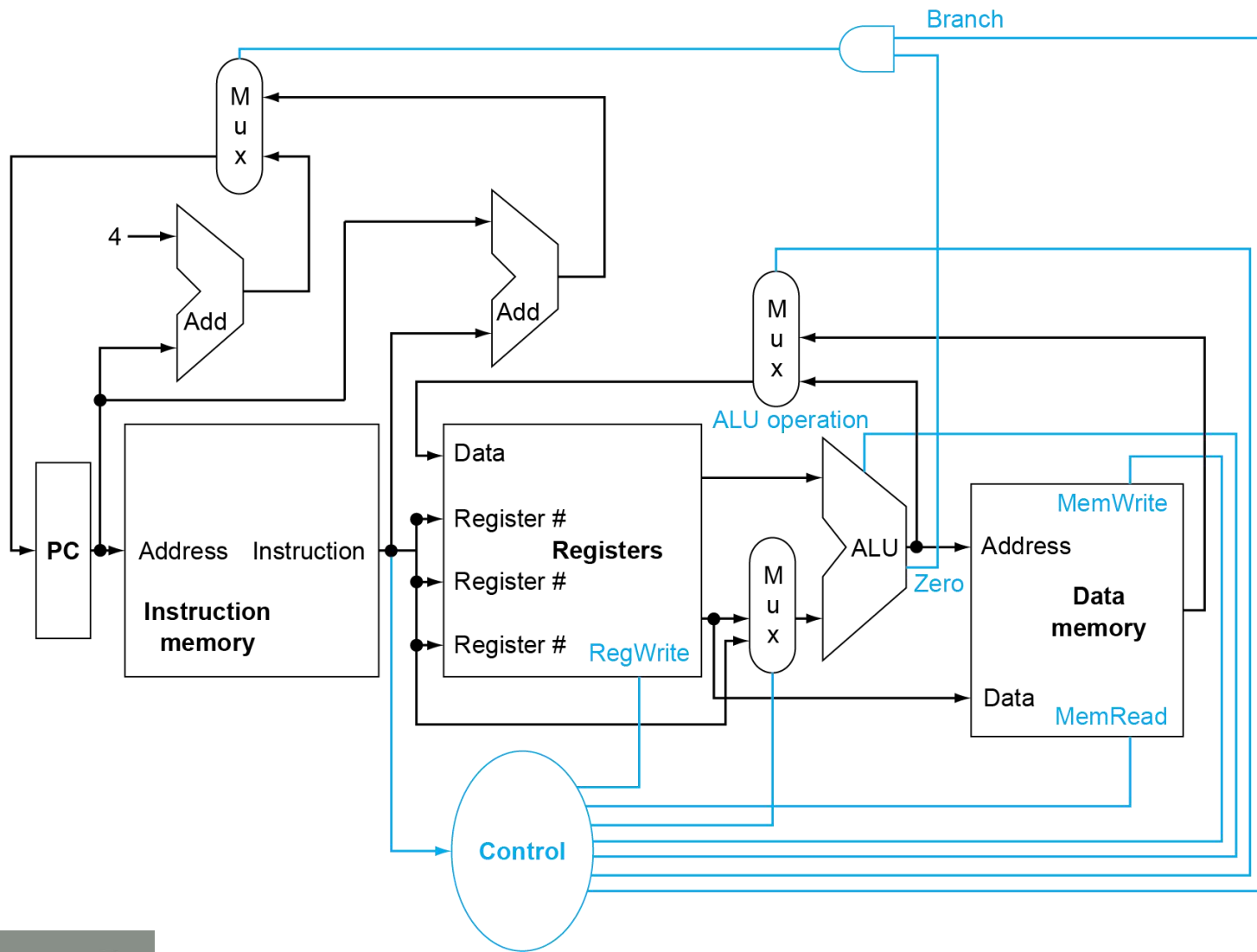
CPU概览



多路选择器



控制



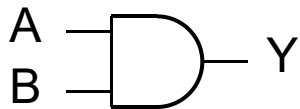
逻辑设计基础

- 信息以二进制编码
 - 低电平 = 0, 高电平 = 1
 - 每条导线占一位
 - 多线构成的总线用多位数据编码
- 组合单元
 - 对数据进行操作
 - 输出是输入的函数
- 状态（时序）单元
 - 存储信息

组合单元

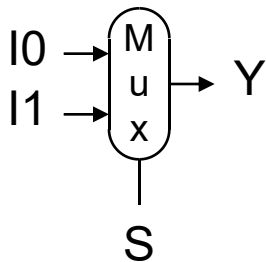
■ 与门

- $Y = A \& B$



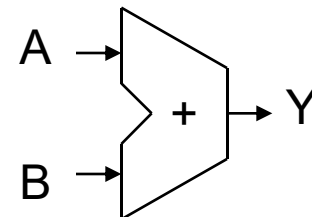
■ 复用器(多选器)

- $Y = S ? I1 : I0$



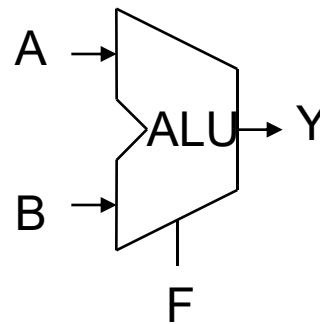
■ 加法器

- $Y = A + B$



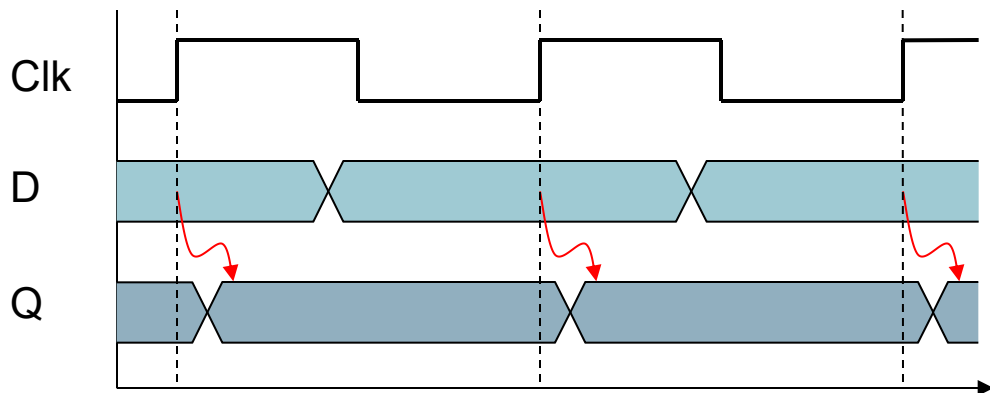
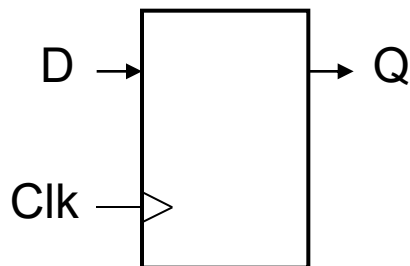
■ 算术/逻辑单元

- $Y = F(A, B)$



时序单元

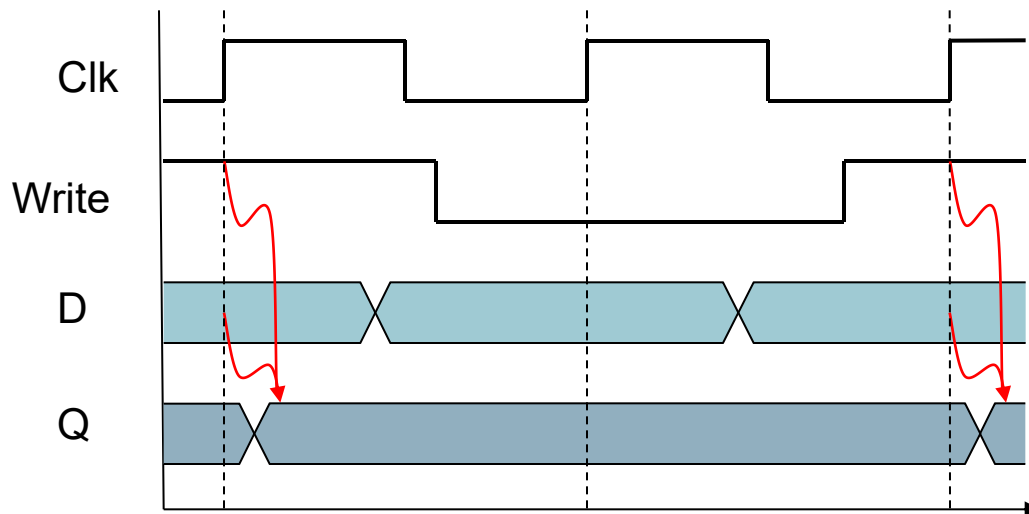
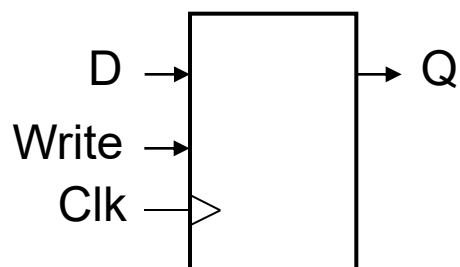
- 寄存器：在电路中保存数据
 - 用时钟信号来决定何时更新存储的值
 - 边沿触发：当时钟由0变为1时更新



时序单元

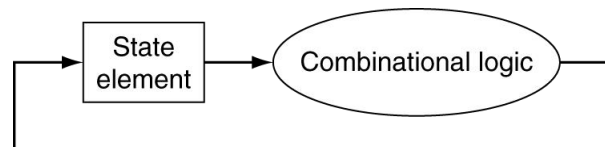
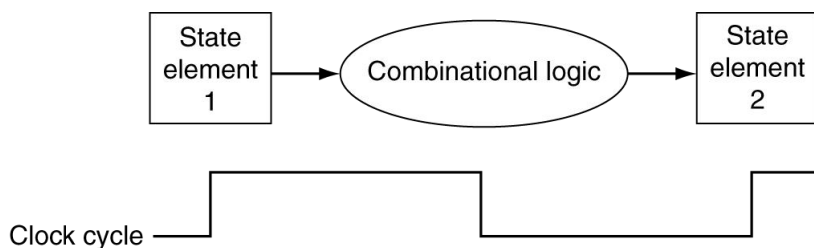
■ 带写控制的寄存器

- 只有当写控制输入为1时在时钟边沿更新
- 适用于所存储的值将在后来使用的情况



时钟方法

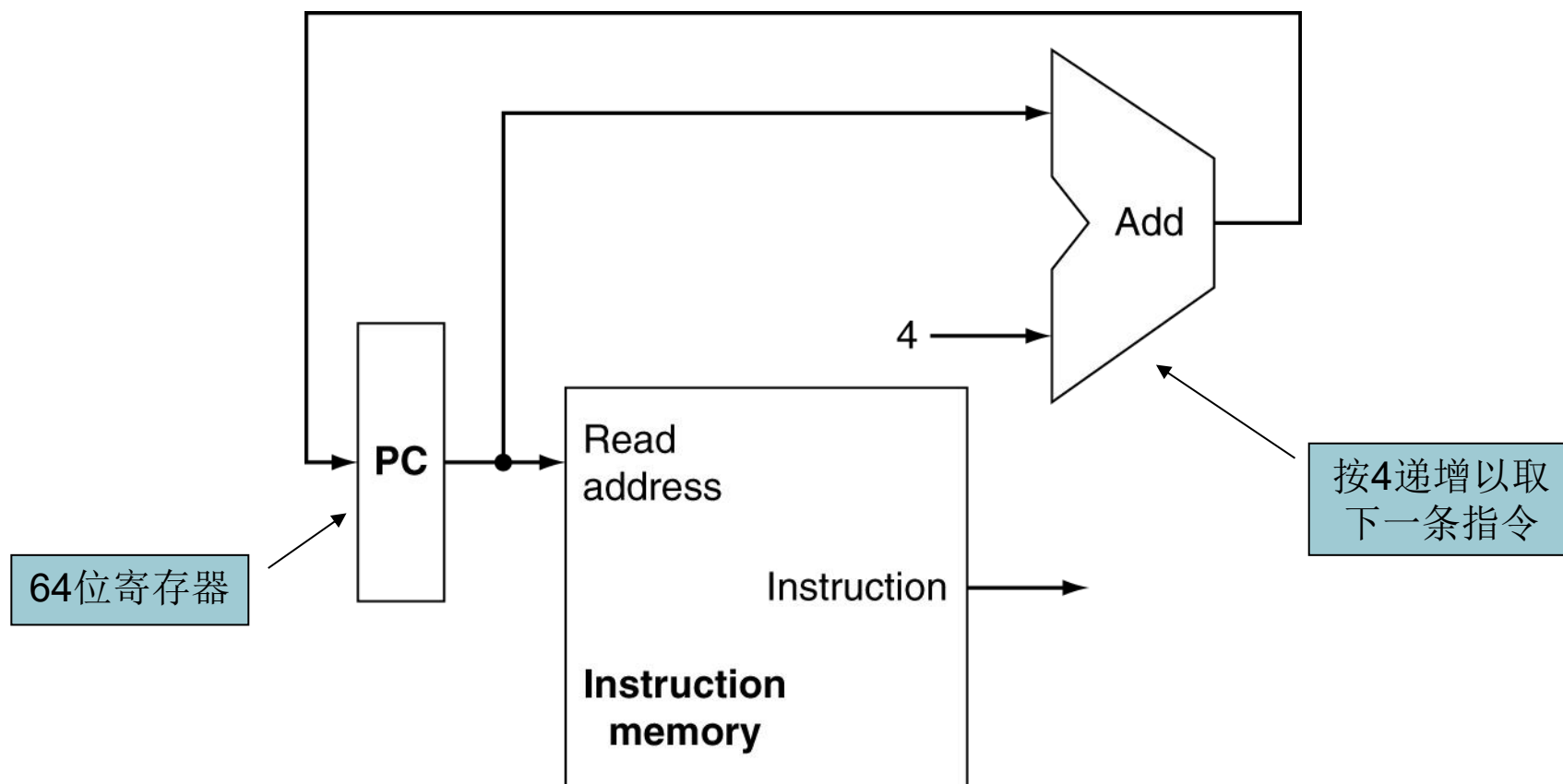
- 组合逻辑在时钟周期中进行数据变换
 - 在时钟边沿之间
 - 从状态单元输入，输出到状态单元
 - 最长延迟决定了时钟周期



建立数据通路

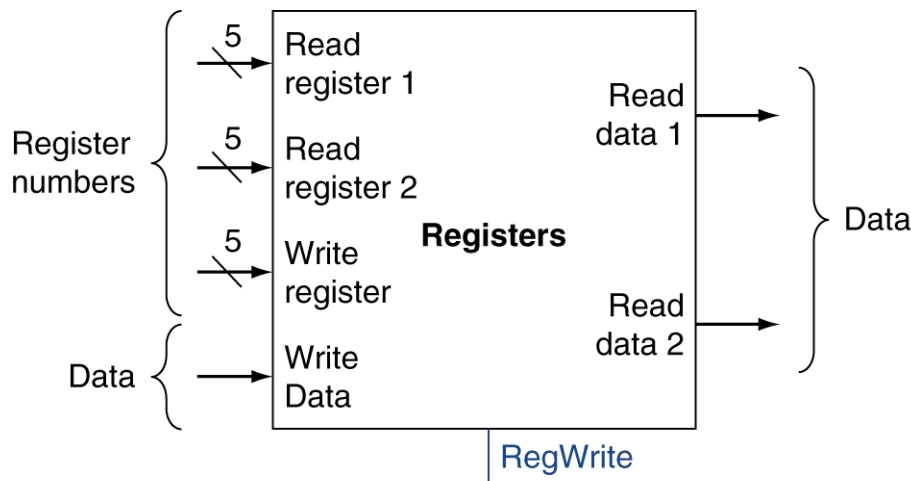
- 数据通路
 - CPU中处理数据和地址的单元
 - 寄存器、ALU、多路选择器、存储器等等
- 我们将逐步建立起一条RISC-V数据通路
 - 完善整体设计

取指

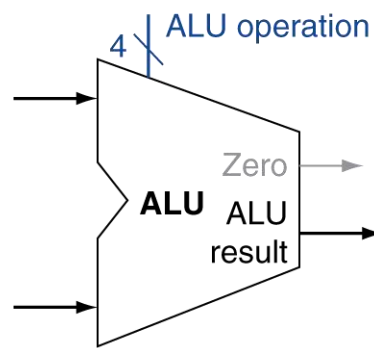


R型指令

- 读两个寄存器操作数
- 执行算术/逻辑操作
- 写寄存器结果



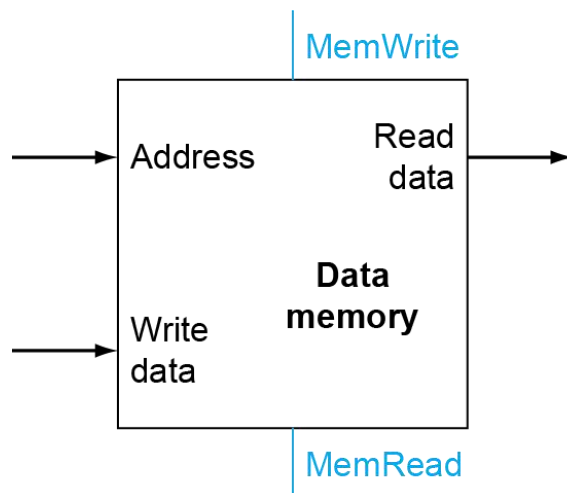
a. Registers



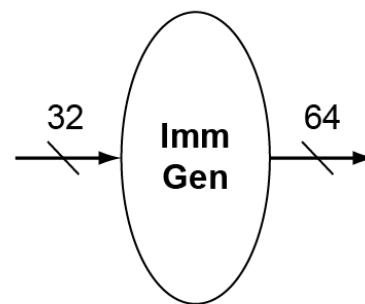
b. ALU

取数/存数指令

- 读寄存器操作数
- 用12位偏移量计算地址
 - 使用ALU，但对偏移量进行符号扩展
- 取数：读存储器并更新寄存器
- 存数：将寄存器值写入存储器



a. Data memory unit

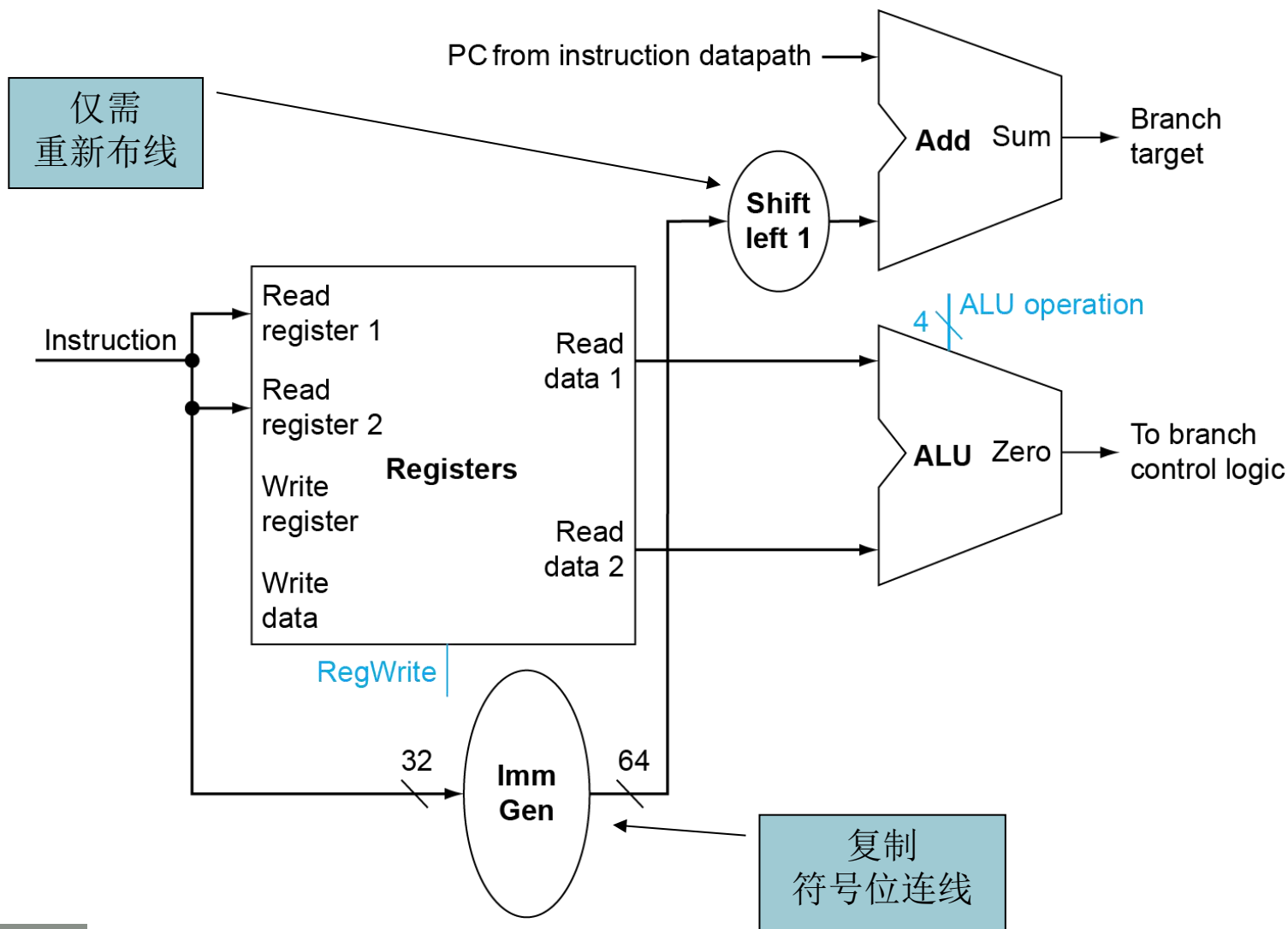


b. Immediate generation unit

分支指令

- 读寄存器操作数
- 比较操作数
 - 用**ALU**做减法并检测零标志位输出
- 计算目标地址
 - 符号扩展位移量
 - 左移**1**位（以半字为单位的位移量）
 - 与**PC**值相加

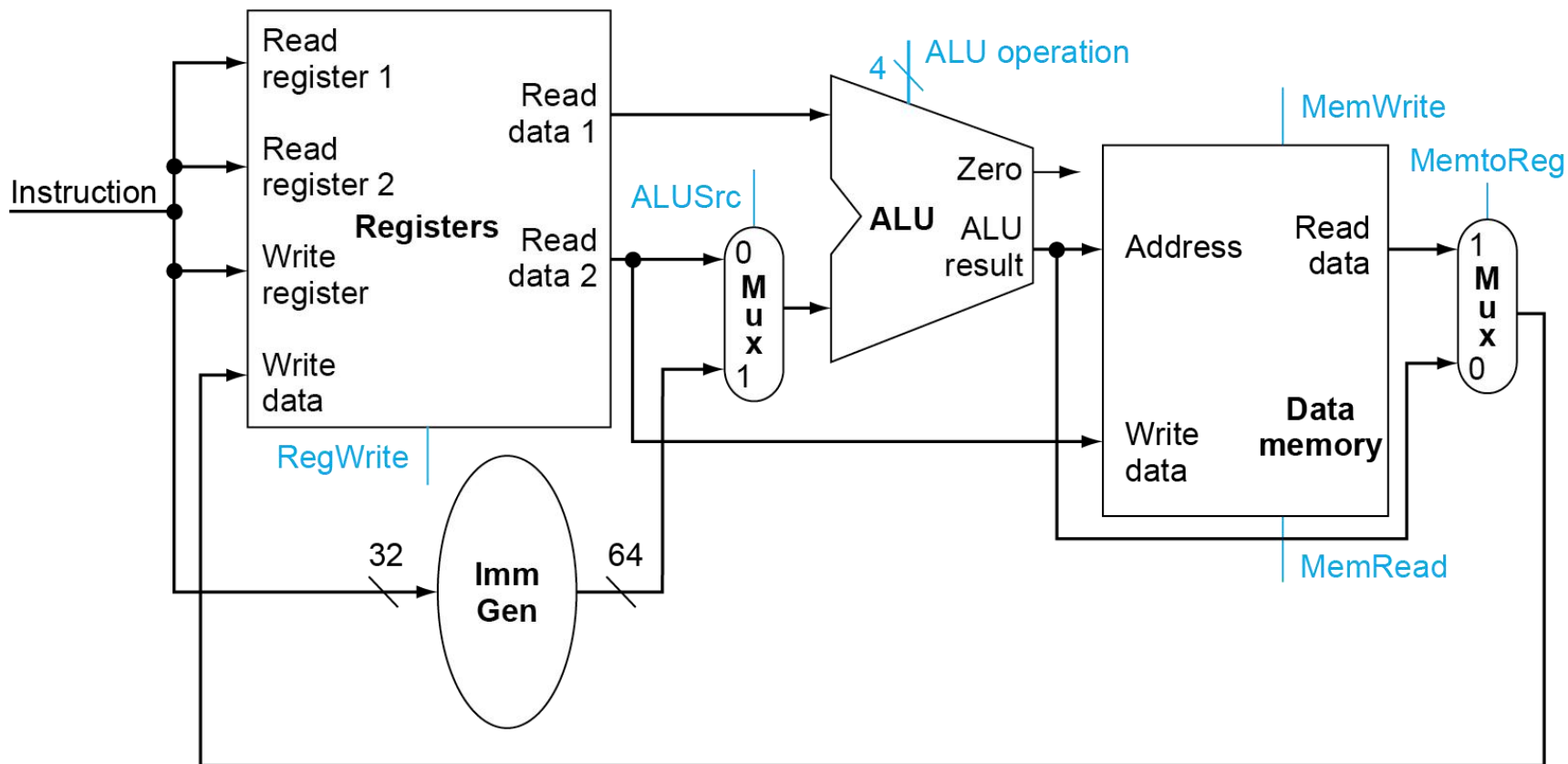
分支指令



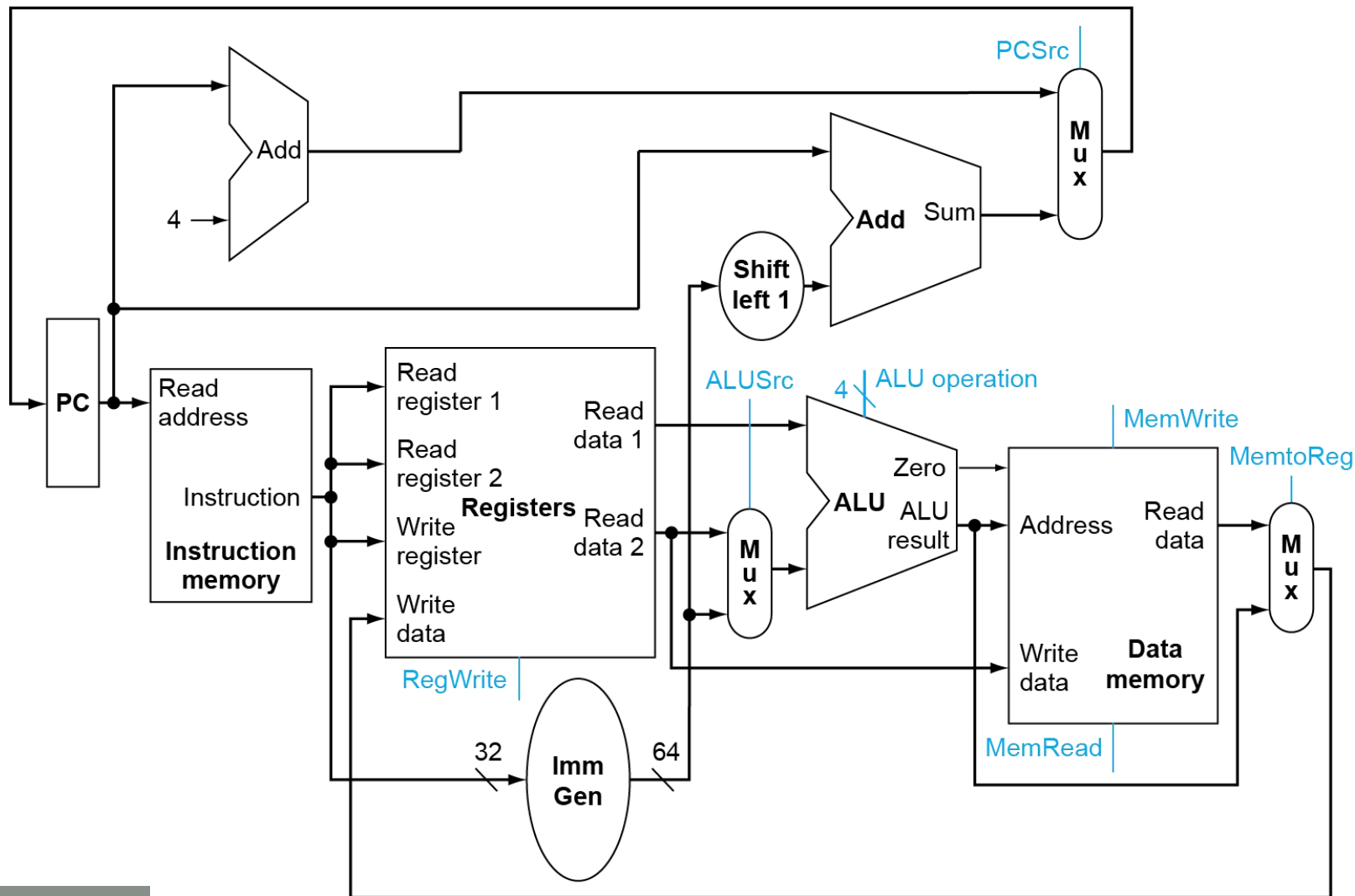
组合各单元

- 首先实现在一个时钟周期内执行一条指令的数据通路
 - 数据通路的每个单元同一时间只具有一种功能
 - 因此我们需要把指令存储器和数据存储器分开
- 在不同指令使用不同数据源的地方使用多路选择器

R型/取数/存数的数据通路



完整的数据通路



ALU控制

- ALU用于
 - 取数/存数：功能为加
 - 分支：功能为减
 - R型：功能取决于opcode字段

ALU控制	功能
0000	与
0001	或
0010	加
0110	减

ALU控制

- 假定从opcode得到2位的ALUOp
 - 组合逻辑得出ALU control码

opcode	ALUOp	Operation	Opcode field	ALU function	ALU control
ld	00	load register	XXXXXXXXXXXX	加	0010
sd	00	store register	XXXXXXXXXXXX	加	0010
beq	01	branch on equal	XXXXXXXXXXXX	减	0110
R-type	10	add	100000	加	0010
		subtract	100010	减	0110
		AND	100100	与	0000
		OR	100101	或	0001

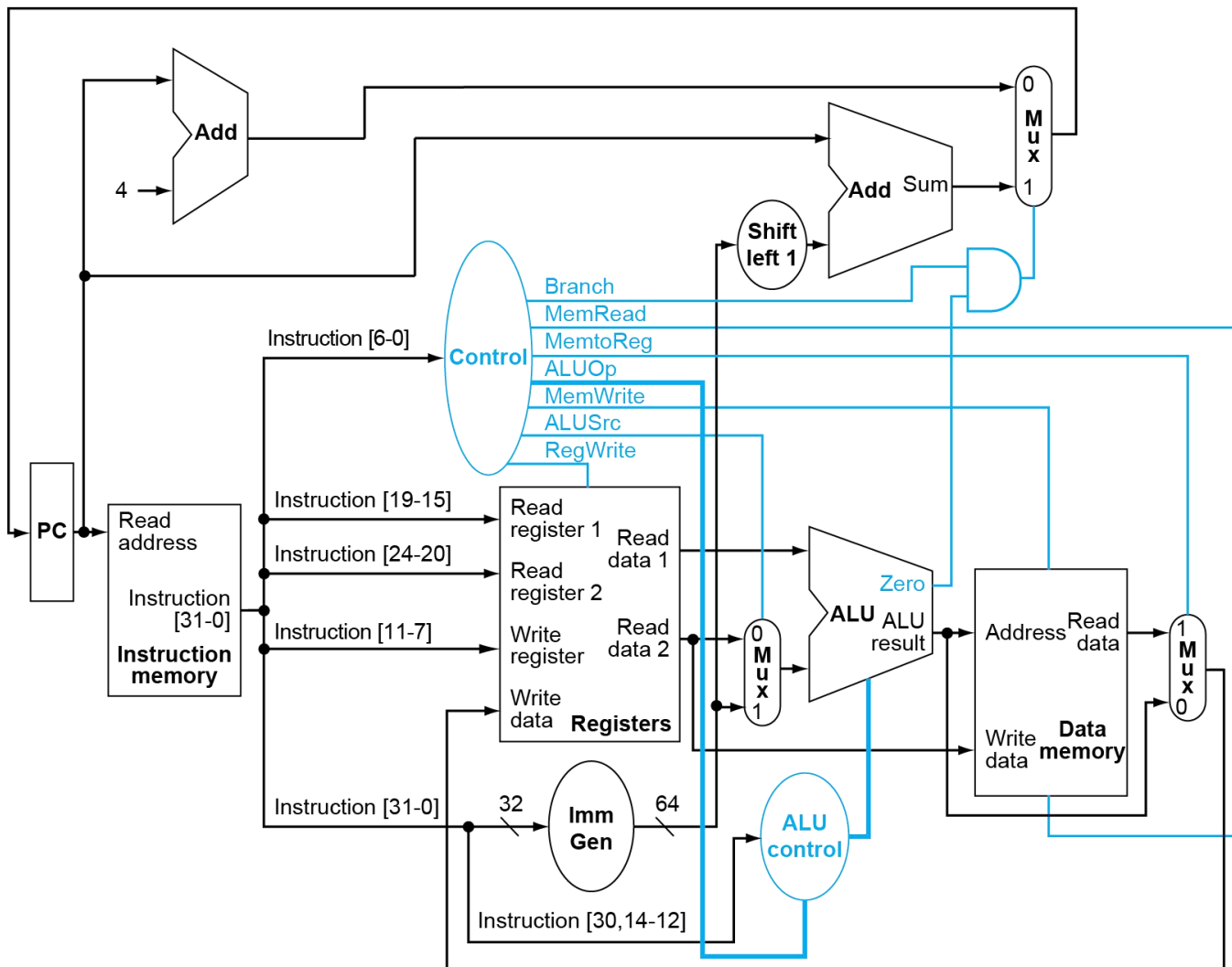
主控制单元

■ 从指令得到控制信号

Name (Bit position)	Fields					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

ALUOp		Funct7 field							Funct3 field			Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

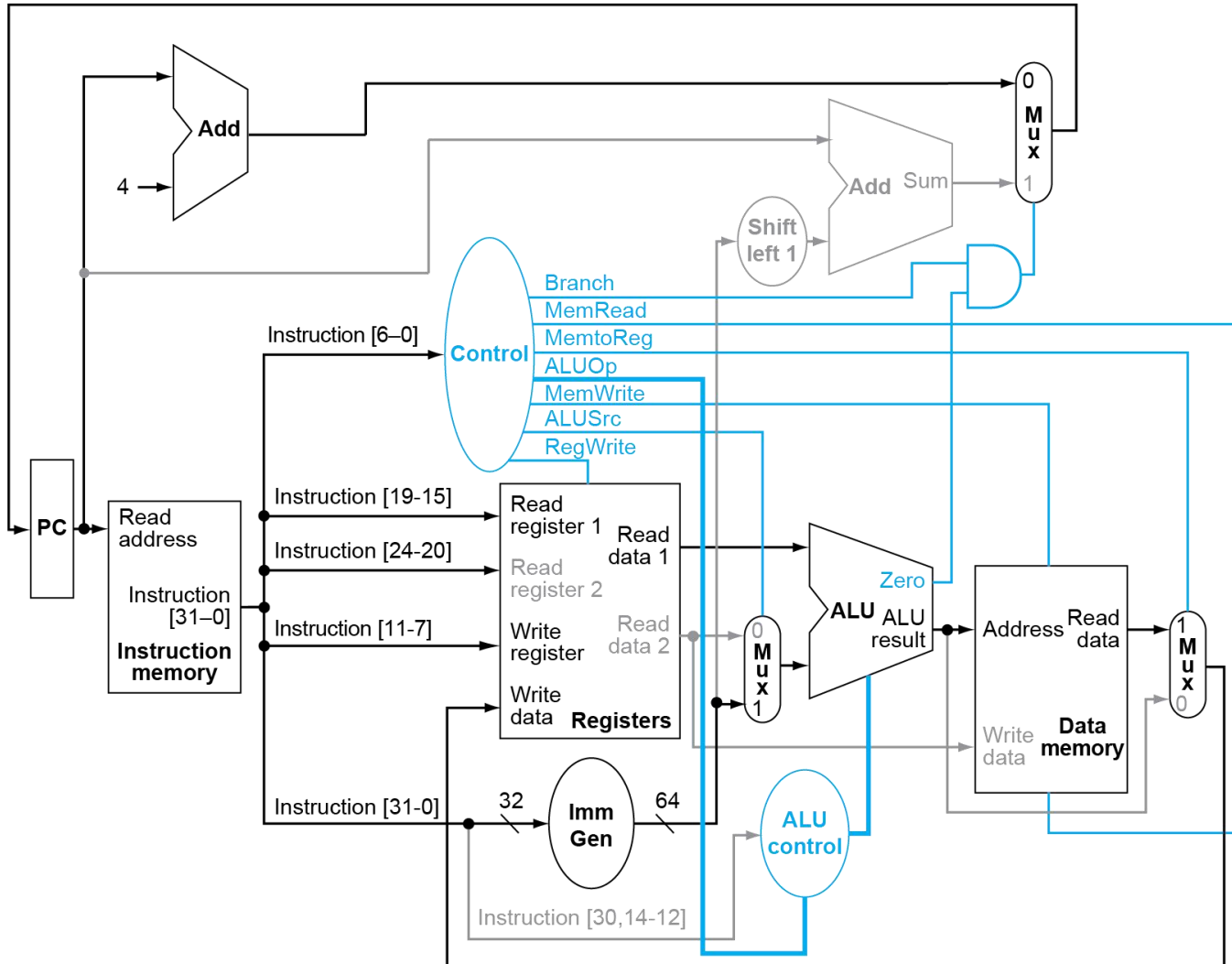
包含控制的数据通路



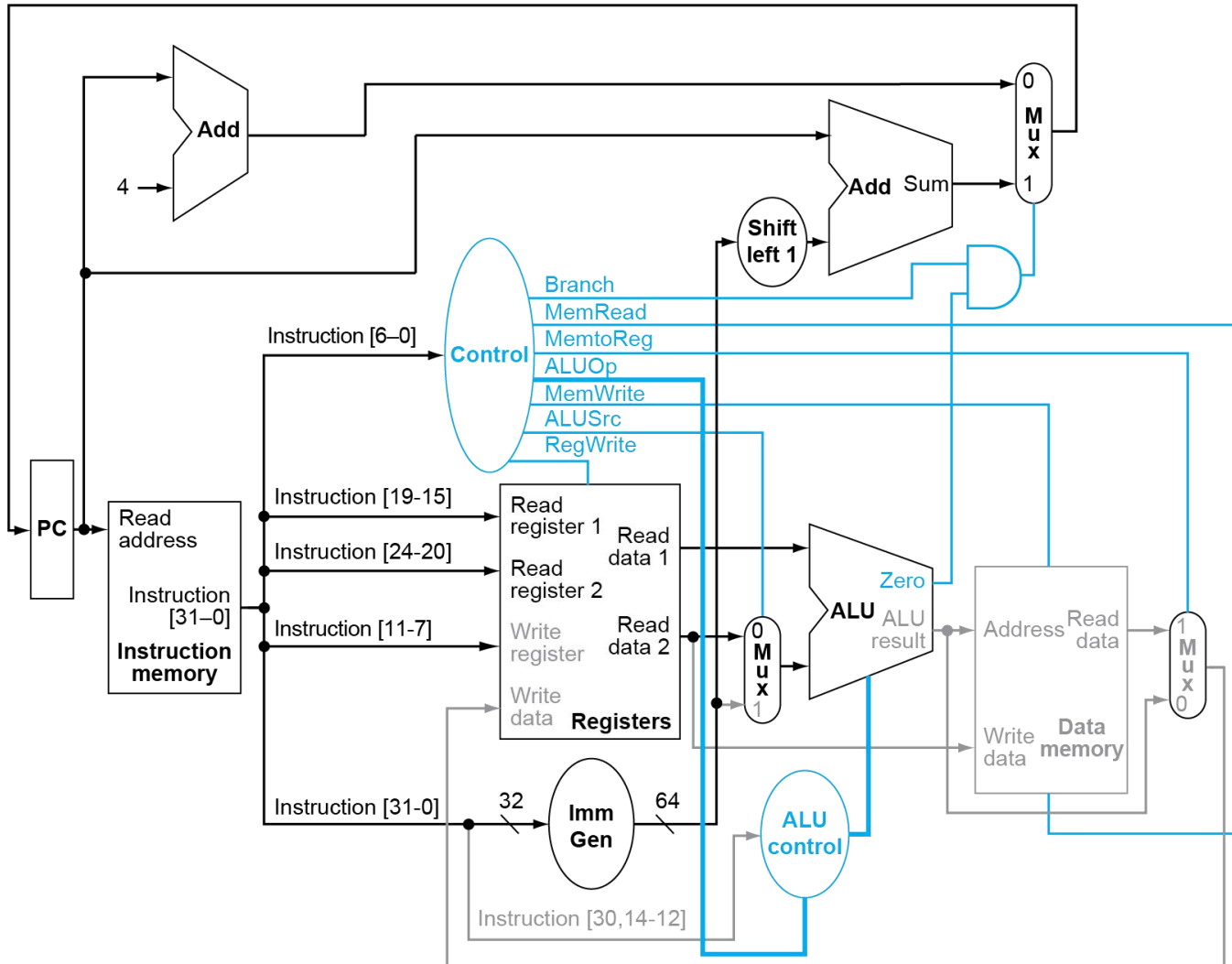
MK



load（取数）指令



beq (相等则分支) 指令

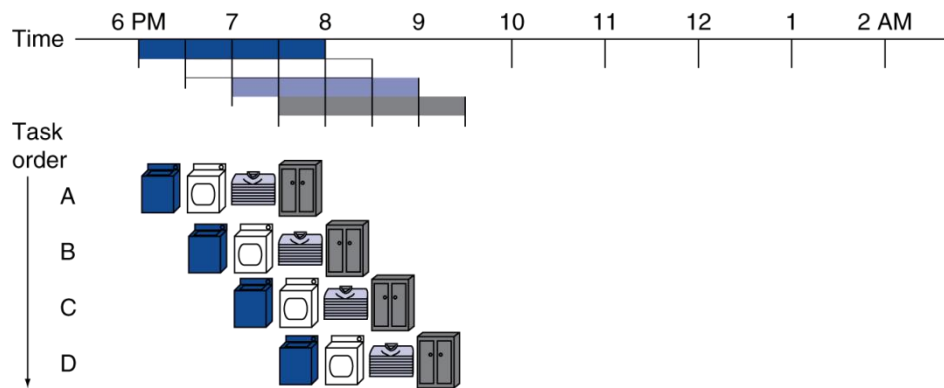
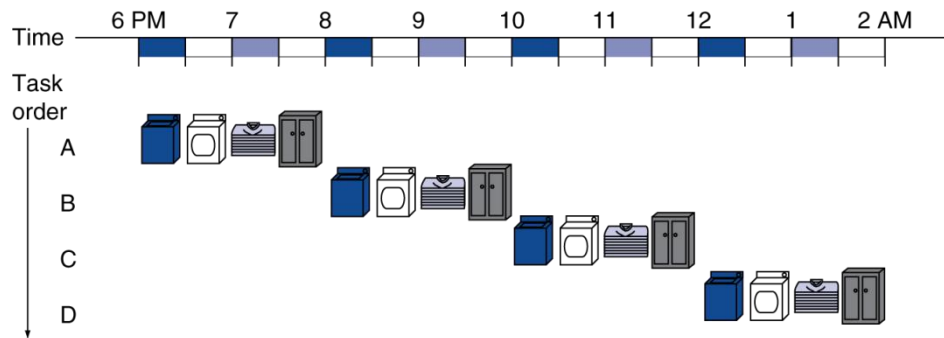


性能问题

- 最长延迟决定了时钟周期
 - 关键路径：load指令
 - 指令存储器→寄存器文件→ALU→数据存储器→寄存器文件
- 为不同指令改变周期是不可行的
- 违反设计原则
 - 加速大概率事件
- 我们将通过流水线化提升性能

流水线的比喻

- 流水线化的洗衣店：重叠执行
 - 并行提升性能



- 4件任务时
 - 加速比
 $= 8 / 3.5 = 2.3$

- 不停机时
 - 加速比
 $= 2n / (0.5n + 1.5) \approx 4$
 $= \text{流水线级数}$

RISC-V流水线

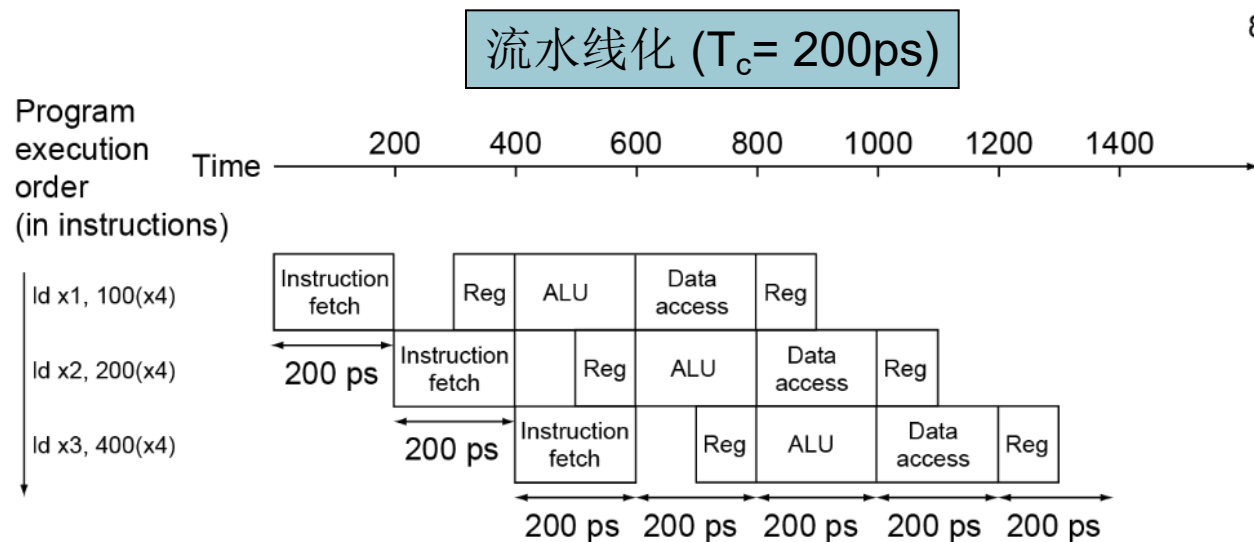
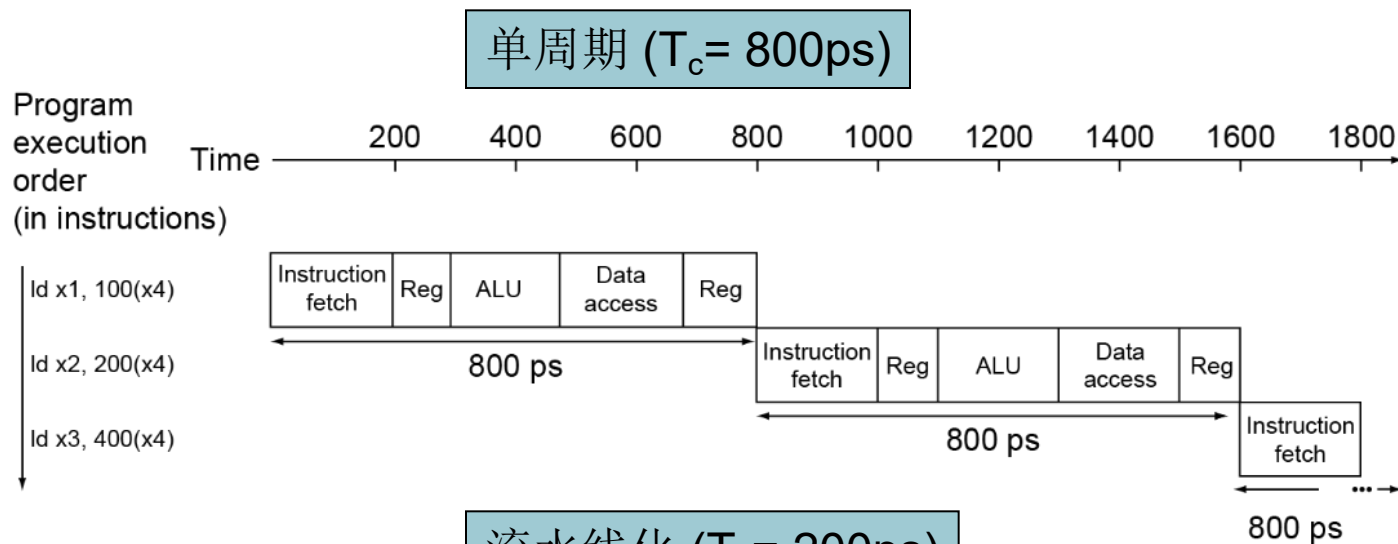
- 分为五级，每级对应一个步骤
 1. IF: 从指令存储器中读取指令
 2. ID: 指令译码以及寄存器读
 3. EX: 执行操作或计算地址
 4. MEM: 访问存储器操作数
 5. WB: 将结果写回寄存器

流水线的性能

- 假定各级的用时为
 - 读写寄存器：100ps
 - 其他级：200ps
- 比较流水线化的数据通路与单周期数据通路

指令	取指令	读寄存器	ALU操作	访存	写寄存器	总时间
ld	200ps	100 ps	200ps	200ps	100 ps	800ps
sd	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

流水线的性能



流水线的加速比

- 如果各级平衡
 - 即耗时相同
 - 指令执行时间_{流水线}
= 指令执行时间_{非流水线} / 流水线级数
- 若不平衡，则加速比降低
- 通过增加吞吐率来加速
 - 延迟（单条指令的时间）没有减少

流水线与ISA设计

- RISC-V ISA是面向流水线设计的
 - 所有指令都是32位
 - 更易于在一个周期内取指令并译码
 - 对比x86：指令长度为1~17字节
 - 少而规整的指令格式
 - 可以在一步内译码并读寄存器
 - 取数/存数寻址
 - 可以在第3级计算地址，在第4级访问存储器

冒险

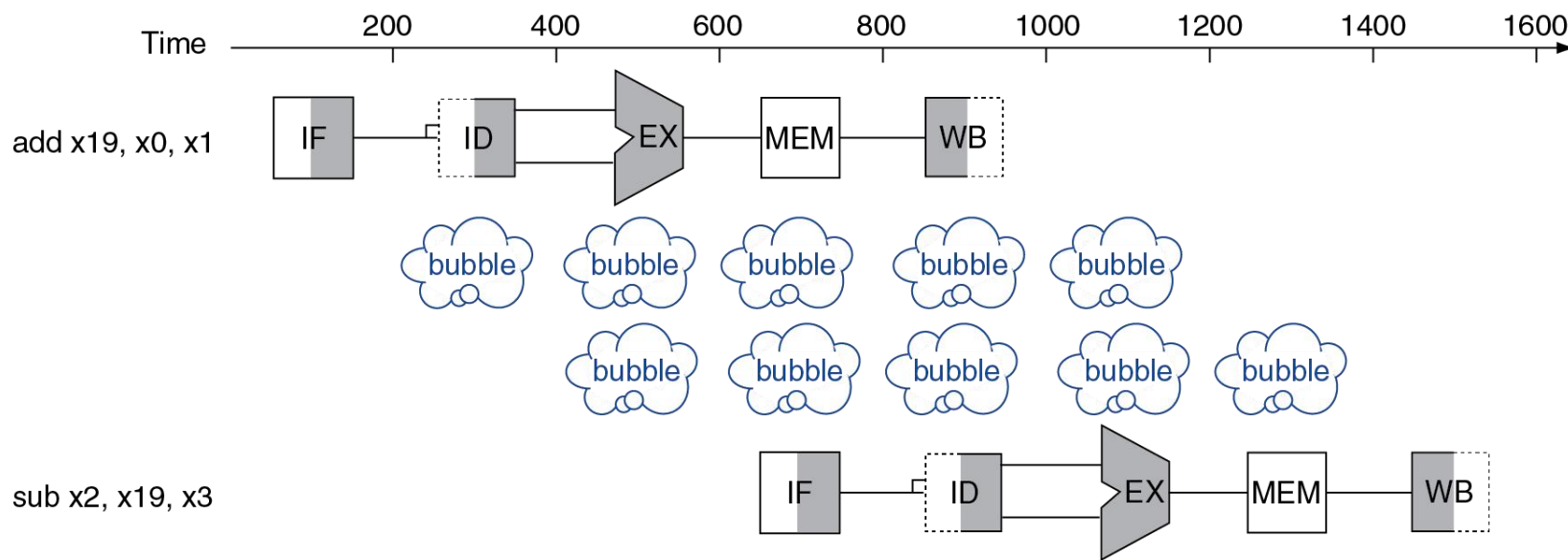
- 阻碍在下个周期执行下条指令的情况
- 结构冒险
 - 所需资源正被占用
- 数据冒险
 - 需要等待先前的指令完成数据读写
- 控制冒险
 - 对控制行为的决策依赖于先前的指令

结构冒险

- 资源使用冲突
- 在单存储器的RISC-V流水线中
 - 取数/存数指令需要访问数据
 - 在这个周期，取指令操作只好阻塞
 - 将产生流水线“气泡”
- 因此，流水线化的数据通路需要分离的指令/数据存储器
 - 或分离的指令/数据cache

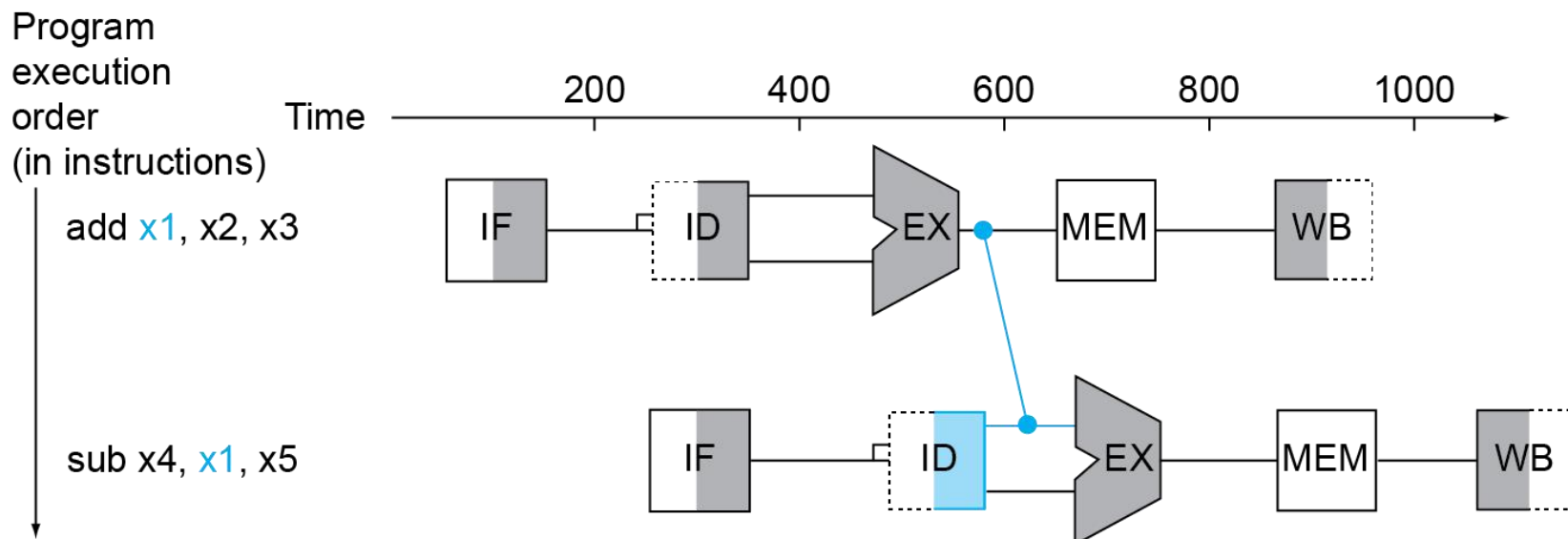
数据冒险

- 一条指令取决于先前的指令完成数据访问
 - add x19, x0, x1
sub x2, x19, x3



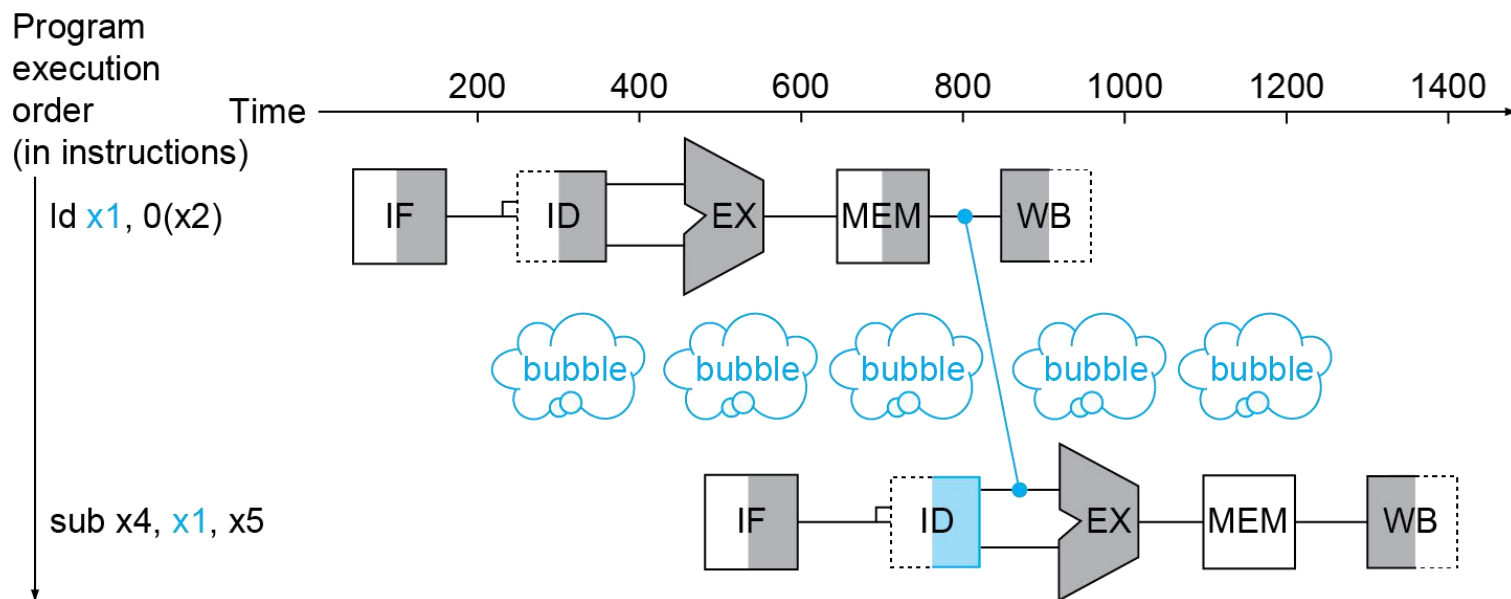
旁路（亦称前推）

- 当算出结果就直接使用
 - 不等到结果被存入寄存器
 - 需要在数据通路中做额外连接



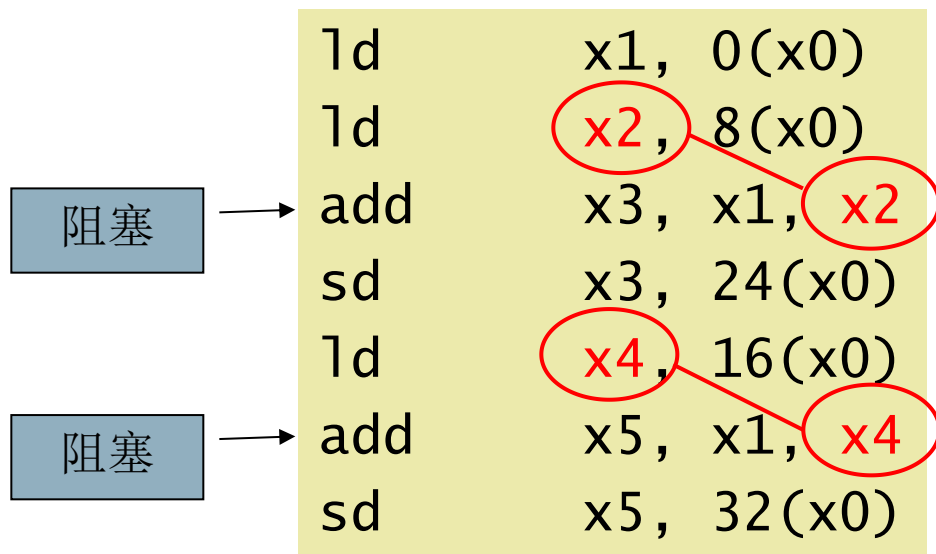
取数-使用型数据冒险

- 无法总靠旁路来避免阻塞
 - 如果在要用到数据时还未算出结果
 - 不能旁路到过去的时间

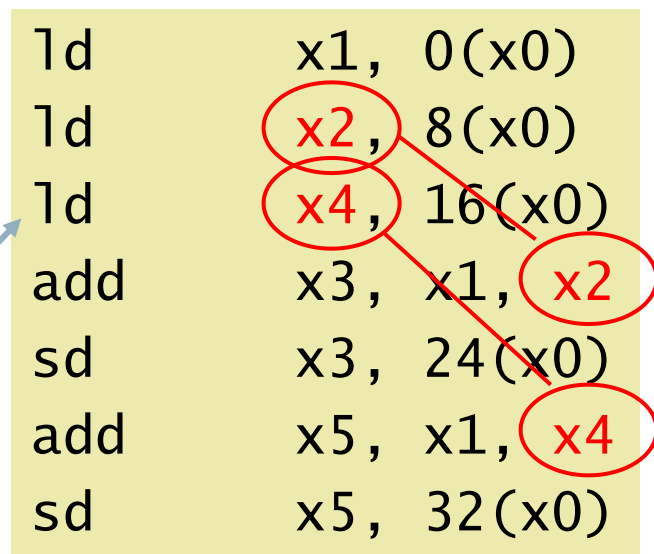


调度代码以避免阻塞

- 重排代码，以避免下一条指令用到取数结果
- C代码: $a = b + e; c = b + f;$



13个周期



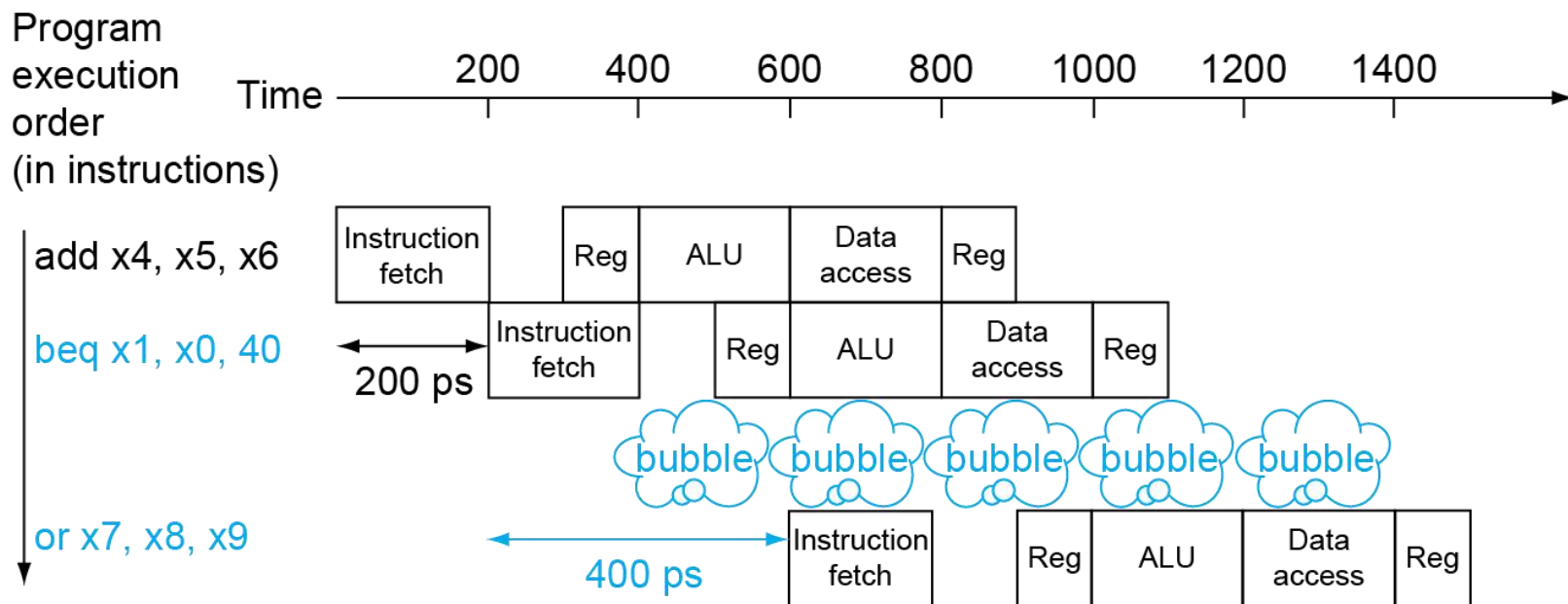
11个周期

控制冒险

- 分支决定着控制的流向
 - 根据分支结果取得下一条指令
 - 流水线无法一直取到正确的指令
 - 分支指令还在ID级处理
- 在RISC-V流水线中
 - 需要提早在流水线中比较寄存器并计算目标地址
 - 增加硬件以便在ID级完成

分支时阻塞

- 等到分支结果确定了再取下一条指令



分支预测

- 更长的流水线难以提早确定分支结果
 - 阻塞代价变得不可接受
- 预测分支结果
 - 仅在预测错误时才阻塞
- 在**RISC-V**流水线中
 - 可以预测分支不发生
 - 取分支后面的指令，不产生延迟

更实际的分支预测

■ 静态分支预测

- 基于典型的分支行为
- 例子：循环和if声明的分支
 - 预测向后的分支发生
 - 预测向前的分支不发生

■ 动态分支预测

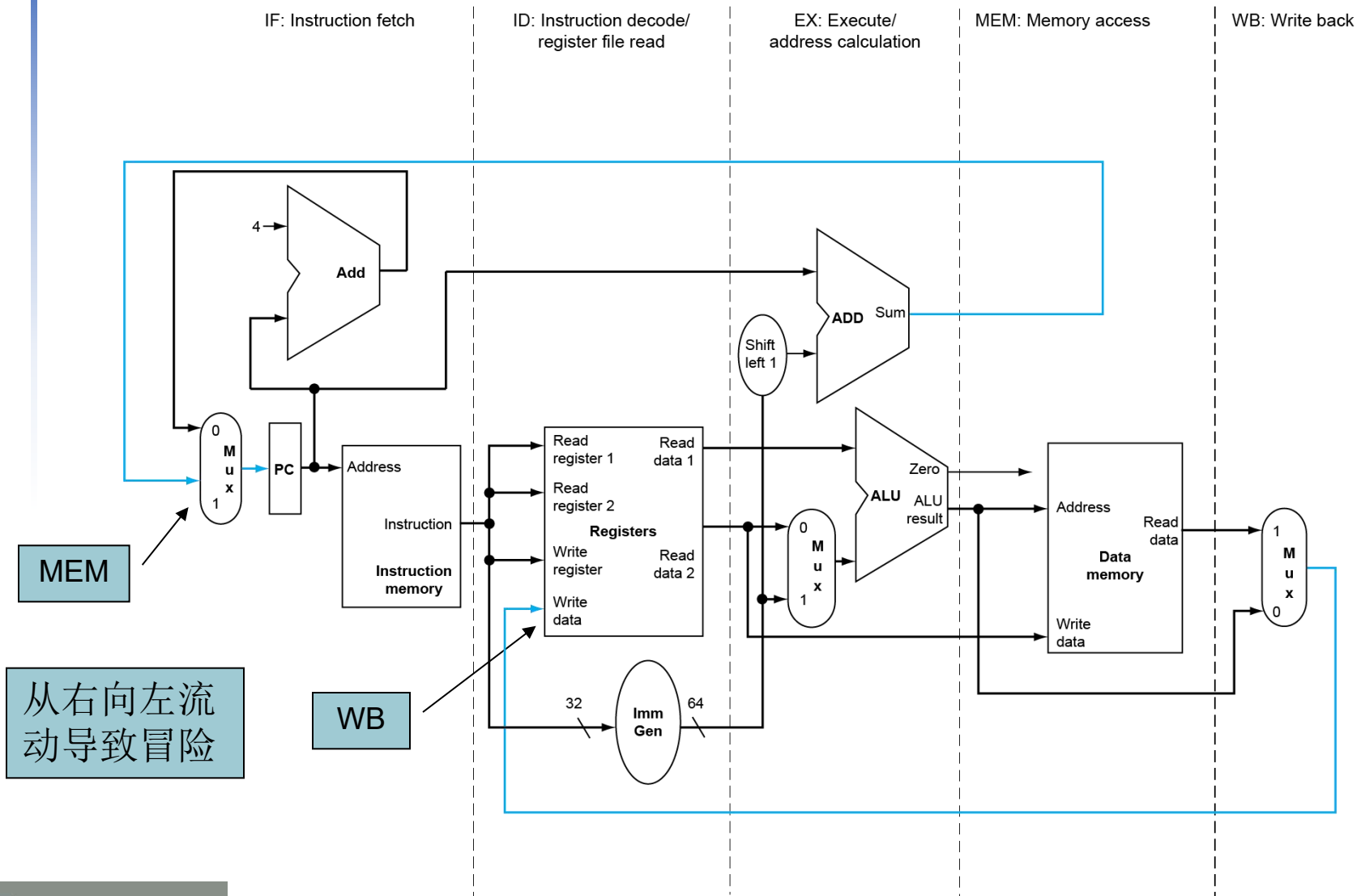
- 硬件计量实际的分支行为
 - 例如，记录每个分支的近期历史
- 假定将来的行为会延续这一趋势
 - 当预测错误，在重取指令时阻塞并更新历史记录

流水线小结

重点

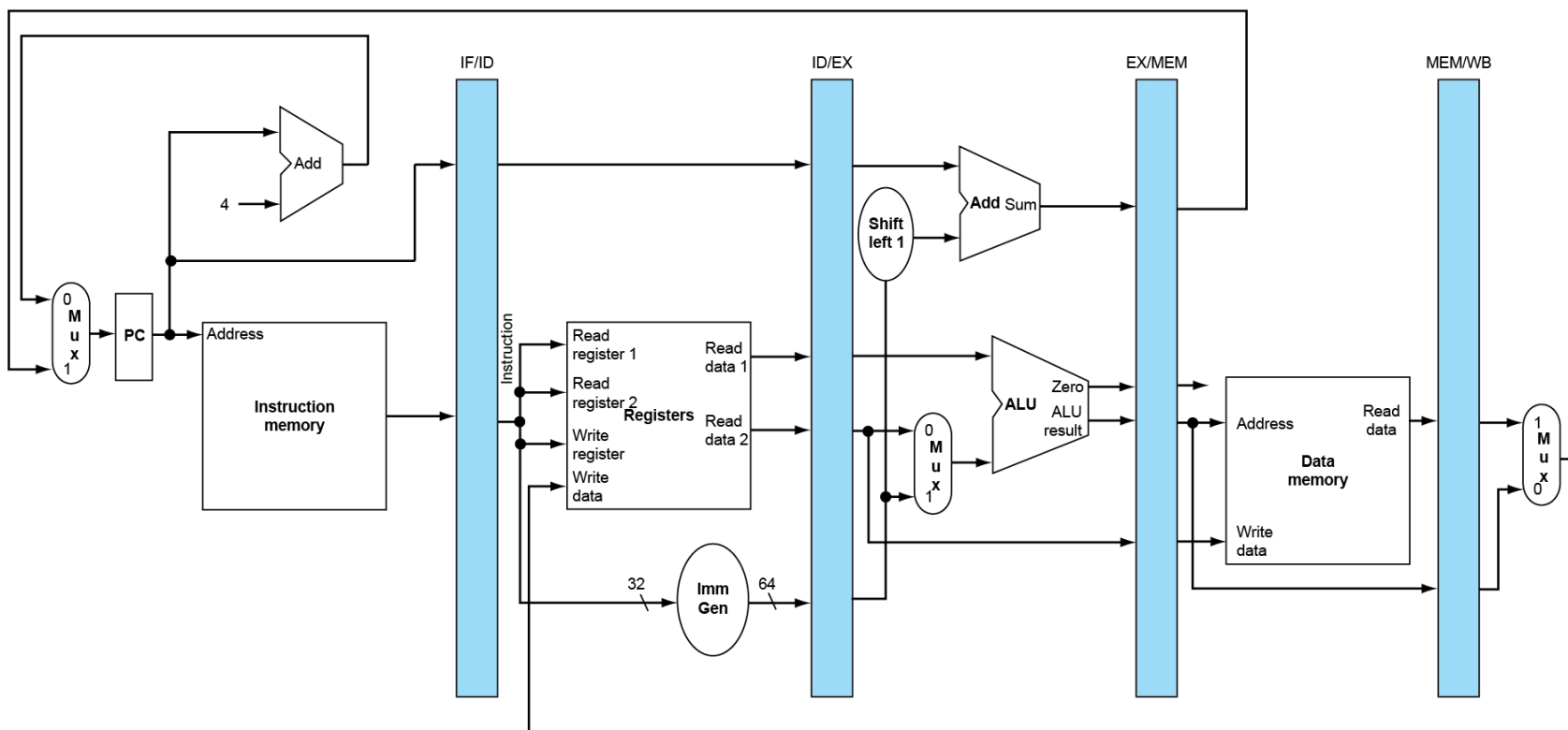
- 流水线通过增加指令的吞吐率来提升性能
 - 并行执行多条指令
 - 每条指令延迟相同
- 受冒险制约
 - 结构、数据、控制冒险
- 指令集设计影响流水线实现的复杂程度

流水线化的RISC-V数据通路



流水线寄存器

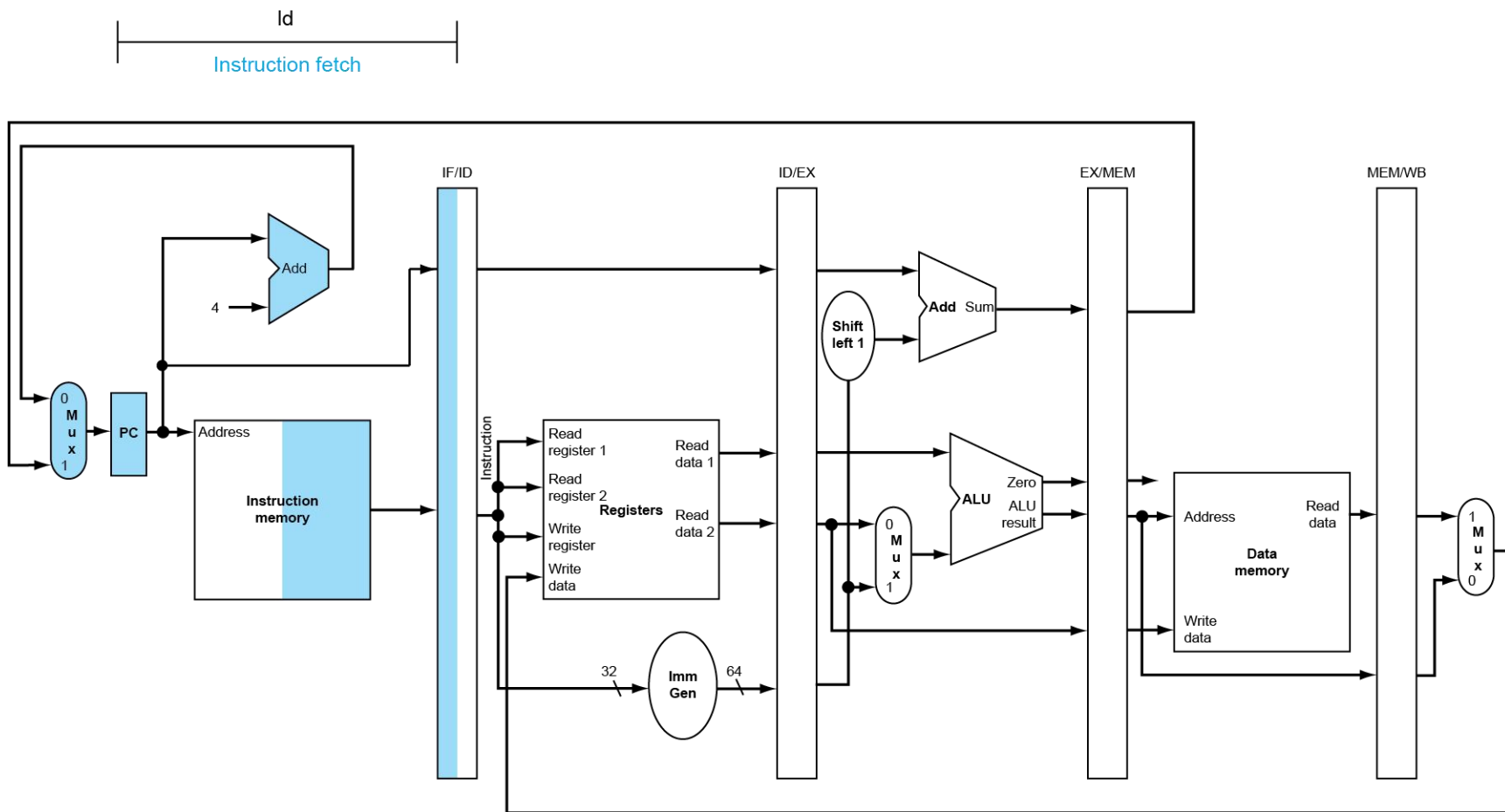
- 流水线各级间需要寄存器
 - 用于保持前一周期产生的信息



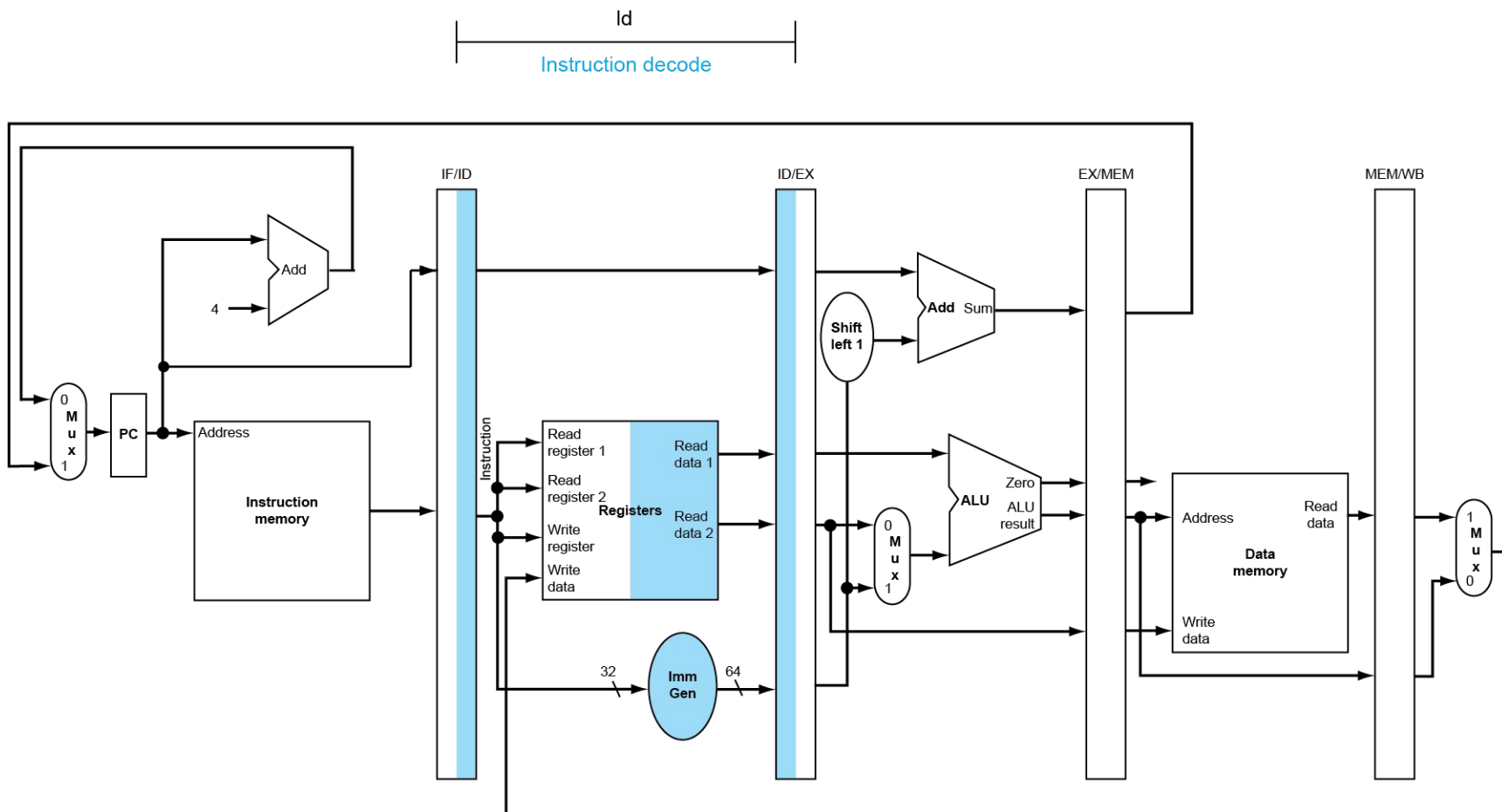
流水线操作

- 一个周期接着一个周期的指令流通过流水线化的数据通路
 - “单时钟周期”流水线图
 - 显示一个周期中流水线的使用情况
 - 高亮显示用到的资源
 - 对比“多时钟周期”图
 - 操作随时间进行的图
- 我们来分析取数&存数的“单时钟周期”图

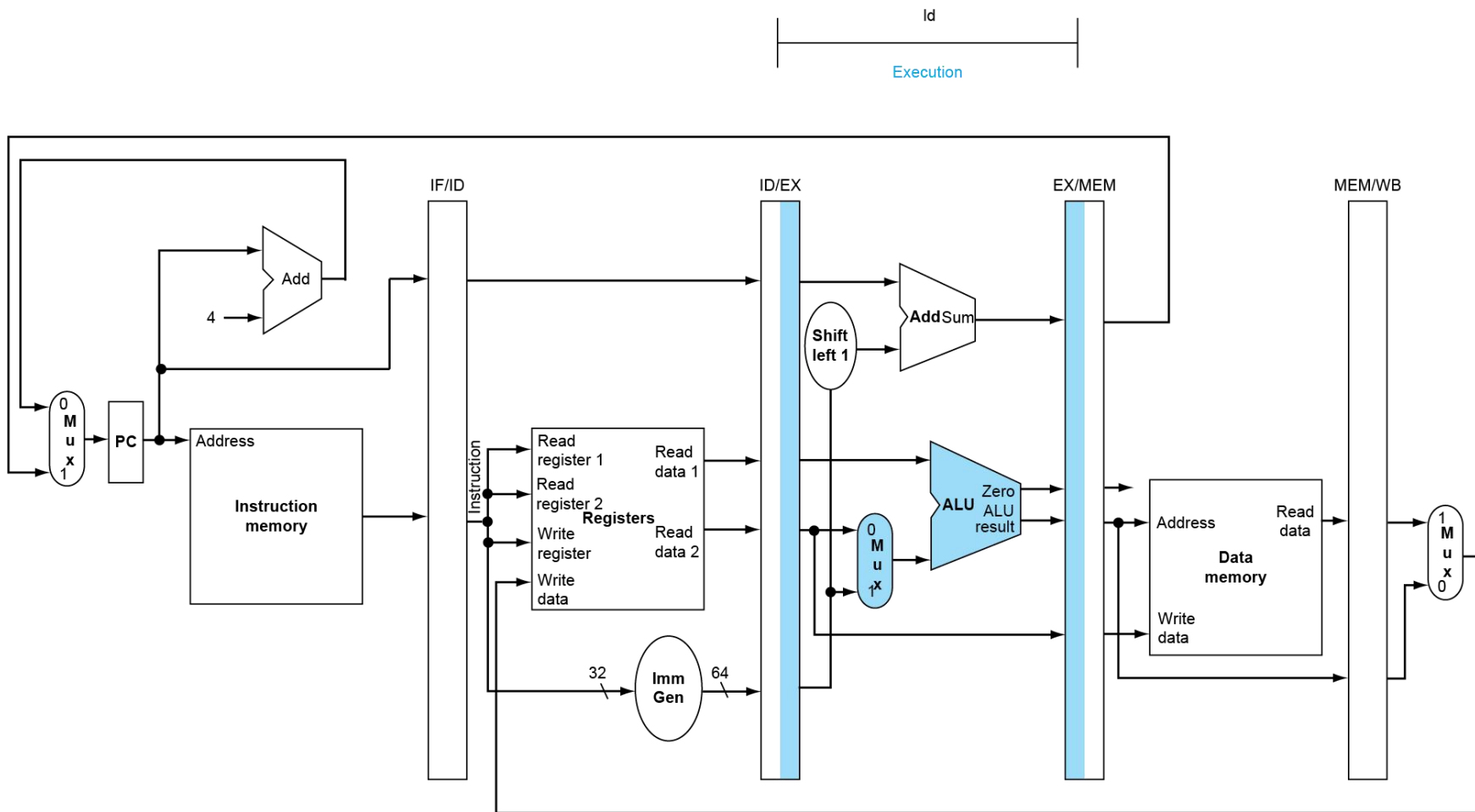
取数、存数等指令的IF级



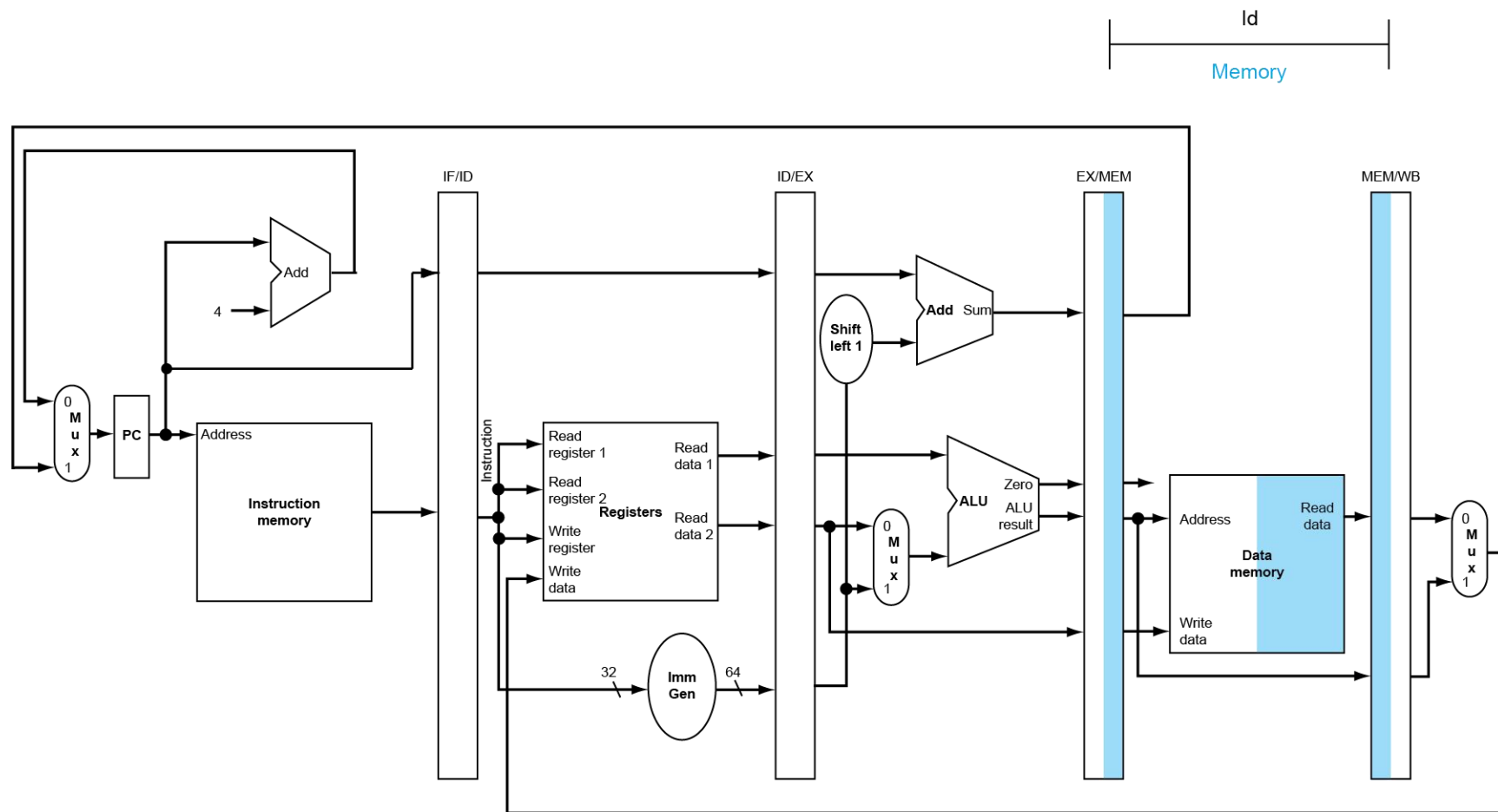
取数、存数等指令的ID级



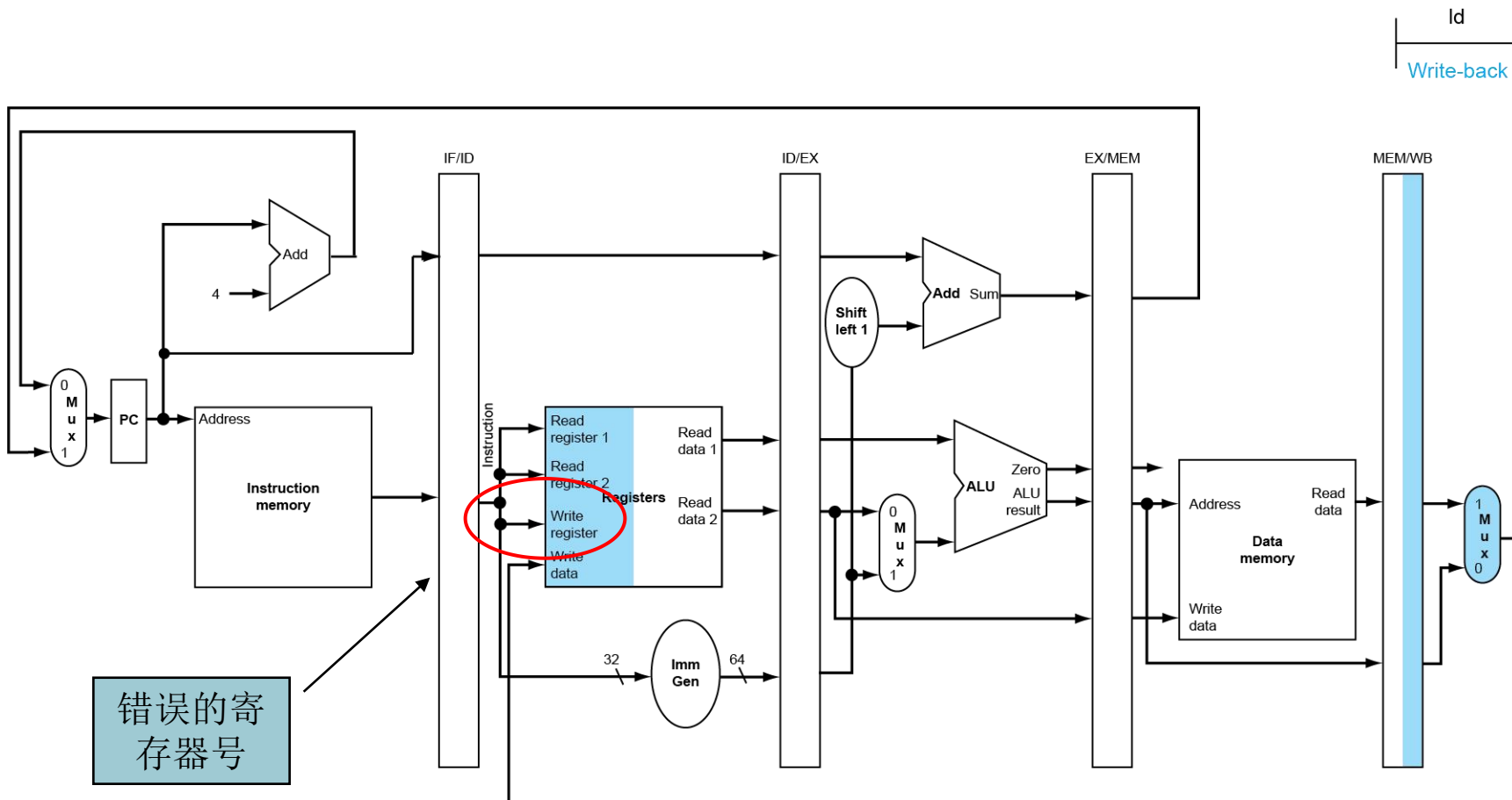
取数的EX级



取数的MEM级

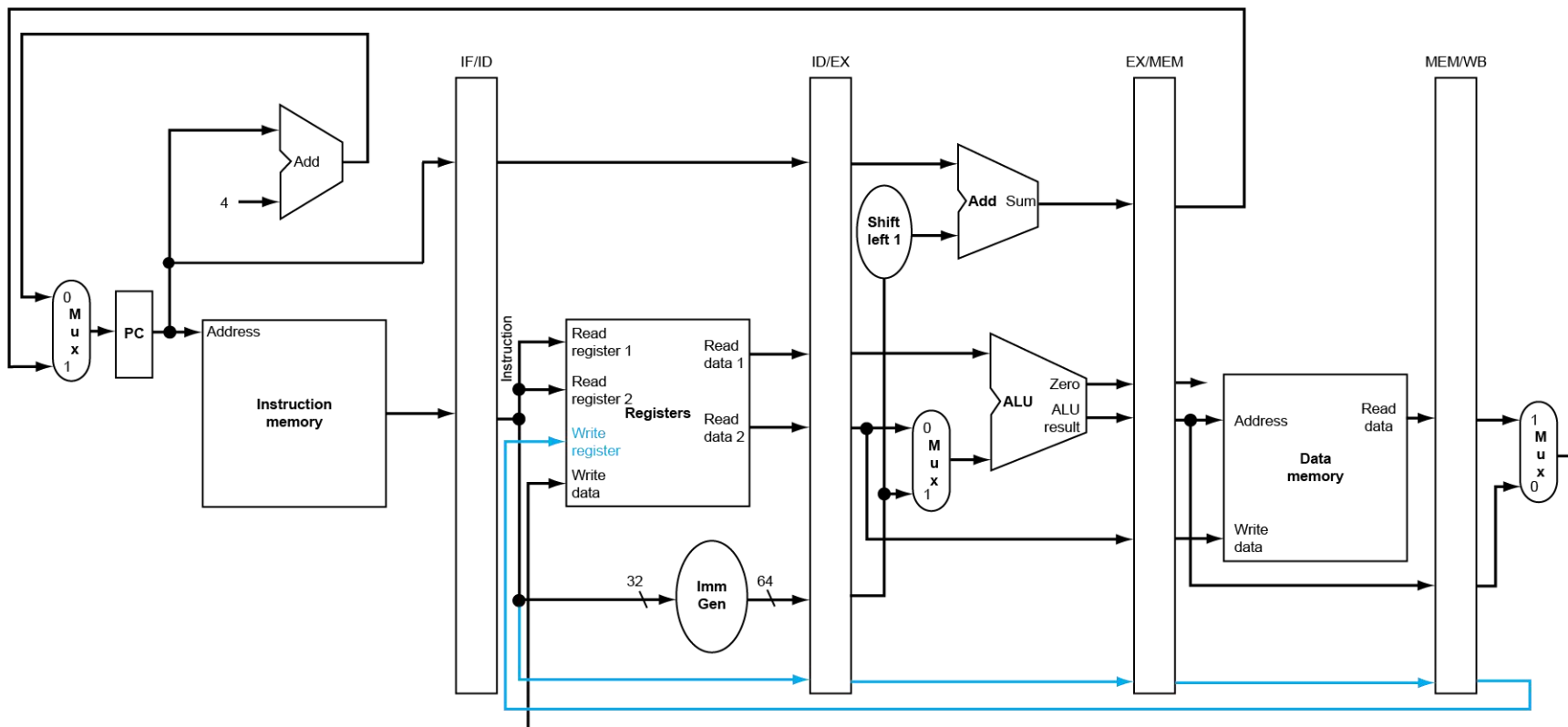


取数的WB级

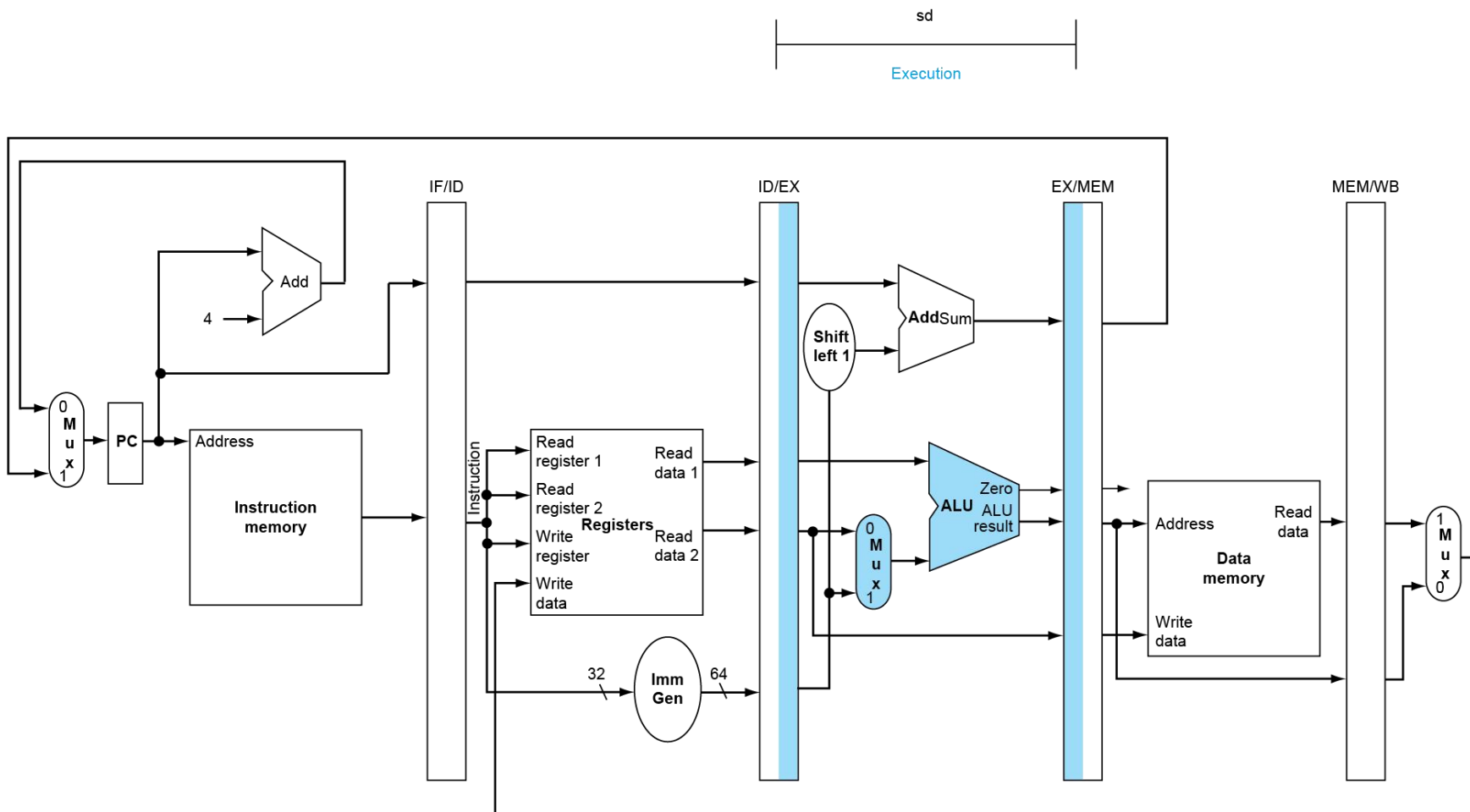


错误的寄存器号

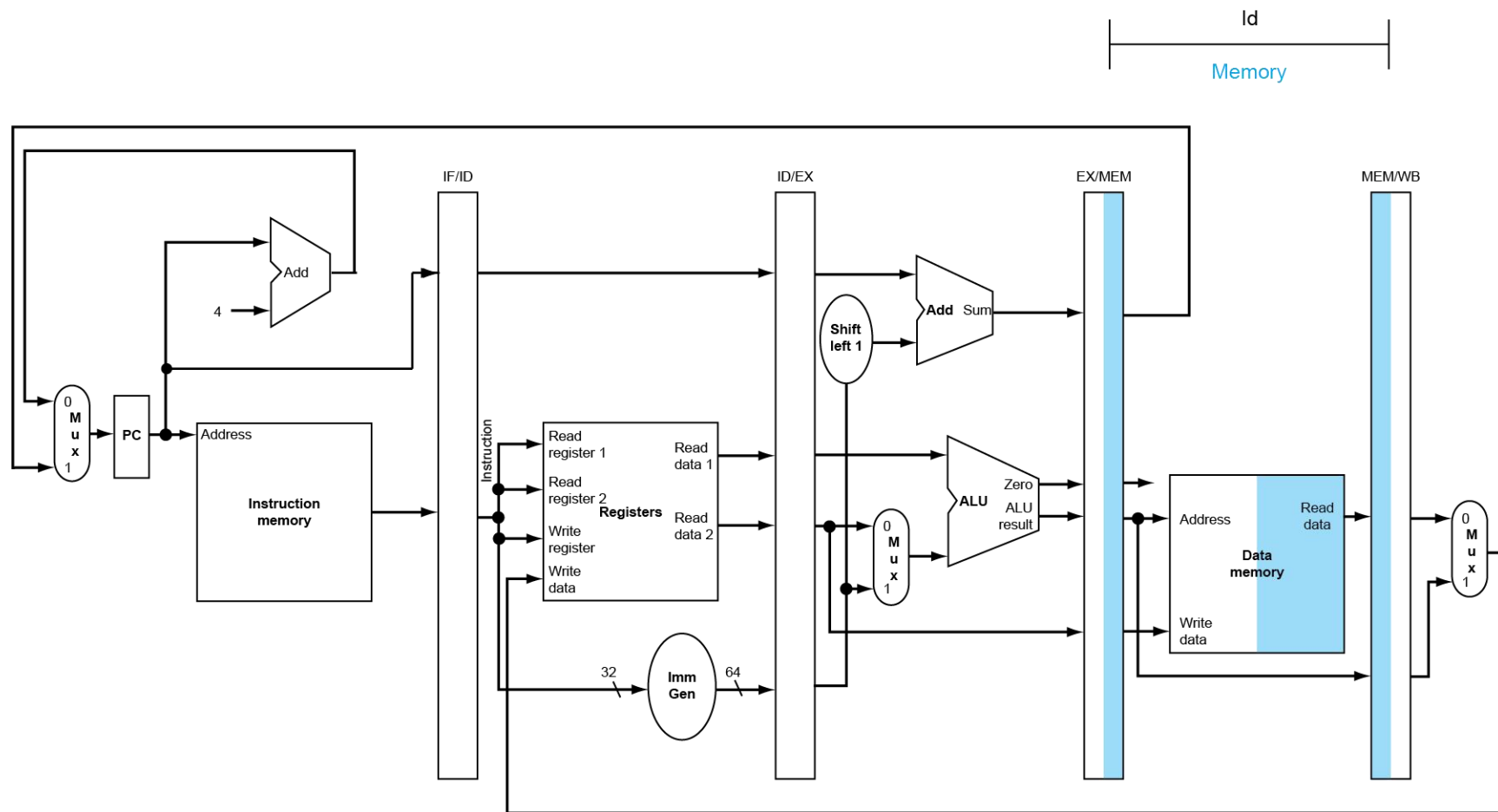
修正后的取数数据通路



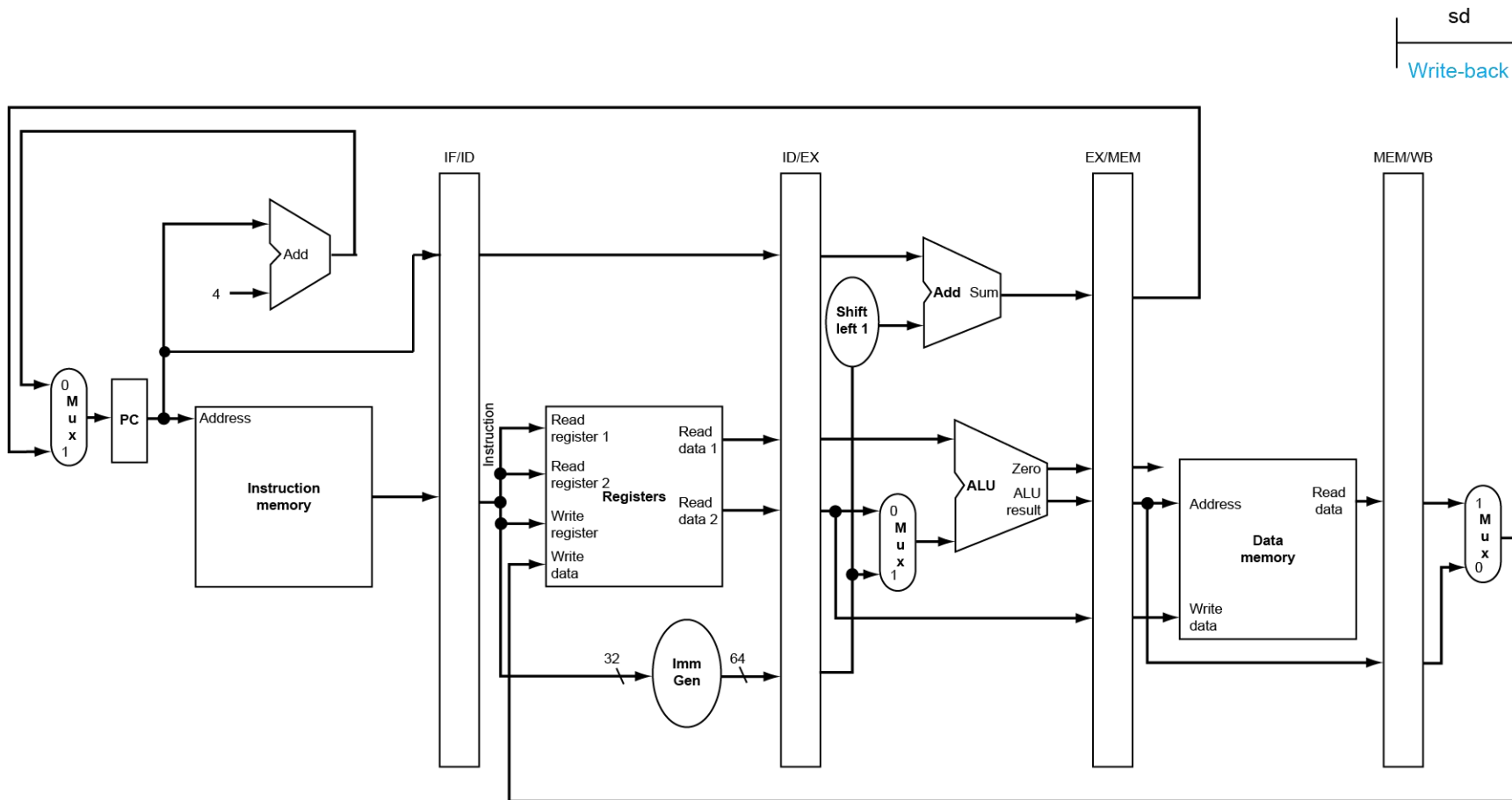
存数的EX级



存数的MEM级

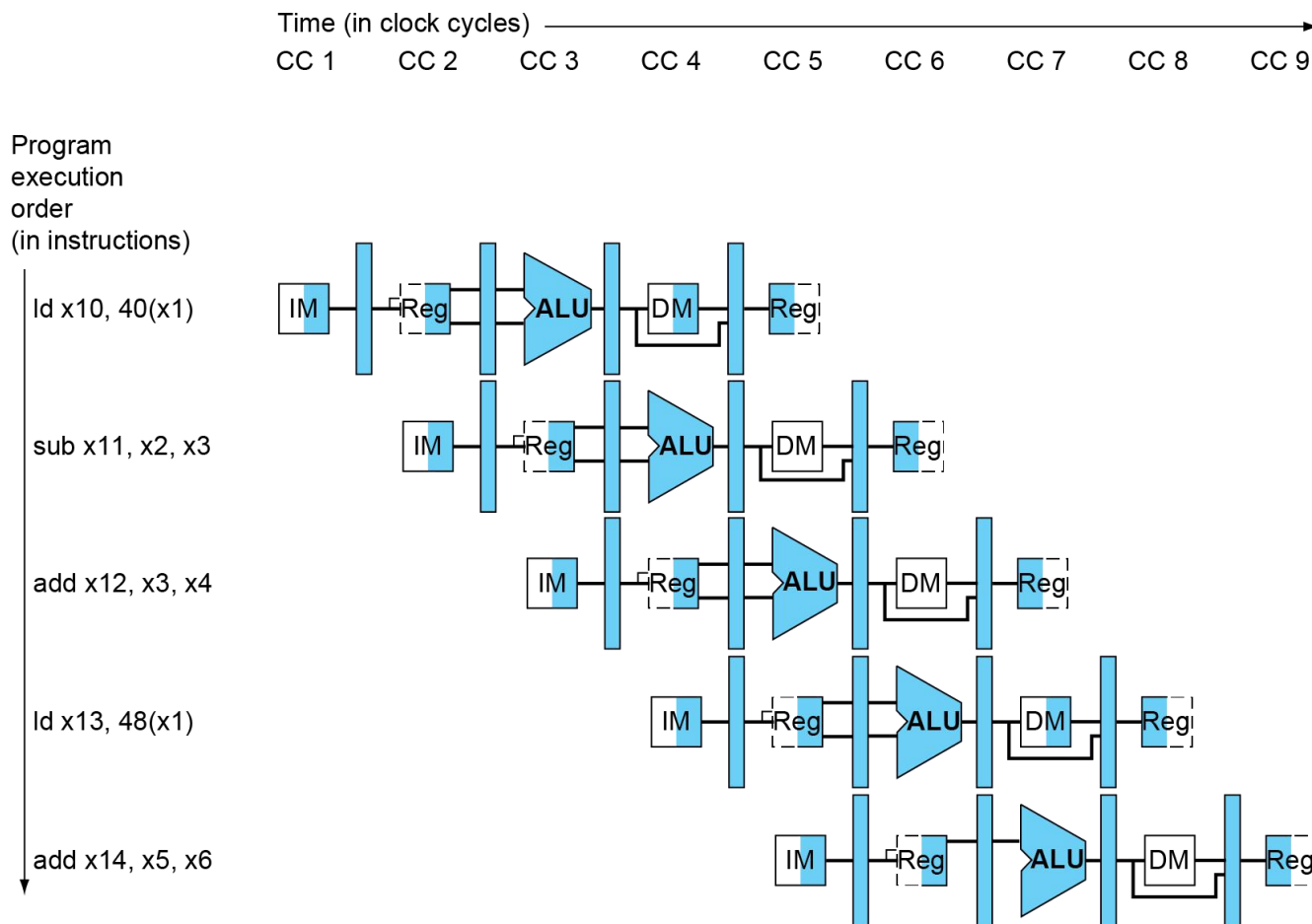


存数的WB级



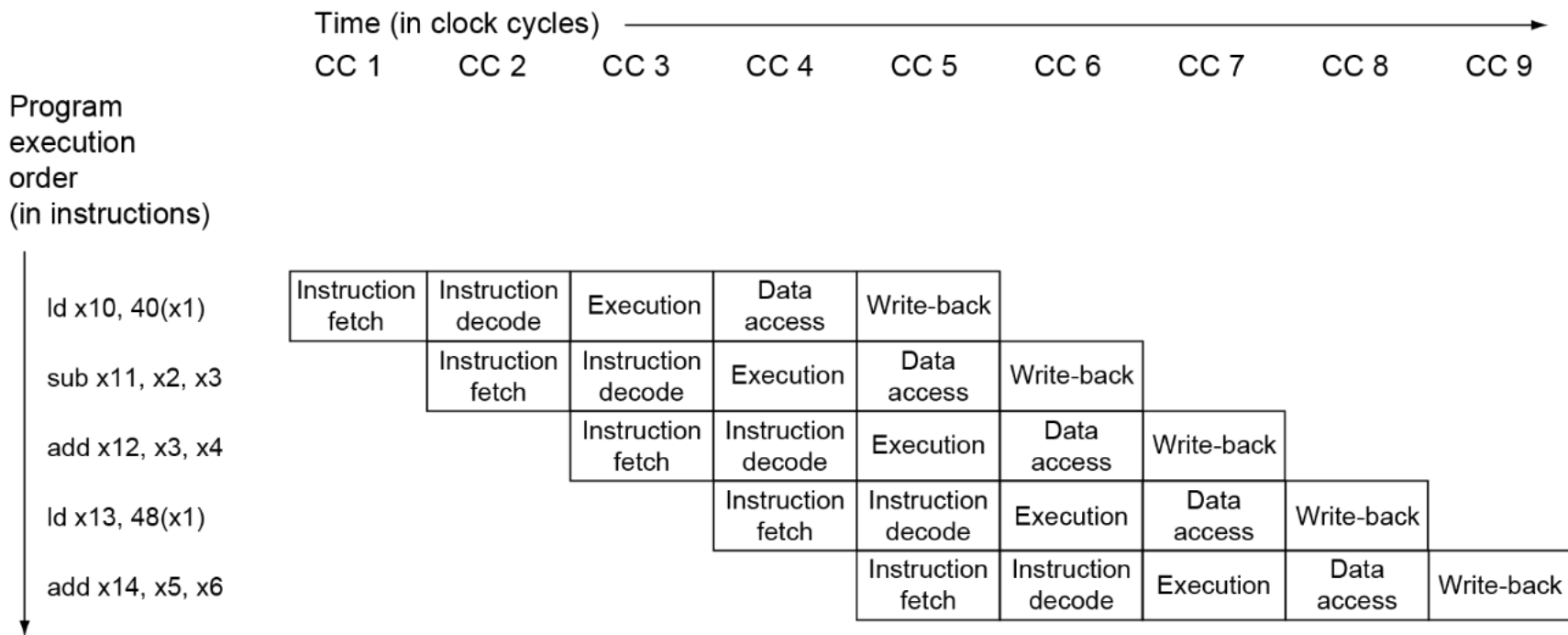
多周期流水线图

■ 显示资源使用情况的图表



多周期流水线图

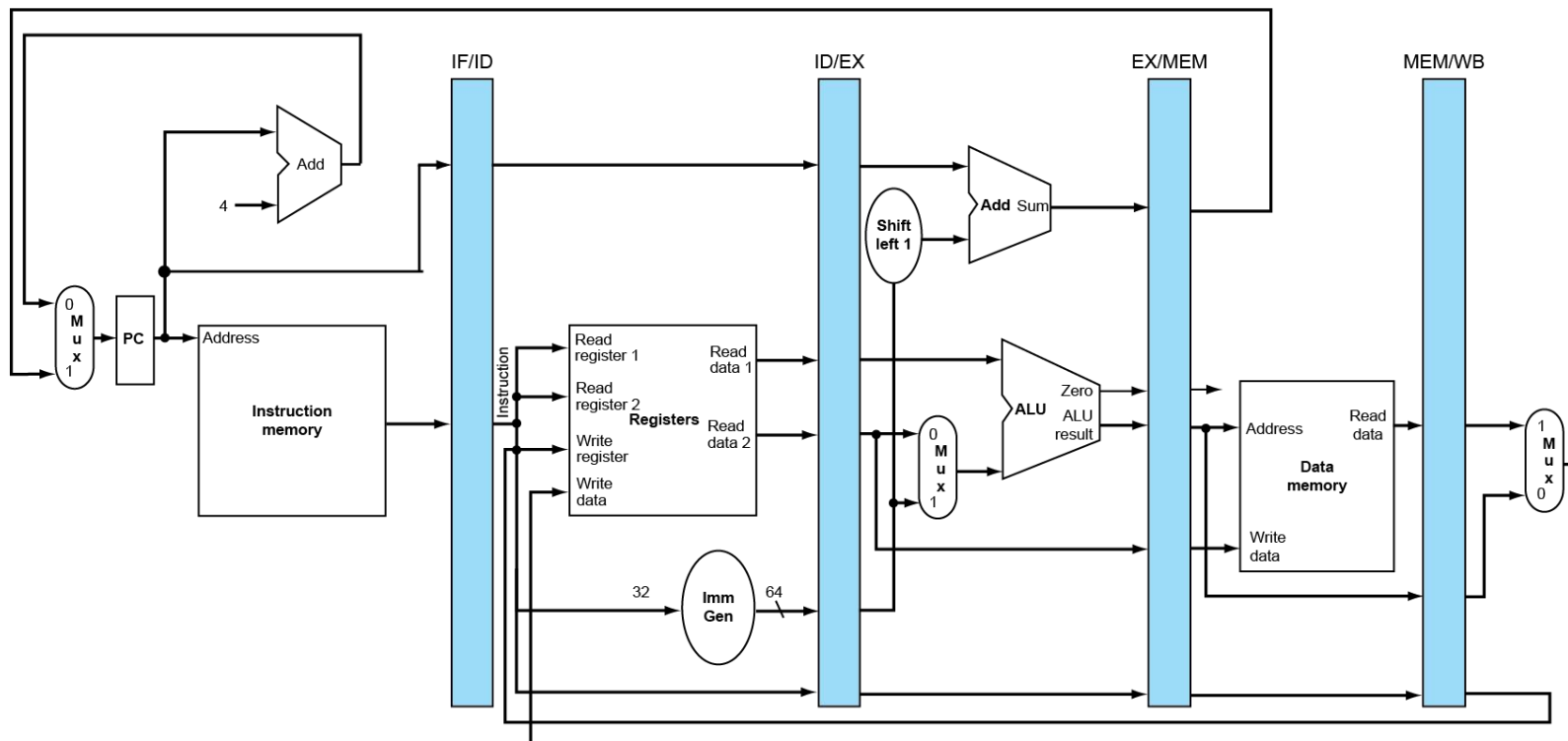
■ 传统图表



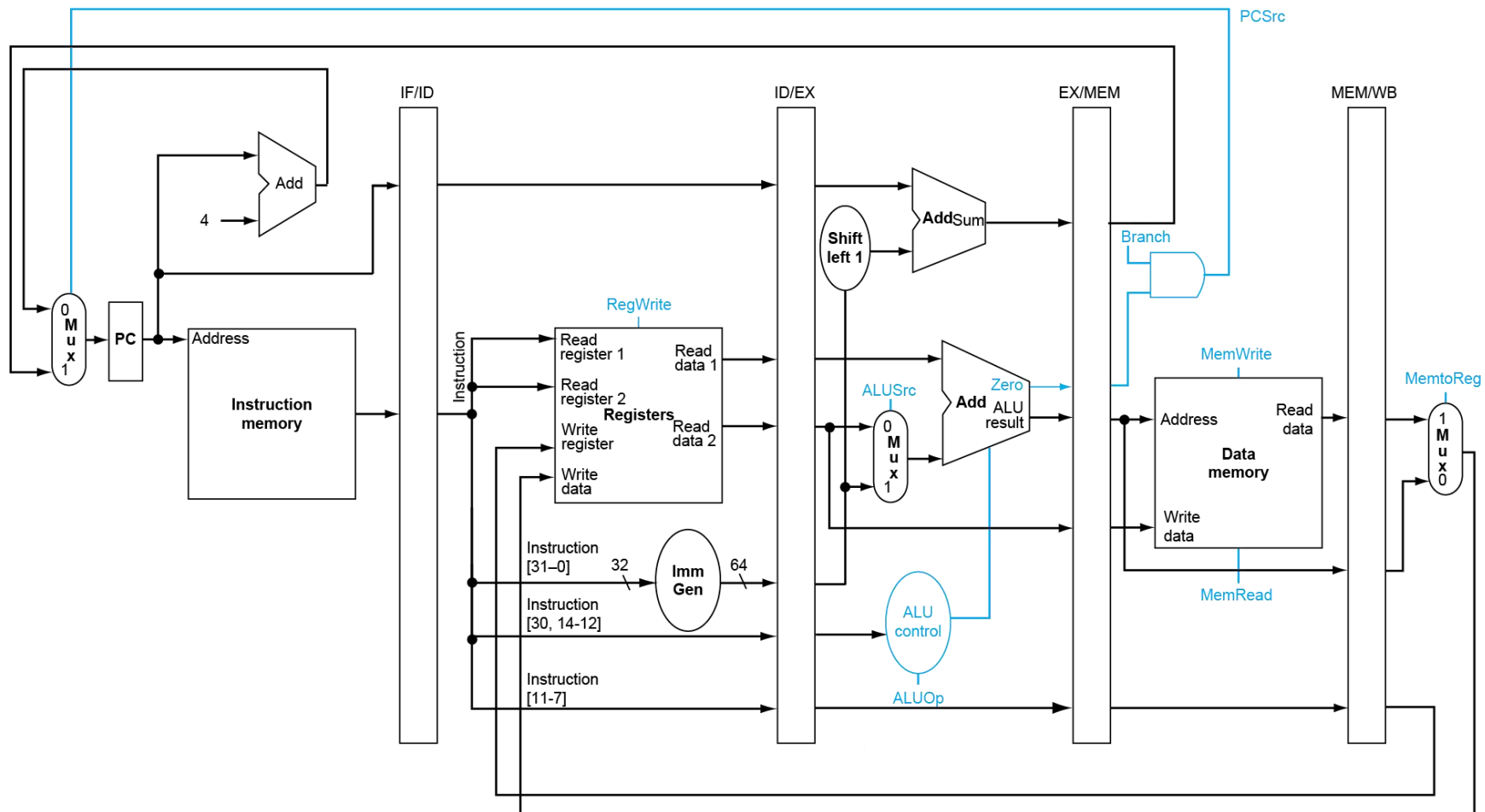
单周期流水线图

- 在给定的一个周期中流水线的状态

add x14, x5, x6	ld x13, 48(x1)	add x12, x3, x4	sub x11, x2, x3	ld x10, 40(x1)
Instruction fetch	Instruction decode	Execution	Memory	Write-back

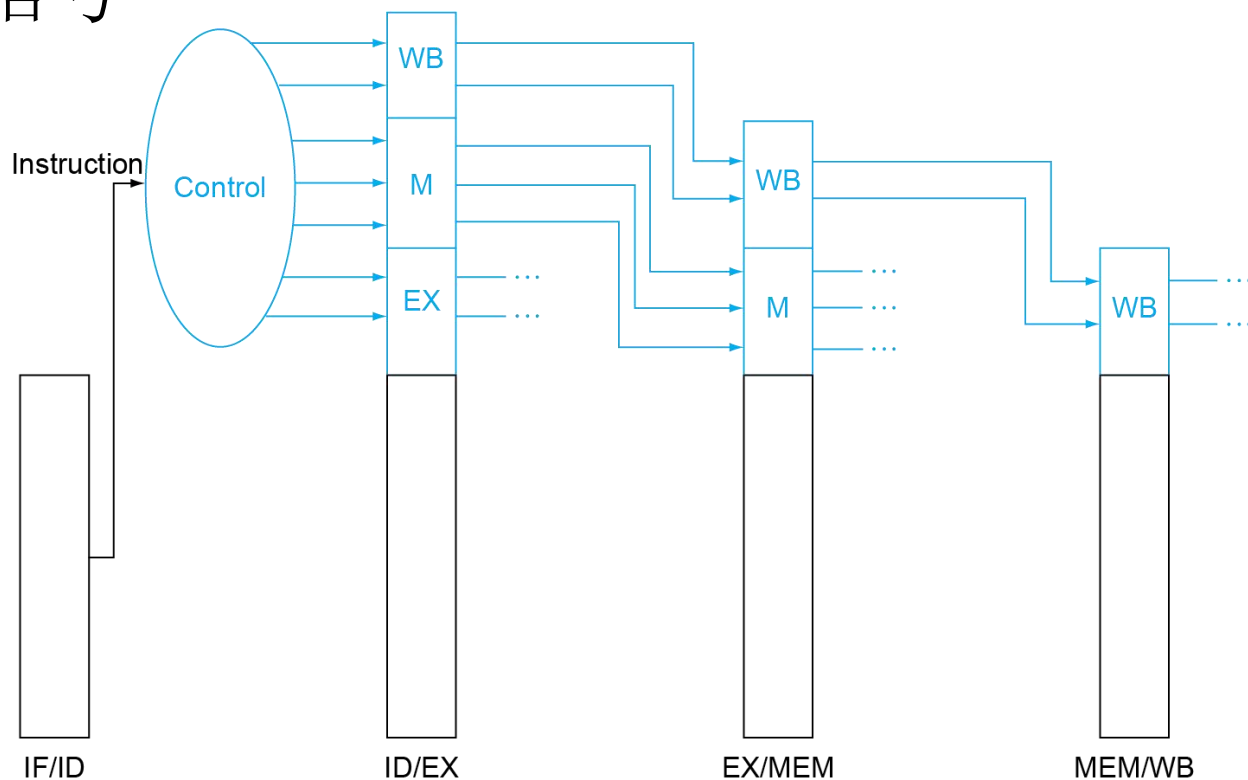


流水线控制（简化版本）

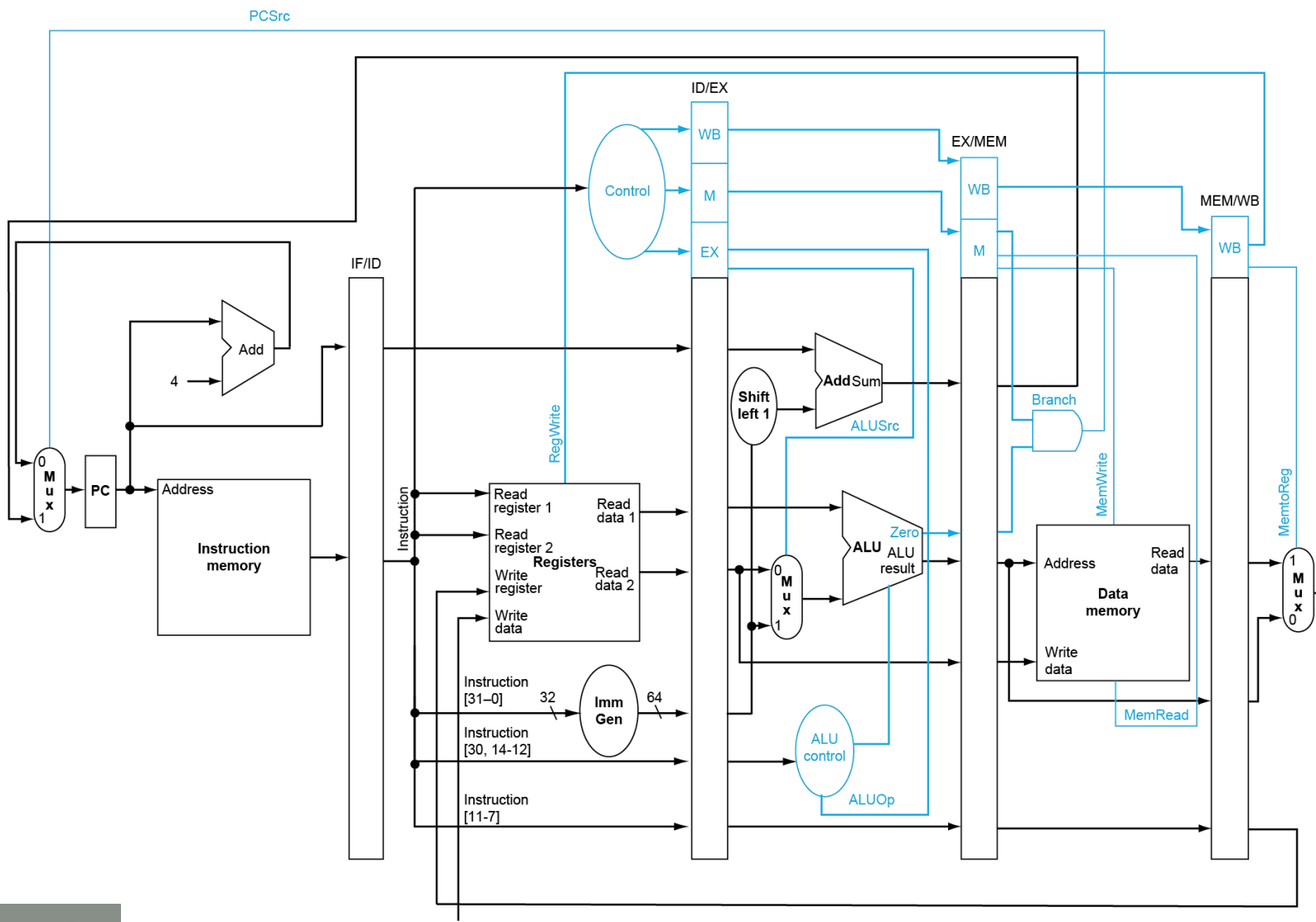


流水线控制

- 从指令得到控制信号
 - 与单周期实现相同，但还需要向后级传递控制信号



流水线控制



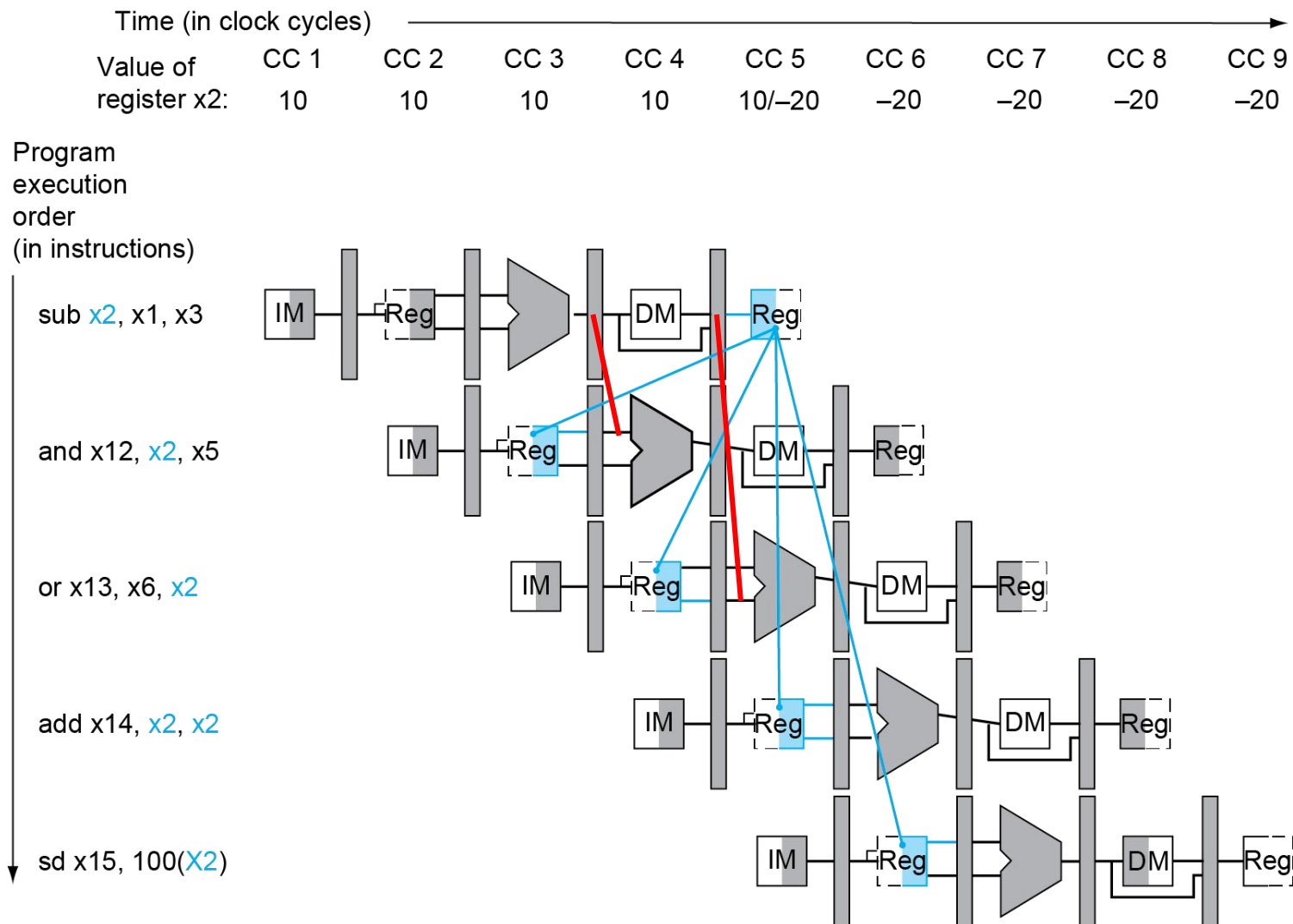
ALU指令中的数据冒险

- 考虑如下序列：

```
sub    x2, x1, x3
and     x12, x2, x5
or      x13, x6, x2
add     x14, x2, x2
sd      x15, 100(x2)
```

- 我们可以通过旁路来解决冒险
 - 我们怎样检测何时进行旁路？

依赖关系与旁路



检测旁路需求

- 沿流水线传递寄存器号
 - 例如，ID/EX.RegisterRs1表示流水线寄存器ID/EX中Rs1的寄存器号
- EX级中ALU操作数寄存器号为
 - ID/EX.RegisterRs1、ID/EX.RegisterRs2
- 以下情况发生数据冒险
 - 1a. EX/MEM.RegisterRd == ID/EX.RegisterRs1
 - 1b. EX/MEM.RegisterRd == ID/EX.RegisterRs2
 - 2a. MEM/WB.RegisterRd == ID/EX.RegisterRs1
 - 2b. MEM/WB.RegisterRd == ID/EX.RegisterRs2

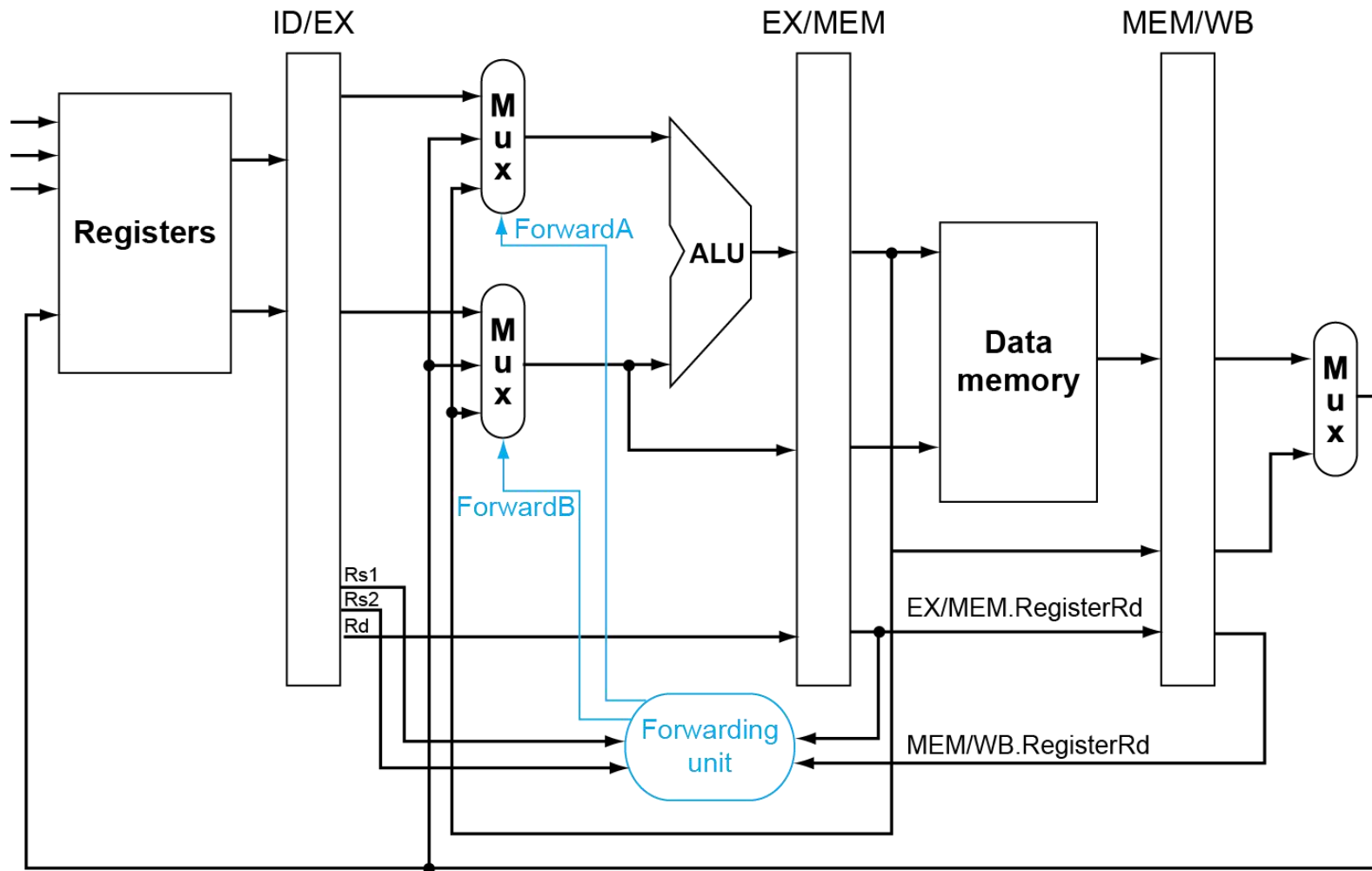
从流水线寄存器EX/MEM旁路

从流水线寄存器MEM/WB旁路

检测旁路需求

- 仅当被旁路的指令将写入寄存器！
 - EX/MEM.RegWrite、MEM/WB.RegWrite（即寄存器写使能信号有效）
- 并且这条指令的Rd不是x0
 - EX/MEM.RegisterRd $\neq$ 0,
MEM/WB.RegisterRd $\neq$ 0

旁路路径



旁路条件

多选器控制	数据源	说明
ForwardA = 00	ID/EX	第一个ALU操作数来自寄存器文件
ForwardA = 10	EX/MEM	第一个ALU操作数从前一条ALU结果旁路
ForwardA = 01	MEM/WB	第一个ALU操作数从数据存储器或更早一条ALU结果旁路
ForwardB = 00	ID/EX	第二个ALU操作数来自寄存器文件
ForwardB = 10	EX/MEM	第二个ALU操作数从前一条ALU结果旁路
ForwardB = 01	MEM/WB	第二个ALU操作数从数据存储器或更早一条ALU结果旁路

双重数据冒险

- 考虑序列:

add x1, x1, x2

add x1, x1, x3

add x1, x1, x4

- 两种冒险均发生

- 要使用最新的结果

- 修改MEM冒险条件

- 仅在EX冒险条件非真时旁路

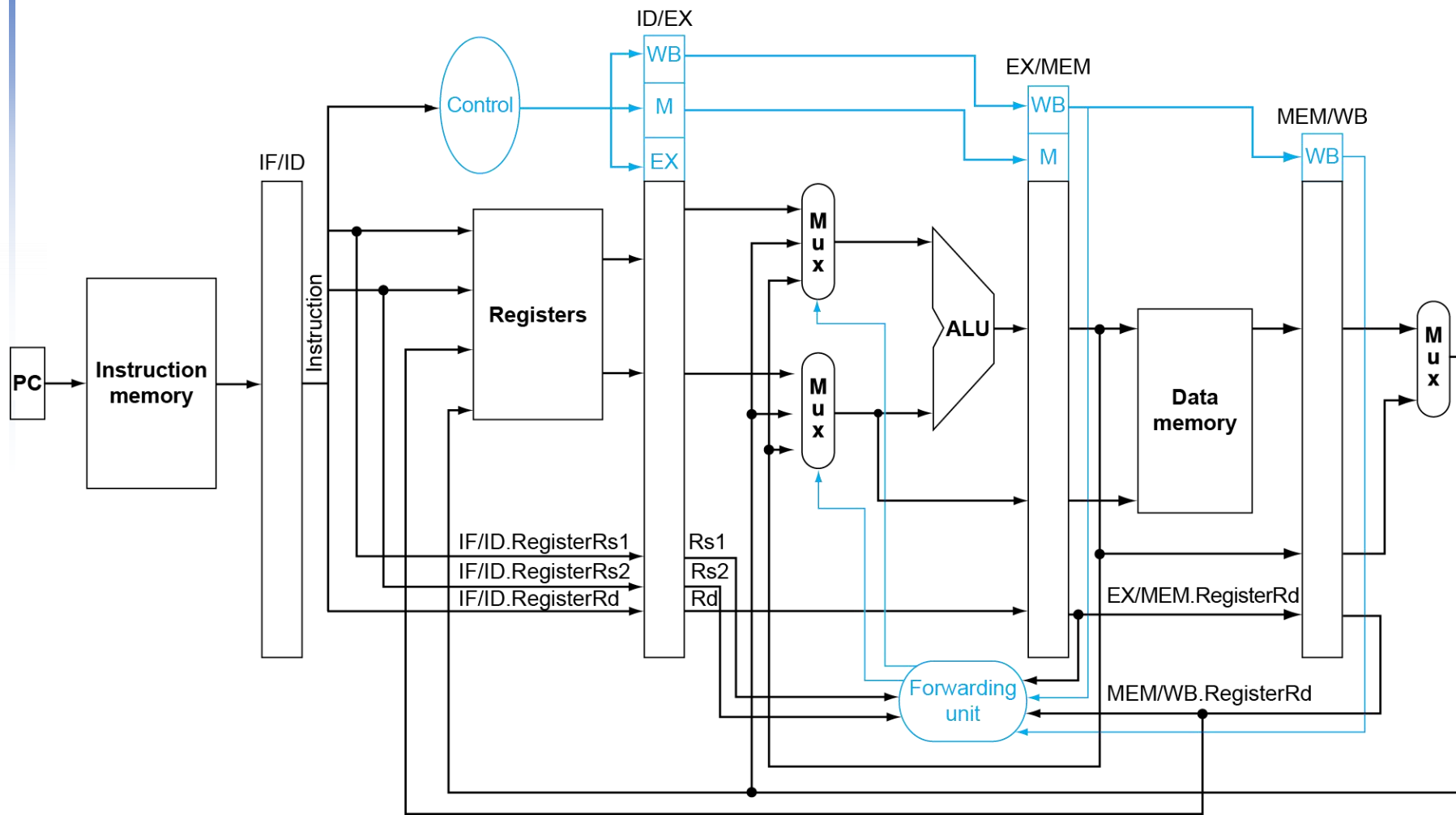
修正后的旁路条件

■ MEM冒险

- if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd == ID/EX.RegisterRs1))
and (MEM/WB.RegisterRd == ID/EX.RegisterRs1))
ForwardA = 01
- if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd == ID/EX.RegisterRs2))
and (MEM/WB.RegisterRd == ID/EX.RegisterRs2))

ForwardB = 01

带旁路的数据通路



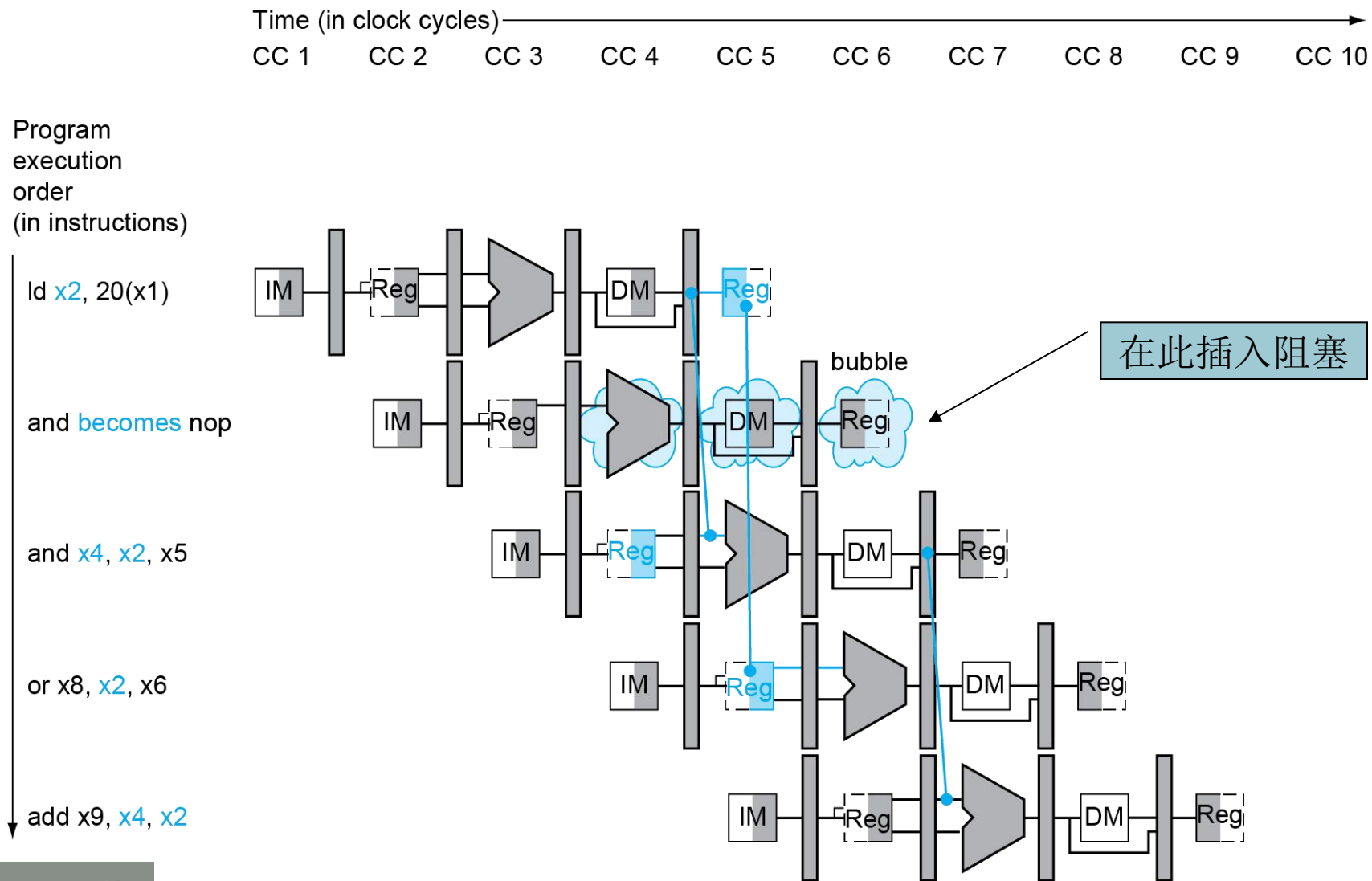
取数-使用型冒险的检测

- 当使用数据的指令在ID级译码时检测
- ID级的ALU操作数寄存器号为
 - IF/ID.RegisterRs1、IF/ID.RegisterRs2
- 取数-使用型冒险的条件
 - ID/EX.MemRead and
((ID/EX.RegisterRd == IF/ID.RegisterRs1) or
(ID/EX.RegisterRd == IF/ID.RegisterRs2))
- 一旦检测到，阻塞并插入气泡

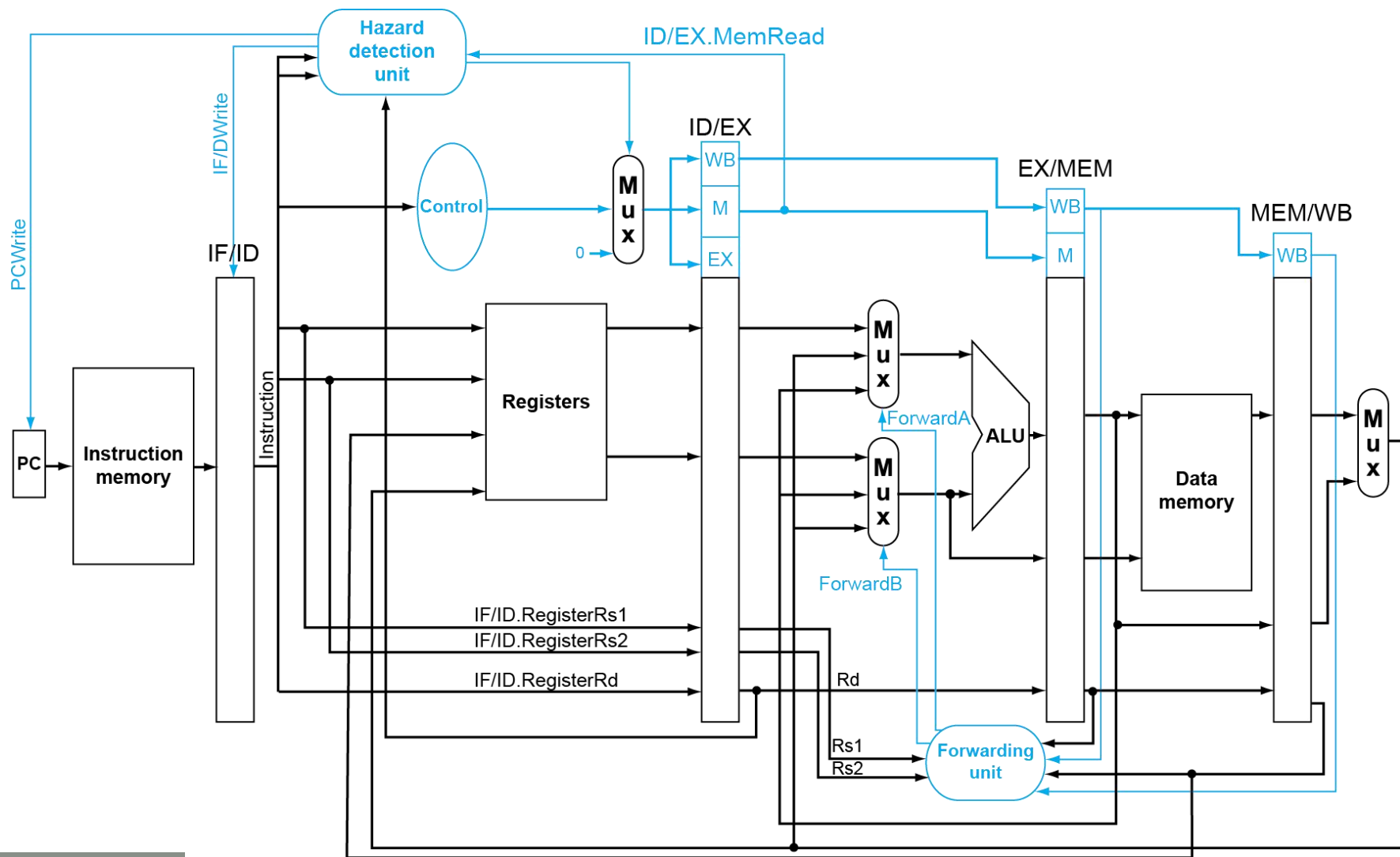
如何阻塞流水线

- 强制将ID/EX寄存器中的控制信号置0
 - EX、MEM和WB级执行空操作(nop)
- 阻止PC和IF/ID寄存器更新
 - 使用数据的指令再次被译码
 - 接下来的指令再次被取指
 - 阻塞1个周期，让MEM级为ld读取数据
 - 接着就可以旁路到EX级了

取数-使用型数据冒险



带冒险检测的数据通路



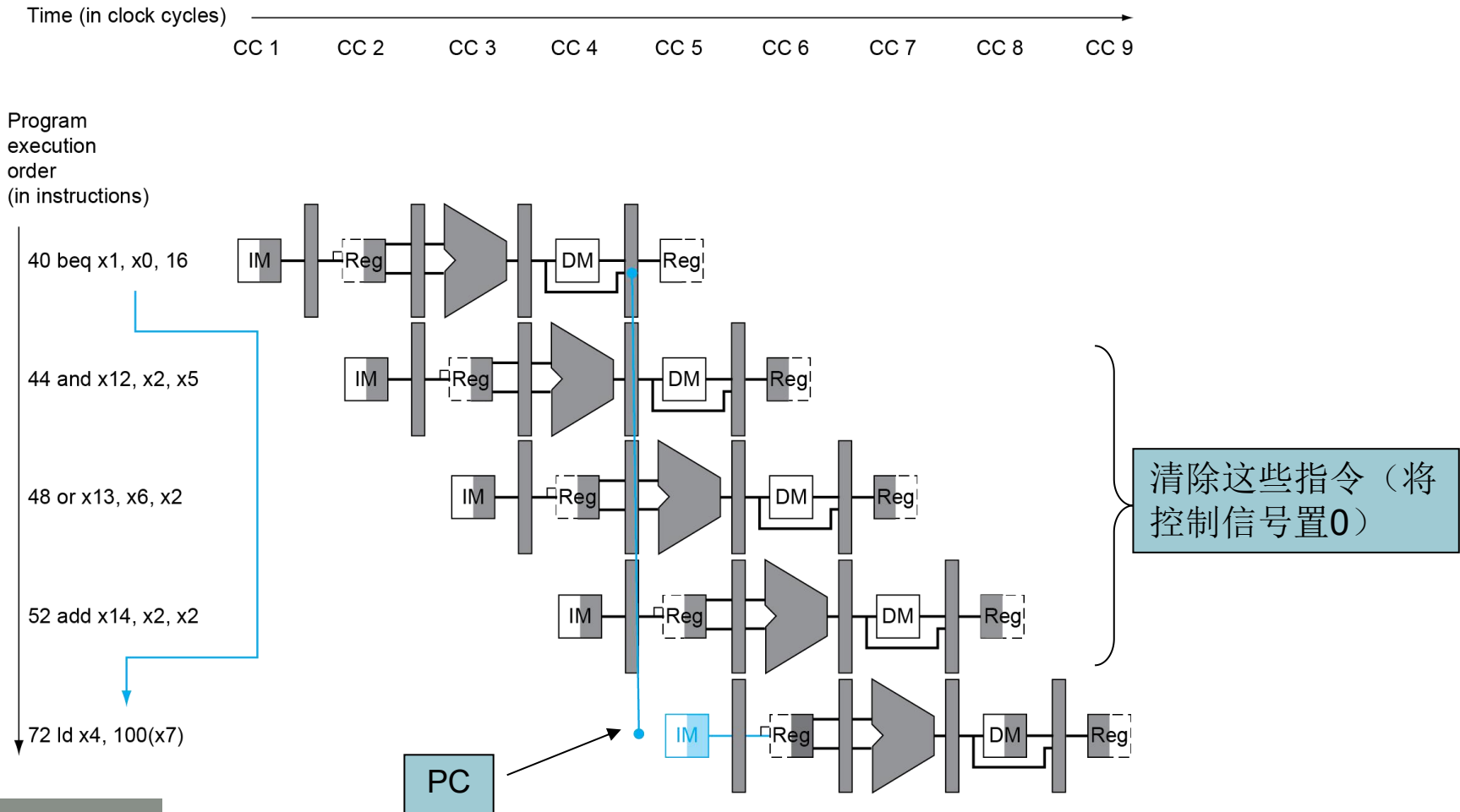
阻塞与性能

重点

- 阻塞降低性能
 - 然而需要阻塞来得到正确结果
- 编译器能调整代码顺序以避免冒险和阻塞
 - 需要了解流水线的构造

分支冒险

■ 如果分支结果在MEM级确定

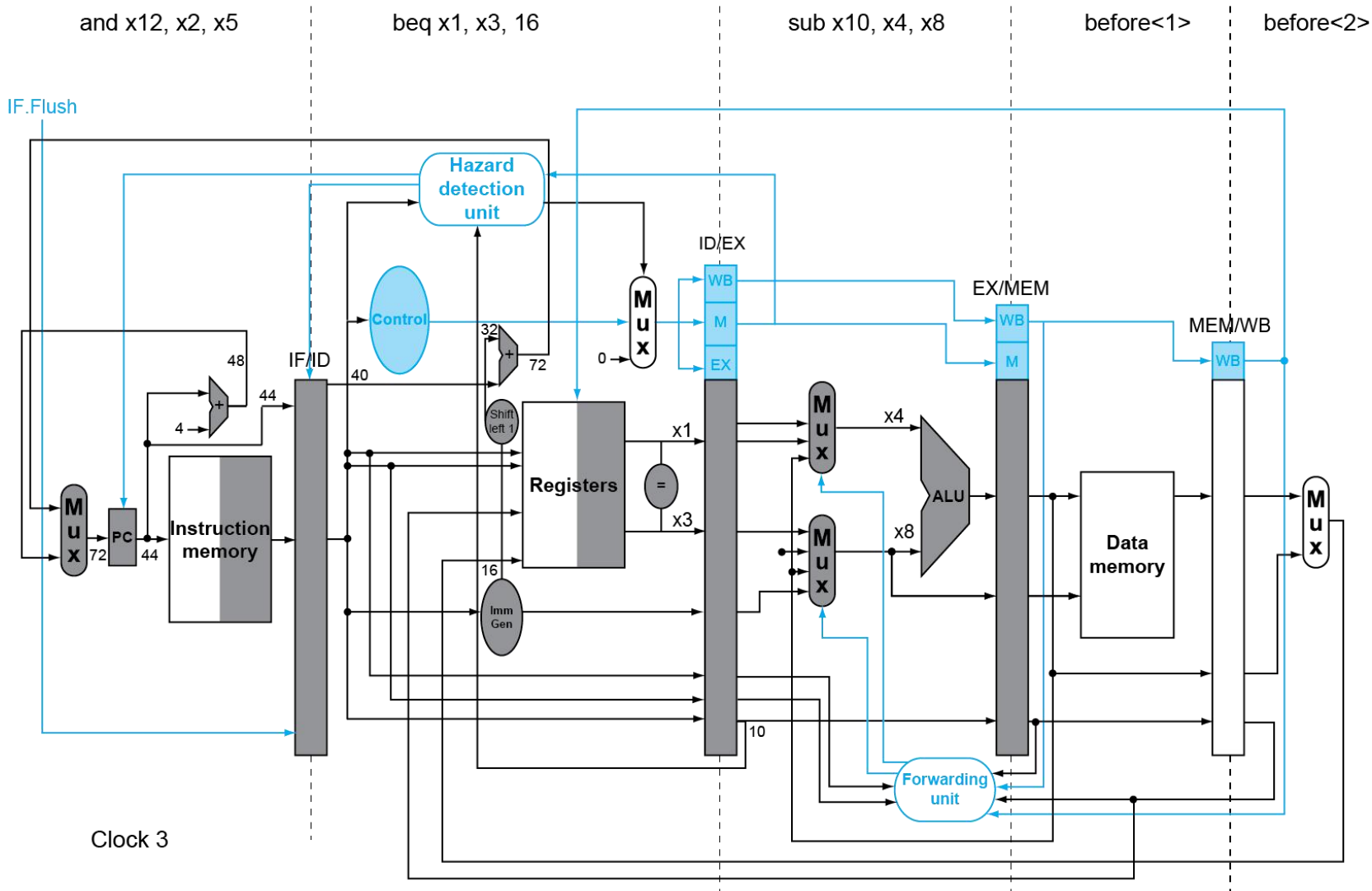


缩短分支的延迟

- 将用于确定结果的硬件移到ID级
 - 目标地址加法器
 - 寄存器比较器
- 例：分支发生

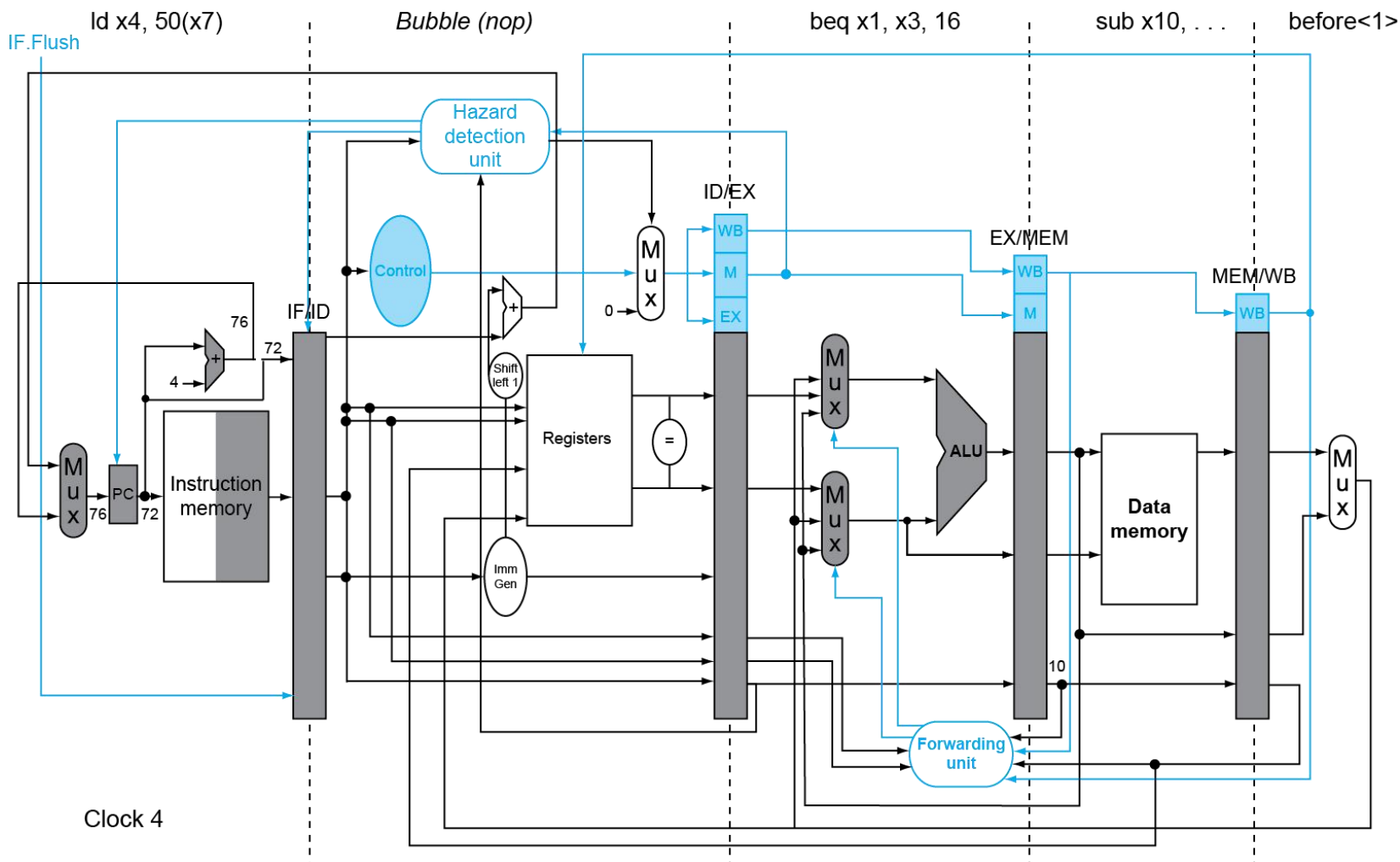
```
36:  sub    x10, x4, x8
40:  beq    x1,  x3, 16    # PC相对寻址: 40+16*2=72
44:  and    x12, x2, x5
48:  orr    x13, x2, x6
52:  add    x14, x4, x2
56:  sub    x15, x6, x7
    ...
72:  ld     x4, 50(x7)
```

例：分支发生



Clock 3

例：分支发生

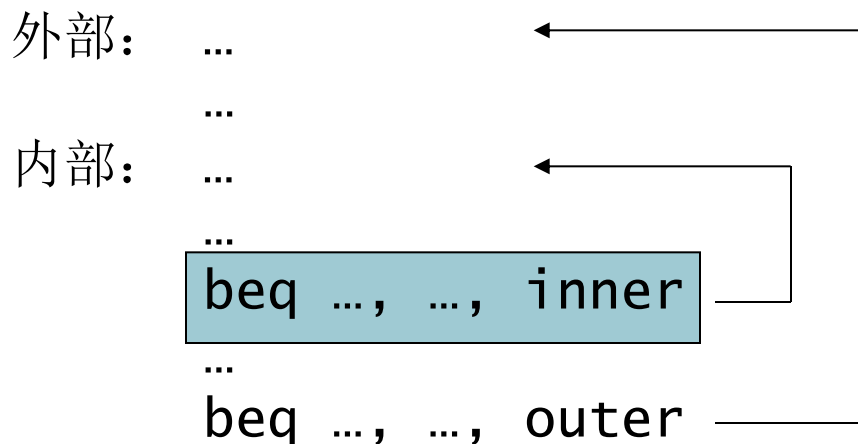


动态分支预测

- 在更深的流水线和超标量流水线中，分支开销更为显著
- 使用动态预测
 - 分支预测缓存（亦称分支历史记录表）
 - 按最近的分支指令地址索引
 - 保存分支结果（发生/未发生）
 - 为了执行分支指令
 - 查表，预计会有相同结果
 - 从不发生转移处或目标地址开始取指
 - 一旦预测错误，清除流水线并反转预测

1位预测：缺点

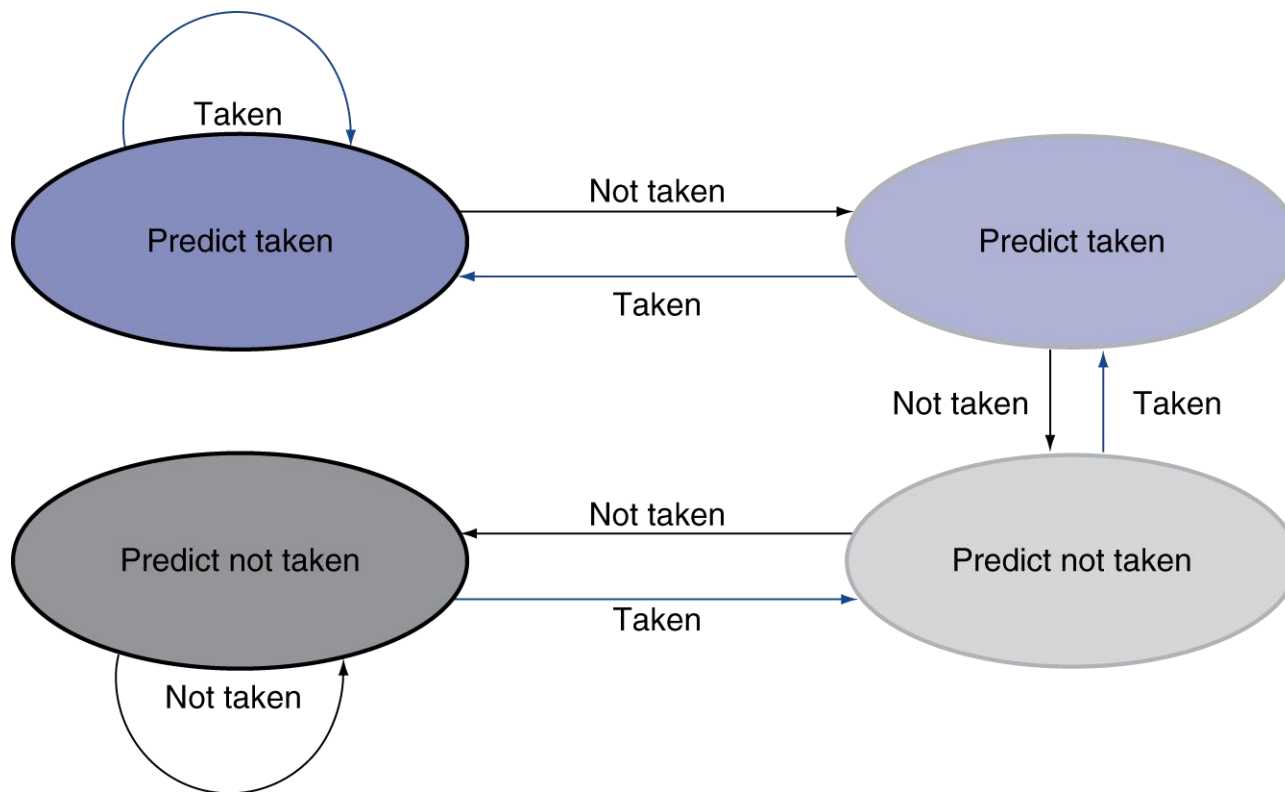
- 内循环分支被误预测两次！



- 在内循环的最后一次迭代时误预测为发生分支
- 然后在下一轮内循环的第一次迭代时误预测为不发生分支

2位预测

- 仅在连续两次预测错误时改变



计算分支目标

- 尽管做了预测，仍需计算目标地址
 - 分支发生则产生1个周期的开销
- 分支目标缓存
 - 分支地址的cache
 - 按取指令时的PC索引
 - 如果cache命中且指令被预测为分支发生，可立即取到目标地址

异常和中断

- 需要改变控制流的“意外”事件
 - 不同的ISA用的术语不一样
- 异常
 - 出现在CPU内部
 - 例如，未定义的操作码、系统调用等等
- 中断
 - 来自外部I/O控制器
- 难以做到处理异常和中断而不牺牲性能

处理异常

- 保存导致异常（或被中断）的指令的PC值
 - RISC-V使用SEPC（Supervisor Exception Program Counter，管理员异常程序计数器）
- 保存问题的表征
 - RISC-V使用SCAUSE（Supervisor Exception Cause Register，管理员异常原因寄存器）
 - 64位，但多数位没有用到
 - 异常编码字段：2为未定义的操作码，12为硬件故障，...
- 跳转到处理程序
 - 假定程序位于0000 0000 1C09 0000_{hex}

另一种机制

- 向量化中断
 - 处理程序的地址由中断原因决定
- 向量表基地址寄存器加上异常向量地址：
 - 未定义的操作码: $00\ 0100\ 0000_{two}$
 - 硬件故障: $01\ 1000\ 0000_{two}$
 - ... :
- 处理程序的功能为
 - 处理中断,
 - 或是跳转到实际的处理程序

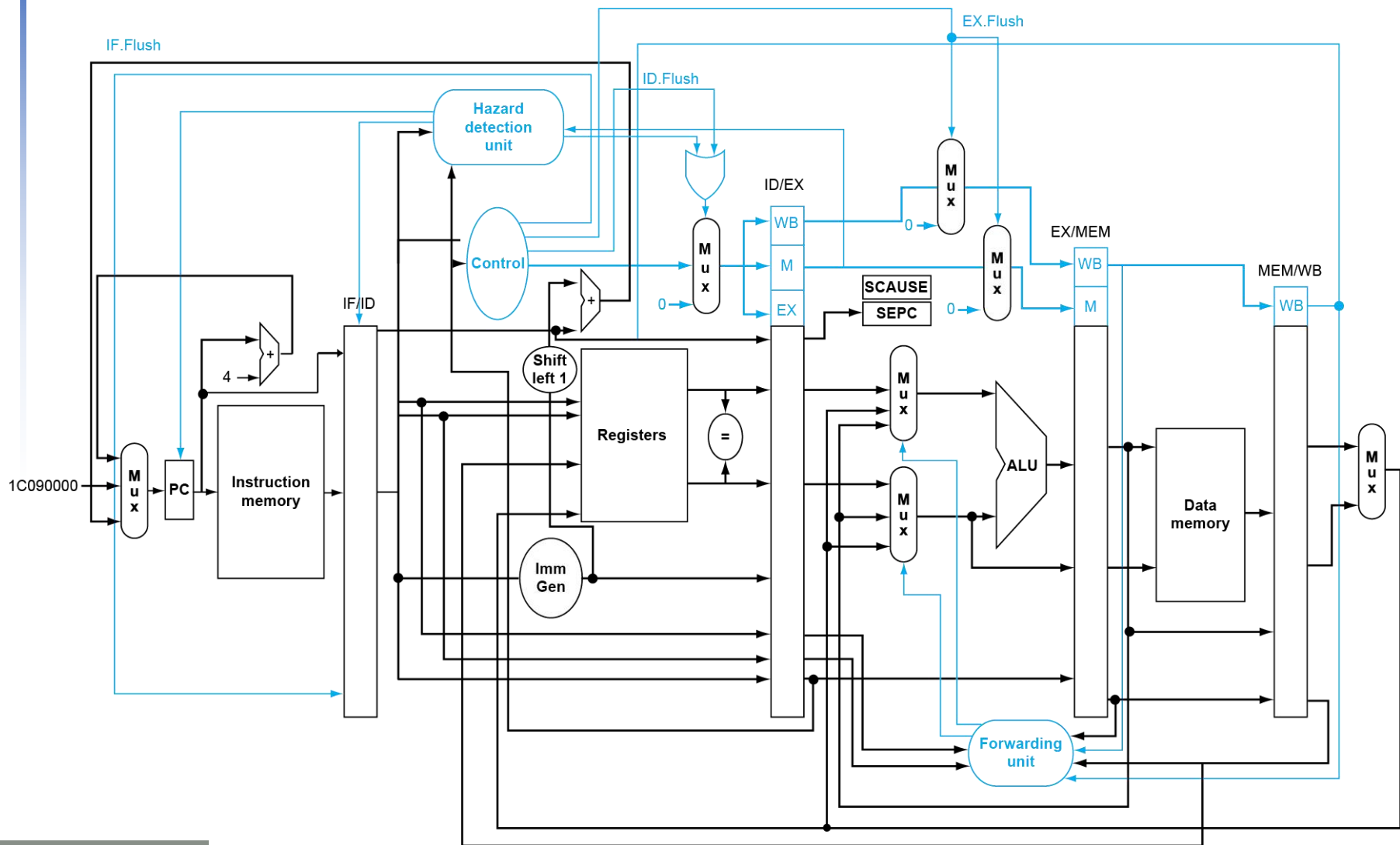
处理程序的动作

- 读取异常原因，并转移到相应的处理程序
- 决定需要采取的动作
- 如果可重启
 - 执行纠正动作
 - 使用**SEPC**来返回原先的程序
- 否则
 - 终止原先的程序
 - 使用**SEPC**和**SCAUSE**等报错

流水线中的异常

- 另一种形式的控制冒险
- 考虑add在EX级发生的故障
add x1, x2, x1
 - 防止x1被错误赋值
 - 完成先前的指令
 - 清除add及之后的指令
 - 设置SEPC和SCAUSE寄存器的值
 - 把控制转移到处理程序
- 与误预测的分支类似
 - 用到的硬件多数相同

带异常处理的流水线



异常的属性

- 可重启的异常
 - 流水线可以清除指令
 - 执行处理程序，然后回到该指令
 - 重新取指并从头执行
- 保存在**SEPC**寄存器中的**PC**
 - 标示导致异常的指令

异常的例子

- 在以下代码中`add`产生异常

```
40      sub    x11, x2, x4
44      and    x12, x2, x5
48      orr    x13, x2, x6
4c      add    x1,  x2, x1
50      sub    x15, x6, x7
54      ld     x16, 100(x7)
```

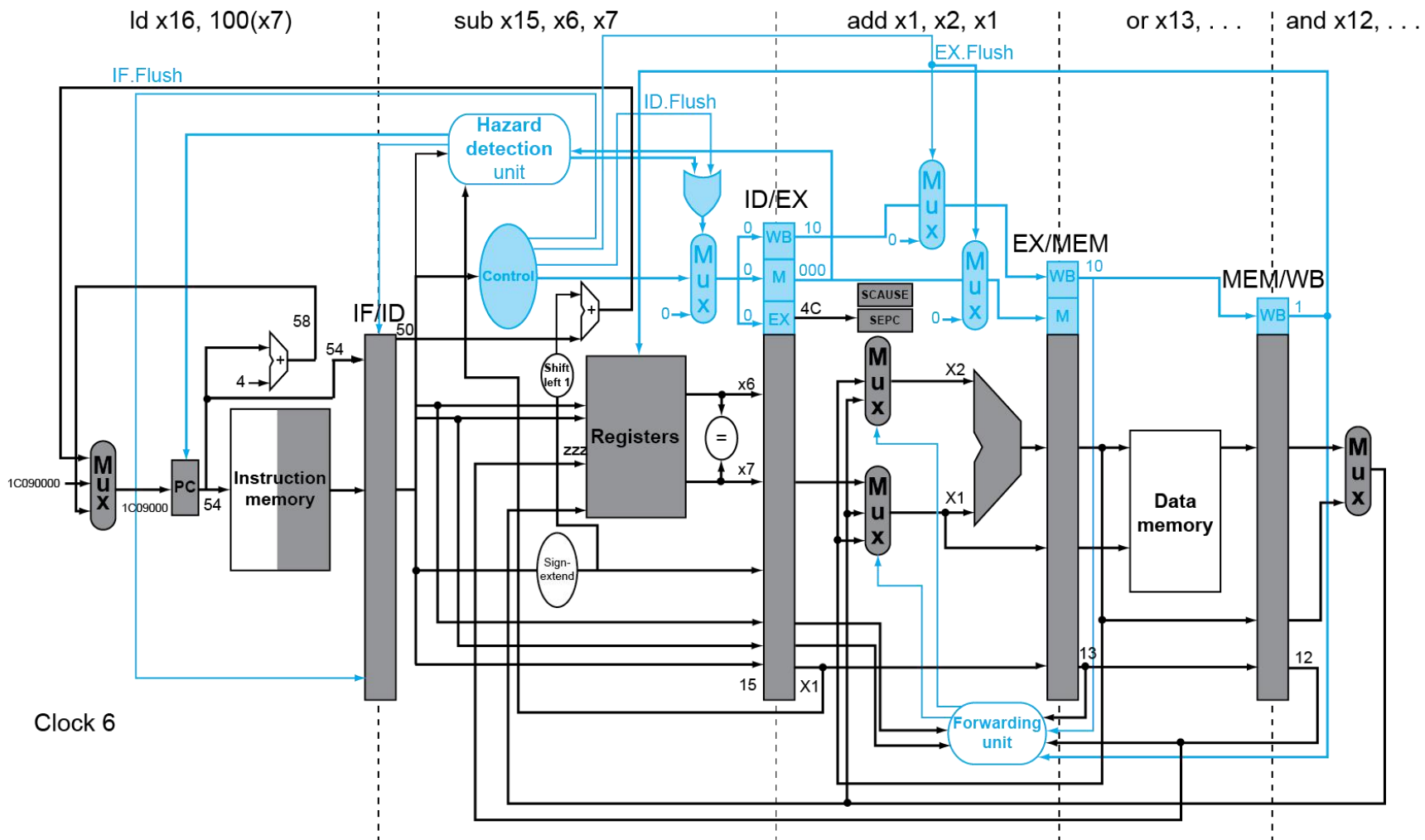
...

- 处理程序

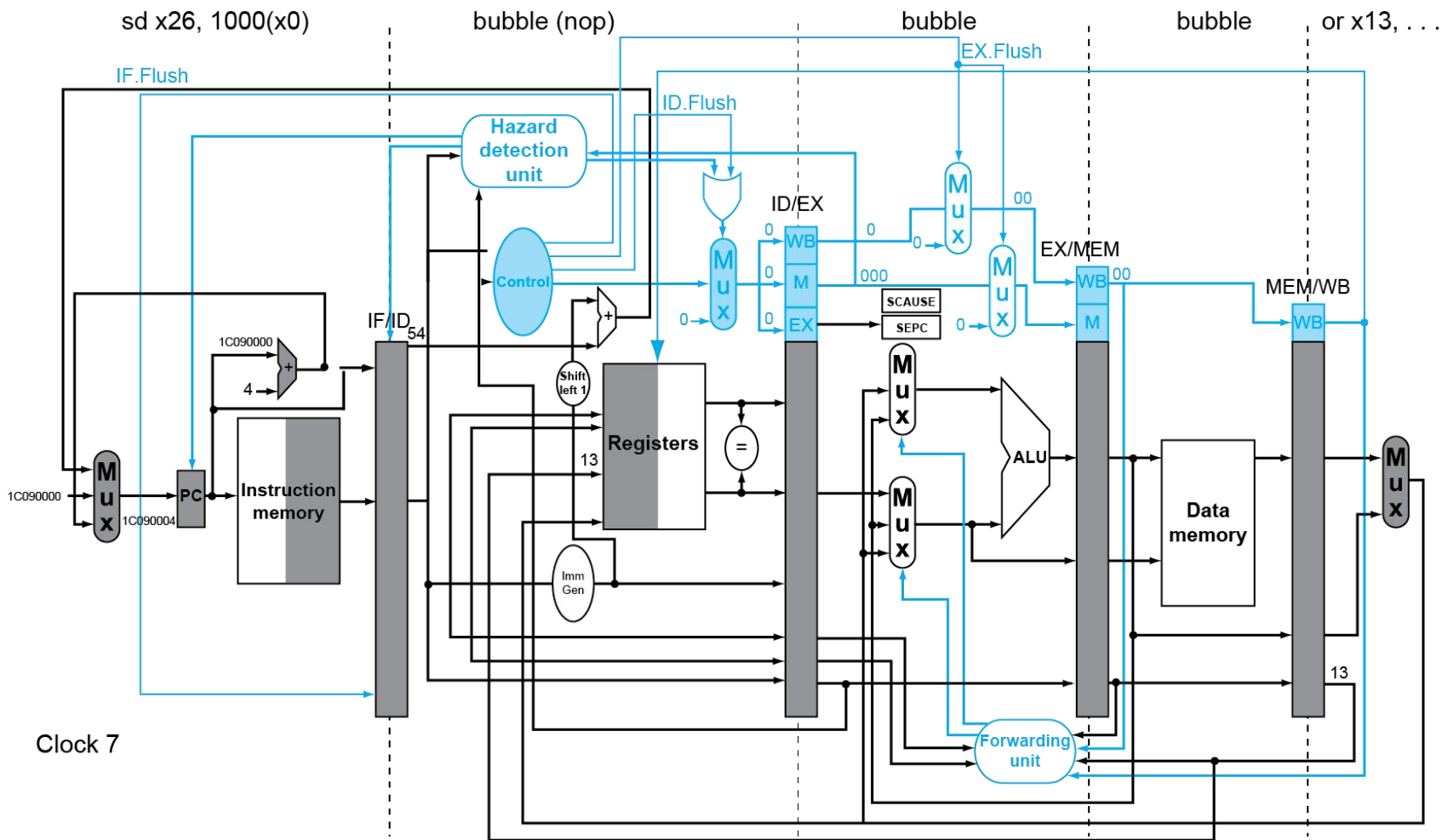
```
1c090000      sd    x26, 1000(x10)
1c090004      sd    x27, 1008(x10)
```

...

异常的例子



异常的例子



多重异常

- 流水线将多条指令重叠
 - 可能同时产生多个异常
- 简单办法：处理来自最早指令的异常
 - 清除接下来的指令
 - “精确”异常
- 在复杂的流水线中
 - 每周期发射多条指令
 - 乱序完成
 - 维持精确异常是困难的！

非精确异常

- 只是停止流水线并保存状态
 - 包括异常的原因
- 让处理程序来确定
 - 哪些指令产生了异常
 - 哪些指令要完成或被清除
 - 可能需要“手动”完成
- 简化了硬件，但使处理程序的软件更复杂
- 对复杂的多发射乱序流水线来说是不可行的

指令级并行(ILP)

- 流水线化：并行执行多条指令
- 为了提高ILP
 - 更深的流水线
 - 每级的工作量更少 $\Rightarrow$ 时钟周期更短
 - 多发射
 - 复制流水级 $\Rightarrow$ 多条流水线
 - 每个时钟周期启动多条指令
 - $CPI < 1$ ，所以使用IPC（每周期执行的指令数）指标
 - 例如，4GHz 4路多发射
 - 16 BIPS，峰值 $CPI = 0.25$ ，峰值 $IPC = 4$
 - 但在实践中依赖关系会降低IPC

多发射

- 静态多发射
 - 编译器将指令分组并同时发射
 - 将指令封装进“发射槽”
 - 编译器检测并避免冒险
- 动态多发射
 - CPU检验指令流并选择每个周期发射的指令
 - 编译器通过重排指令可有所帮助
 - CPU使用先进技术在运行期解决冒险

推测

- “猜” 怎样处理一条指令
 - 尽快开始操作
 - 检查猜测是否正确
 - 如果对了，完成操作
 - 如果错了，回滚并做出正确操作
- 对静态和动态多发射都通用
- 例
 - 推测分支结果
 - 如果走了另一条路径则回滚
 - 取数时推测
 - 如果该位置的数据被更新则回滚

编译器/硬件推测

- 编译器能够重排指令
 - 例如，将取数移到分支前面
 - 可以包含“修正”指令来从错误的推测中恢复
- 硬件可以前瞻要执行的指令
 - 先缓存结果，直至确定这些结果实际被用到
 - 推测错误则清除缓存

推测和异常

- 如果经推测而执行的指令产生异常该怎么办？
 - 例如，在进行空指针检查之前推测要取数
- 静态推测
 - 可以增加**ISA**对延后异常的支持
- 动态推测
 - 可以在指令完成之前缓存异常（异常可能不发生）

静态多发射

- 编译器将指令分组为“发射包”
 - 一组能在一个周期内被发射的指令
 - 由所需流水线资源确定
- 把一个发射包想成一条超长指令
 - 指定多个同时进行的操作
 - $\Rightarrow$ 超长指令字(VLIW)

调度静态多发射

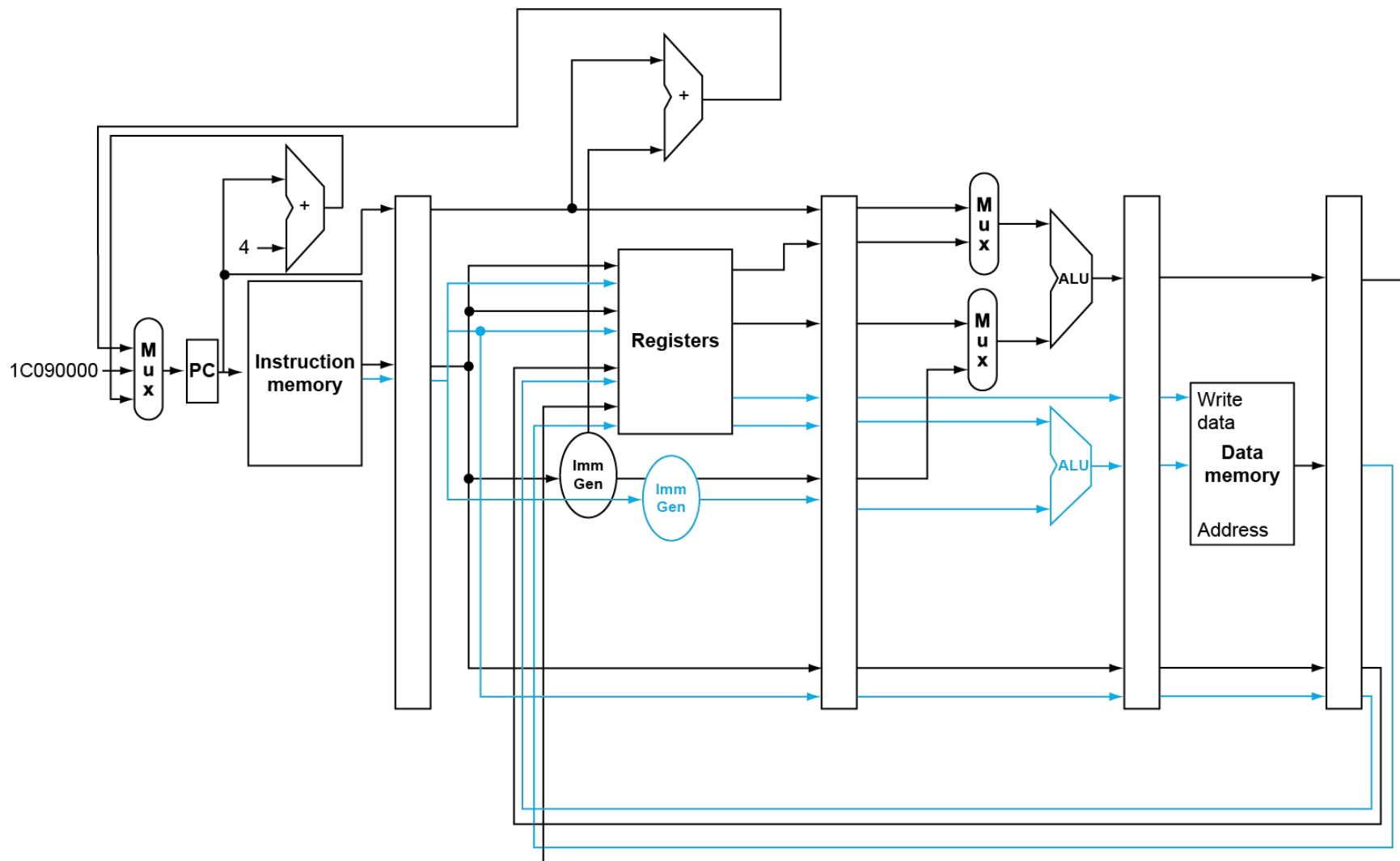
- 编译器必须消除部分/全部冒险
 - 将指令重排进发射包
 - 包中的指令之间没有依赖关系
 - 可能在包与包之间有依赖关系
 - 视ISA而不同；编译器必须了解！
 - 必要时用nop填充

带静态双发射的RISC-V

- 双发射包
 - 一条ALU/分支指令
 - 一条取数/存数指令
 - 按64位对齐
 - ALU/分支指令在前，取数/存数指令在后
 - 用nop填充未使用的指令

地址	指令类型	流水线阶段						
n	ALU/分支	IF	ID	EX	MEM	WB		
n + 4	取数/存数	IF	ID	EX	MEM	WB		
n + 8	ALU/分支		IF	ID	EX	MEM	WB	
n + 12	取数/存数		IF	ID	EX	MEM	WB	
n + 16	ALU/分支			IF	ID	EX	MEM	WB
n + 20	取数/存数			IF	ID	EX	MEM	WB

带静态双发射的RISC-V



双发射RISC-V中的冒险

- 并行执行更多指令
- EX数据冒险
 - 旁路可避免单发射的阻塞
 - 现在在同一个发射包中的取数/存数指令无法使用ALU结果
 - `add x10, x0, x1`
`ld x2, 0(x10)`
 - 分成两个包，实际上是一次阻塞
- 取数-使用型冒险
 - 使用数据的延迟仍为一个周期，但现在是延迟两条指令
- 需要更积极的调度方式

调度的例子

■ 为双发射RISC-V调度以下指令

```
Loop: ld    x31,0(x20)      # x31=array element
      add   x31,x31,x21     # add scalar in x21
      sd    x31,0(x20)     # store result
      addi  x20,x20,-8      # decrement pointer
      blt   x22,x20,Loop    # branch if x22 < x20
```

	ALU/分支	取数/存数	周期
Loop:	nop	ld x31,0(x20)	1
	addi x20,x20,-8	nop	2
	add x31,x31,x21	nop	3
	blt x22,x20,Loop	sd x31,8(x20)	4

■ $IPC = 5/4 = 1.25$ （对比峰值 $IPC = 2$ ）

循环展开

- 复制循环体以展现更高的并行度
 - 减小循环控制的开销
- 每份复制使用不同的寄存器
 - 称作“寄存器重命名”
 - 避免循环带来的“反相关”
 - 存数后跟着同一寄存器的取数
 - 亦称“名字相关”
 - 重用某个寄存器名称

循环展开的例子

	ALU/分支	取数/存数	周期
Loop:	addi x20,x20,-32	ld x28, 0(x20)	1
	nop	ld x29, 24(x20)	2
	add x28,x28,x21	ld x30, 16(x20)	3
	add x29,x29,x21	ld x31, 8(x20)	4
	add x30,x30,x21	sd x28, 32(x20)	5
	add x31,x31,x21	sd x29, 24(x20)	6
	nop	sd x30, 16(x20)	7
	blt x22,x20,Loop	sd x31, 8(x20)	8

- $IPC = 14/8 = 1.75$
 - 更接近2，但代价是更多的寄存器和更长的代码

动态多发射

- “超标量” 处理器
- CPU决定每周期发射0条、1条，还是多条指令
 - 避免结构冒险和数据冒险
- 避免了对编译器调度的需求
 - 尽管编译器可能仍有帮助
 - 由CPU保证代码的语义

动态流水线调度

- 允许CPU通过乱序执行指令来避免阻塞
 - 不过要按顺序将结果提交给寄存器
- 例

ld x31, 20(x21)

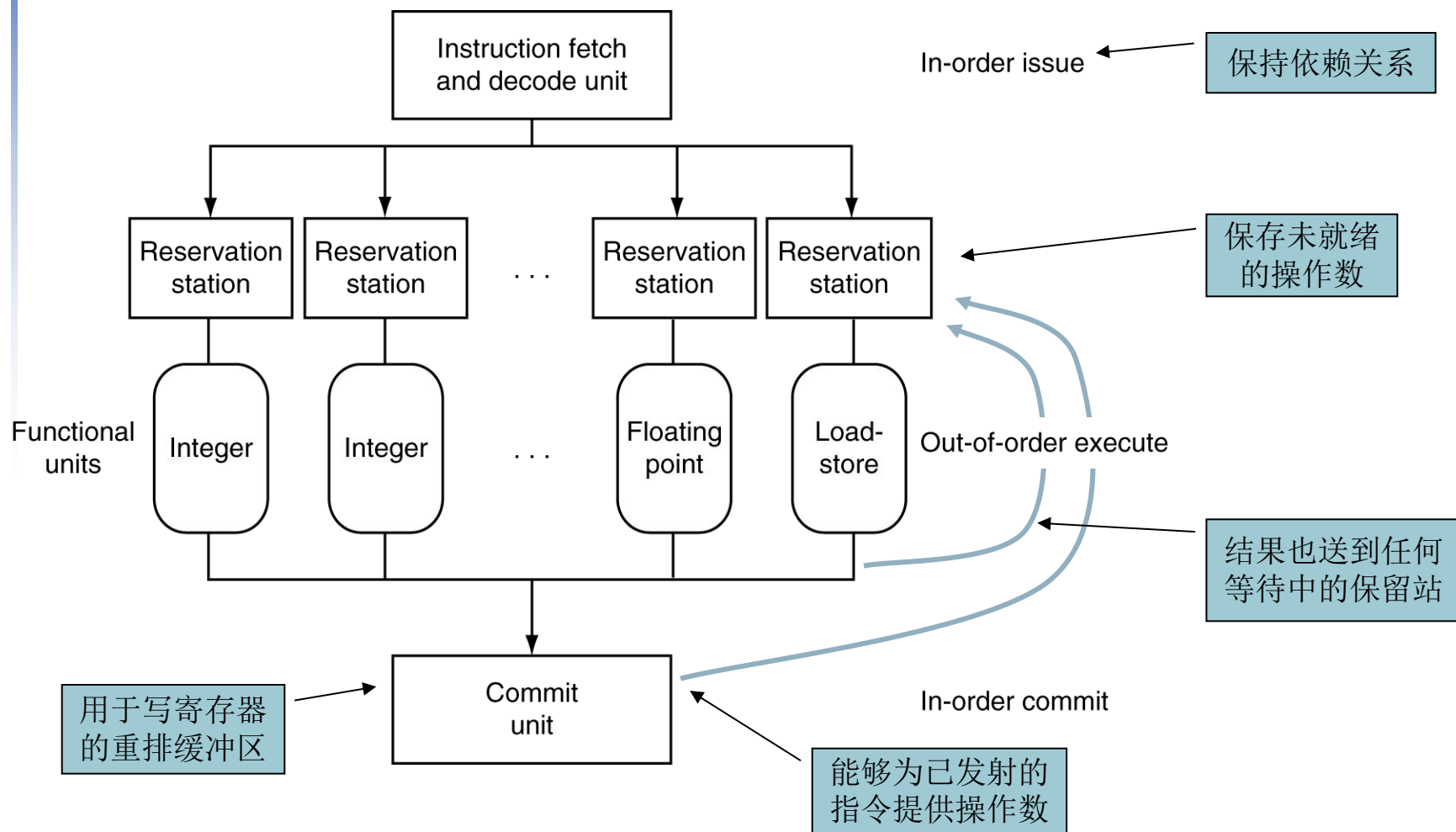
add x1, x31, x2

sub x23, x23, x3

andi x5, x23, 20

- 可以在add等待ld时开始sub

动态调度的CPU



寄存器重命名

- 保留站和重排序缓冲区实际上提供了寄存器重命名
- 在指令被发送到保留站时
 - 如果操作数在寄存器文件或重排缓冲区中可用
 - 复制到保留站
 - 不再需要保存在寄存器中，可以被覆盖
 - 如果操作数还不可用
 - 将由某个功能单元把操作数提供给保留站
 - 可能不需要更新寄存器

推测

- 预测分支并继续发射
 - 在分支结果确定之前不提交
- 取数推测
 - 避免取数延迟和**cache**缺失延迟
 - 预测有效地址
 - 预测取到的值
 - 在存数操作完成前取数
 - 将保存的值转发给取数单元
 - 在推测结果确定之前不提交取数操作

为什么要进行动态调度？

- 为什么不只是让编译器来调度代码？
- 不是所有阻塞都可预测
 - 例如，**cache**缺失
- 无法总是对分支做静态调度
 - 分支结果是动态确定的
- 同一**ISA**的不同实现有着不同的延迟和冒险

多发射灵不灵？

重点

- 有用，但没有想象的那么好
- 程序中实际的依赖关系限制了ILP
- 有些依赖关系难以消除
 - 例如，指针别名
- 有些并行性难以展现
 - 发射指令期间的窗口大小有限
- 存储器延迟及有限的带宽
 - 难以保持流水线满载
- 推测做得好会有助益

功耗效率

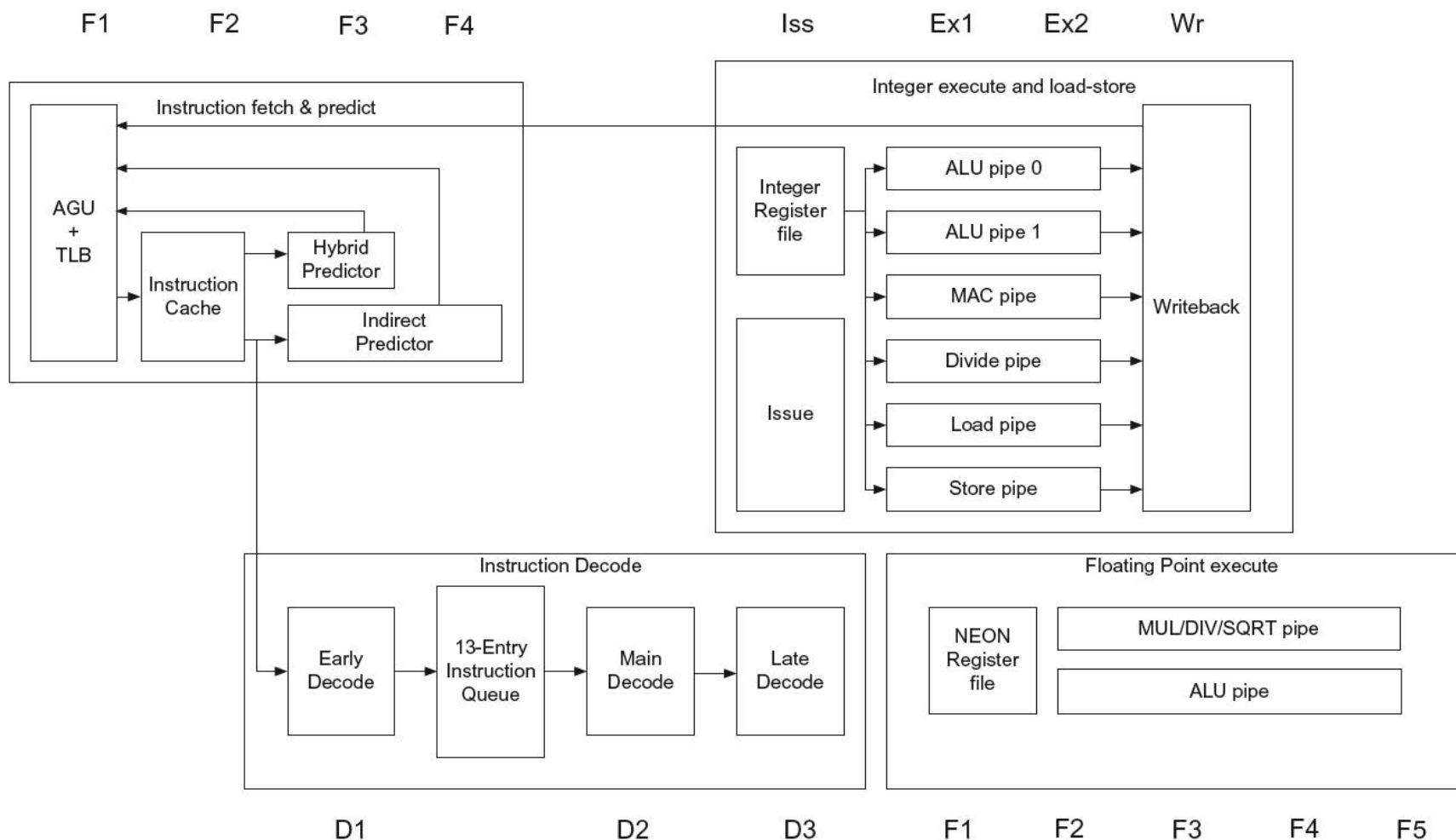
- 动态调度和推测的复杂性需要功耗
- 多个更简单的处理器核也许更好

微处理器	年份	时钟频率	流水线级数	发射宽度	乱序/推测	核数	功耗
i486	1989	25MHz	5	1	No	1	5W
Pentium	1993	66MHz	5	2	No	1	10W
Pentium Pro	1997	200MHz	10	3	Yes	1	29W
P4 Willamette	2001	2000MHz	22	3	Yes	1	75W
P4 Prescott	2004	3600MHz	31	3	Yes	1	103W
Core	2006	2930MHz	14	4	Yes	2	75W
UltraSparc III	2003	1950MHz	14	4	No	1	90W
UltraSparc T1	2005	1200MHz	6	1	No	8	70W

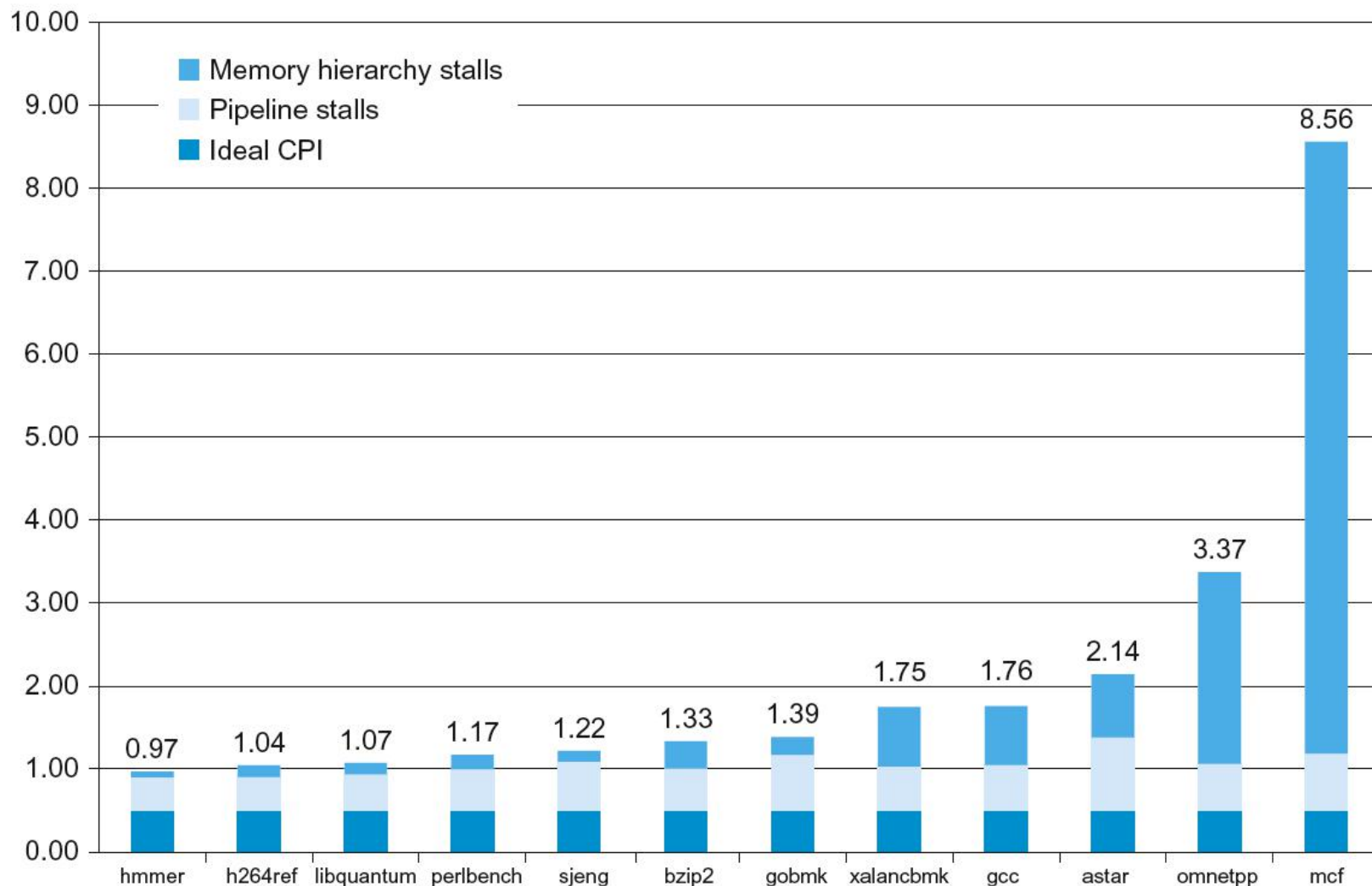
Cortex A53与Intel i7

Processor	ARM A53	Intel Core i7 920
Market	Personal Mobile Device	Server, cloud
Thermal design power	100 milliWatts (1 core @ 1 GHz)	130 Watts
Clock rate	1.5 GHz	2.66 GHz
Cores/Chip	4 (configurable)	4
Floating point?	Yes	Yes
Multiple issue?	Dynamic	Dynamic
Peak instructions/clock cycle	2	4
Pipeline stages	8	14
Pipeline schedule	Static in-order	Dynamic out-of-order with speculation
Branch prediction	Hybrid	2-level
1 st level caches/core	16-64 KiB I, 16-64 KiB D	32 KiB I, 32 KiB D
2 nd level caches/core	128-2048 KiB	256 KiB (per core)
3 rd level caches (shared)	(platform dependent)	2-8 MB

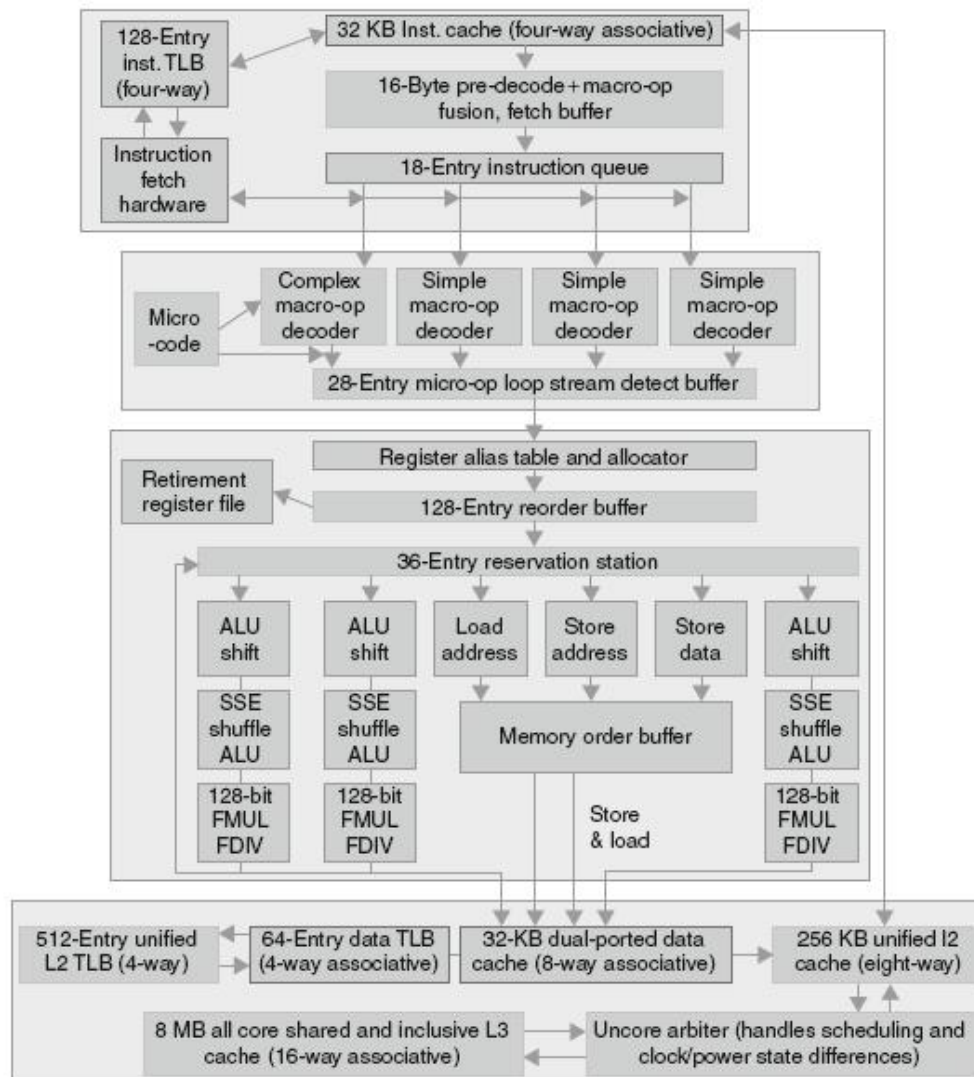
ARM Cortex-A53的流水线



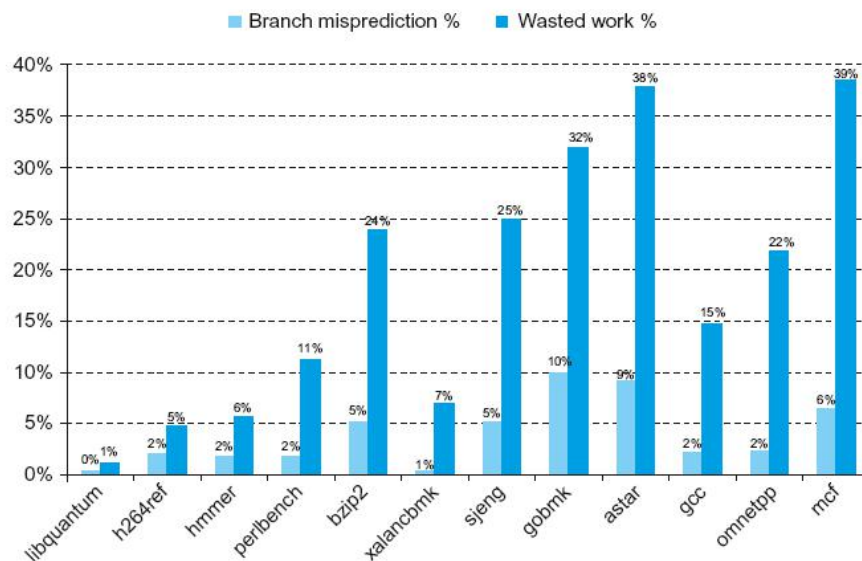
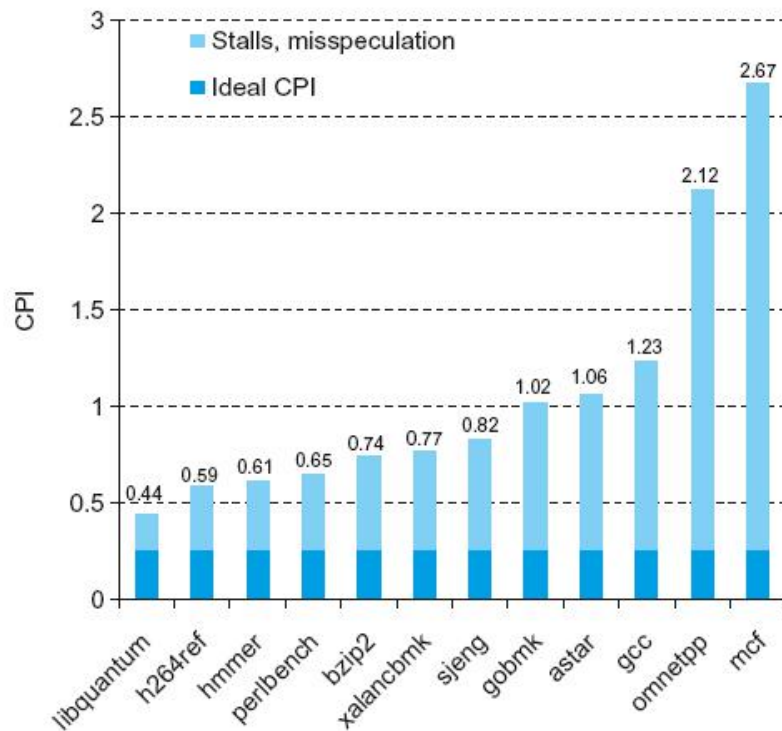
ARM Cortex-A53的性能



Core i7的流水线



Core i7的性能



矩阵乘法

■ 循环展开版本的C代码

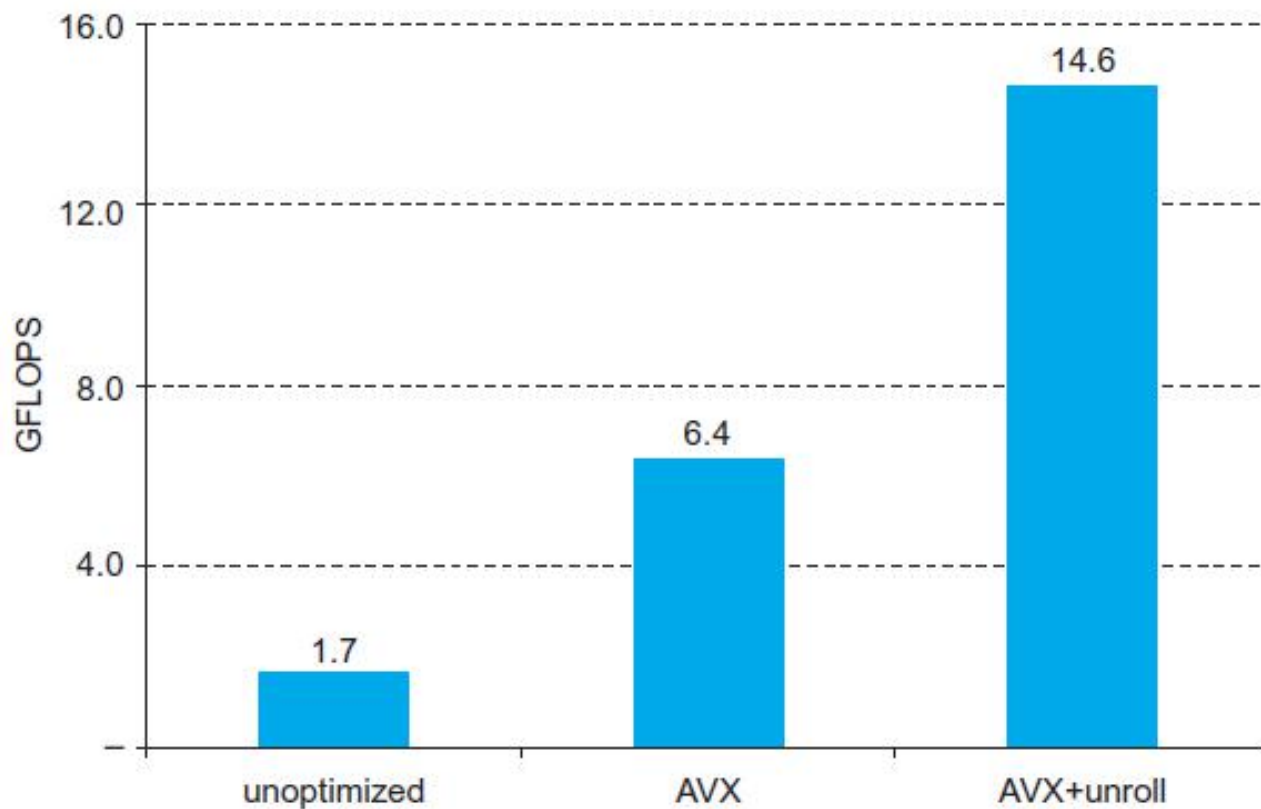
```
1 #include <x86intrin.h>
2 #define UNROLL (4)
3
4 void dgemm (int n, double* A, double* B, double* C)
5 {
6     for ( int i = 0; i < n; i+=UNROLL*4 )
7         for ( int j = 0; j < n; j++ ) {
8             __m256d c[4];
9             for ( int x = 0; x < UNROLL; x++ )
10                 c[x] = _mm256_load_pd(C+i*x*4+j*n);
11
12             for( int k = 0; k < n; k++ )
13             {
14                 __m256d b = _mm256_broadcast_sd(B+k+j*n);
15                 for (int x = 0; x < UNROLL; x++)
16                     c[x] = _mm256_add_pd(c[x],
17                                         _mm256_mul_pd(_mm256_load_pd(A+n*k+x*4+i), b));
18             }
19
20             for ( int x = 0; x < UNROLL; x++ )
21                 _mm256_store_pd(C+i*x*4+j*n, c[x]);
22         }
23 }
```

矩阵乘法

■ 汇编代码:

```
1 vmovapd (%r11),%ymm4          # Load 4 elements of C into %ymm4
2 mov %rbx,%rax                 # register %rax = %rbx
3 xor %ecx,%ecx                 # register %ecx = 0
4 vmovapd 0x20(%r11),%ymm3      # Load 4 elements of C into %ymm3
5 vmovapd 0x40(%r11),%ymm2      # Load 4 elements of C into %ymm2
6 vmovapd 0x60(%r11),%ymm1      # Load 4 elements of C into %ymm1
7 vbroadcastsd (%rcx,%r9,1),%ymm0 # Make 4 copies of B element
8 add $0x8,%rcx # register %rcx = %rcx + 8
9 vmulpd (%rax),%ymm0,%ymm5     # Parallel mul %ymm1,4 A elements
10 vaddpd %ymm5,%ymm4,%ymm4     # Parallel add %ymm5, %ymm4
11 vmulpd 0x20(%rax),%ymm0,%ymm5 # Parallel mul %ymm1,4 A elements
12 vaddpd %ymm5,%ymm3,%ymm3     # Parallel add %ymm5, %ymm3
13 vmulpd 0x40(%rax),%ymm0,%ymm5 # Parallel mul %ymm1,4 A elements
14 vmulpd 0x60(%rax),%ymm0,%ymm0 # Parallel mul %ymm1,4 A elements
15 add %r8,%rax                 # register %rax = %rax + %r8
16 cmp %r10,%rcx               # compare %r8 to %rax
17 vaddpd %ymm5,%ymm2,%ymm2     # Parallel add %ymm5, %ymm2
18 vaddpd %ymm0,%ymm1,%ymm1     # Parallel add %ymm0, %ymm1
19 jne 68 <dgemm+0x68>         # jump if not %r8 != %rax
20 add $0x1,%esi               # register % esi = % esi + 1
21 vmovapd %ymm4, (%r11)       # Store %ymm4 into 4 C elements
22 vmovapd %ymm3,0x20(%r11)    # Store %ymm3 into 4 C elements
23 vmovapd %ymm2,0x40(%r11)    # Store %ymm2 into 4 C elements
24 vmovapd %ymm1,0x60(%r11)    # Store %ymm1 into 4 C elements
```

对性能的影响



谬误

- 流水线是简单的(!)
 - 基本思想确实简单
 - 知易行难
 - 例如，检测数据冒险
- 流水线与工艺无关
 - 那为什么我们一直没能用上流水线？
 - 晶体管数量的增多使更多先进技术变得可行
 - 与流水线相关的**ISA**设计要考虑技术趋势
 - 例如，指令预测

陷阱

- 糟糕的ISA设计会使流水线更难实现
 - 例如，复杂指令集(VAX、IA-32)
 - 流水线工作时的巨大开销
 - IA-32的微操作方式
 - 例如，复杂的寻址模式
 - 更新寄存器的副作用、间接访存
 - 例如，延迟分支
 - 先进的流水线有长的延迟槽

本章小结

- ISA影响数据通路和控制的设计
- 数据通路和控制亦影响ISA的设计
- 流水线化通过并行来提高指令的吞吐率
 - 每秒完成更多指令
 - 每条指令的延迟没有减少
- 冒险：结构、数据、控制
- 多发射和动态调度(ILP)
 - 依赖关系限制了可达到的并行度
 - 复杂性导致功耗墙